Thesis Plan

Administrative Details

Student name:	Kunwar Singh
Student number:	46978107
Program enrolment:	Bachelor of Engineering (Honours)
Course enrolment:	METR4912
Number of units:	4 units
Expected workload:	10 hours per week

Supervision Details

Primary Supervisor:	Dr Siamak Layeghy
Other Supervisors:	Dr Marius Portmann

Have you met your supervisor this semester?
Yes, in person. Most recent meeting: Monday 24 February 2026, 12:00pm in person on campus.

Do you have a regular meeting scheduled?
Weekly, every Monday at 12:00pm, in person on campus.

Thesis Plan (Thesis Draft)

The attached thesis skeleton follows the official UQ LaTeX template and contains the full structure of the final thesis, including all chapter and section headings, referenced background and related work, and a complete bibliography. Material from the previously submitted thesis proposal (METR4912, Semester 2 2025) has been reused and updated to reflect what was actually achieved rather than what was originally planned. In particular, the architecture evolved from an MCP-centric tool orchestration approach to a multi-agent LLM debate architecture, as motivated by experimental findings documented in the thesis.

Use of AI Statement

I acknowledge the use of generative AI tools in completing this assessment. Details of which tools were used and how they were used are provided in the table below, along with appropriate in-text and full references. I take responsibility for critically

evaluating and integrating the AI-generated content, and ensuring it adheres to academic integrity standards.

Have you used AI in your project?

- **To explore your topic?** Yes. Claude (Anthropic) and ChatGPT (OpenAI) were used to explore the landscape of LLM applications in cybersecurity, identify relevant sub-areas, and generate initial keywords for literature searches. All identified topics were independently verified through manual database searches.
- **To define your research question?** No. Research questions were defined through supervisor meetings, iterative refinement based on experimental results, and manual review of the literature. The pivot from MCP-centric to multi-agent architecture was driven by empirical findings.
- **To assist with your literature review?** Yes. AI assistants were used to summarise papers already located through manual database searches and to suggest thematic groupings. All references were independently verified against the original papers, and all claims attributed to cited works were checked for accuracy.
- **To assist with formatting?** Yes. AI assistants were used to convert content to LaTeX format, resolve compilation issues, and ensure consistent formatting with the UQ template. The intellectual content, structure, and arguments are the author's own.
- **Did you generate figures using AI?** No. All figures, diagrams, and charts were created manually using Python (matplotlib), TikZ, or the React-based dashboard.
- **Did you analyse data using AI?** Yes. AI assistants (Claude Code) were used to write Python scripts for metrics calculation, result aggregation, and cost analysis. All scripts were reviewed, and results were independently verified against raw experimental data.
- **Did you generate data using AI?** No. All data comes from the CICIDS2018 NetFlow v3 dataset, a publicly available benchmark. No synthetic data was generated.
- **Did you generate text for your thesis using AI?** Yes. AI assistants (Claude) were used to draft sections of this thesis. All AI-generated text was critically reviewed, edited for accuracy and argument coherence, fact-checked against experimental results and cited sources, and revised to reflect the author's own

analysis and conclusions. The intellectual structure, arguments, and interpretation of results are the author's own work.

- **For other aspects?** Yes. AI assistants (Claude Code, GitHub Copilot) were used extensively to write the AMATAS system code, including the multi-agent pipeline, MCP server, evaluation dashboard, and experimental scripts. All code was reviewed, tested, and debugged by the author. The architectural decisions (the six-agent design, Devil's Advocate mechanism, two-tier pre-filter, and consensus weighting) were made by the author based on experimental evidence and supervisor guidance. GPT-4o was also used as the underlying LLM within the AMATAS multi-agent pipeline itself (i.e. the six agents that analyse network flows are powered by GPT-4o API calls).

School of Electrical Engineering and Computer Science

Advanced Multi-Agent Threat Analysis System: Explainable Network Intrusion Detection through LLM-Based Multi-Agent Debate

by

Kunwar Singh

Submitted for the degree of
Bachelor of Engineering (Honours)
as part of the course METR4912.

April 2026

Brisbane, QLD

16th March 2026

Prof Michael Bruenig
Head of School
School of Electrical Engineering and Computer Science
The University of Queensland
St Lucia, Q 4072

Dear Professor Bruenig,

In accordance with the requirements of the Degree of Bachelor of Engineering (Honours) in the School of Electrical Engineering and Computer Science, I submit the following thesis entitled "Advanced Multi-Agent Threat Analysis System: Explainable Network Intrusion Detection through LLM-Based Multi-Agent Debate". The thesis was performed under the supervision of Dr Siamak Layeghy and Dr Marius Portmann. I declare that the work submitted in the thesis is my own, except as acknowledged in the text and footnotes, and that it has not previously been submitted for a degree at the University of Queensland or any other institution.

Yours sincerely,

Kunwar Singh

Declaration by Author

This thesis is composed of my original work, and contains no material previously published, written by another person, or generated by artificial intelligence (AI), except where clearly referenced and acknowledged either in the main text or the preliminary pages of this thesis. I have clearly stated the contribution of others to jointly authored works that I have included in my thesis.

I have clearly stated the contribution of others to my thesis as a whole, including statistical assistance, survey design, data analysis, significant technical procedures, professional editorial advice, use of AI tools, financial support, and any other original research work used or reported in my thesis. The content of my thesis is the result of work I have carried out since the commencement of my enrollment in the course code indicated on the title page, and does not include a substantial part of work that has been submitted to qualify for the award of any other degree or diploma in any university or other tertiary institution. I have clearly stated which parts of my thesis, if any, have been submitted to qualify for another award.

I acknowledge that copyright of all material contained in my thesis resides with the copyright holder(s) of that material. Where appropriate I have obtained copyright permission from the copyright holder to reproduce material in this thesis and have sought permission from co-authors for any jointly authored works included in the thesis.

Use of AI Statement

I acknowledge the use of generative AI tools in completing this thesis. Details of which tools were used and how they were used are provided in the table below, along with appropriate in-text and full references. I take responsibility for critically evaluating and integrating the AI-generated content, and ensuring it adheres to academic integrity standards.

How AI was used in this project:

- **Topic exploration:** Yes. Claude (Anthropic) and ChatGPT (OpenAI) were used to explore the landscape of LLM applications in cybersecurity, identify relevant sub-areas, and generate initial keywords for literature searches. All identified topics were independently verified through manual database searches.
- **Research question definition:** No. Research questions were defined through supervisor meetings, iterative refinement based on experimental results, and manual review of the literature. The pivot from MCP-centric to multi-agent architecture was driven by empirical findings, not AI suggestions.
- **Literature review:** Yes. AI assistants were used to summarise papers already located through manual database searches and to suggest thematic groupings. All references were independently verified against the original papers, and all claims attributed to cited works were checked for accuracy.
- **Figure generation:** No. All figures, diagrams, and charts were created manually using Python (matplotlib), TikZ, or the React-based dashboard.
- **Data analysis:** Yes. AI assistants (Claude Code) were used to write Python scripts for metrics calculation, result aggregation, and cost analysis. All scripts were reviewed, and results were independently verified against raw data.
- **Data generation:** No. All data comes from the CICIDS2018 NetFlow v3 dataset, a publicly available benchmark. No synthetic data was generated.
- **Text generation:** Yes. AI assistants (Claude) were used to draft sections of this thesis. All AI-generated text was critically reviewed, edited for accuracy and

argument coherence, fact-checked against experimental results and cited sources, and revised to reflect the author's own analysis and conclusions. The intellectual structure, arguments, and interpretation of results are the author's own work.

- **Formatting:** Yes. AI assistants were used to convert content to LaTeX format, resolve compilation issues, and ensure consistent formatting. The intellectual content, structure, and arguments are the author's own.

- **Code generation:** Yes. AI assistants (Claude Code, GitHub Copilot) were used extensively to write the AMATAS system code, including the multi-agent pipeline, MCP server, dashboard, and evaluation scripts. All code was reviewed, tested, and debugged by the author. The architectural decisions (the six-agent design, Devil's Advocate mechanism, two-tier pre-filter, and consensus weighting) were made by the author based on experimental evidence and supervisor guidance.

AI Model & date	Lang. Transl.	Grammar/style	Topic expl.	Research Qs	Lit. review	Data analysis	Gen. data	Gen. text	Visual content	Code	Feedback	Other
Claude 3.5/4 (Anthropic) Jul '25–Apr '26		✓	✓		✓	✓		✓		✓	✓	
GPT-4o (OpenAI) Jul '25–Apr '26			✓									AMATAS agent
GitHub Copilot Jul '25–Apr '26										✓		

Acknowledgements

I would like to thank my supervisors, Dr Siamak Layeghy and Dr Marius Portmann, for their guidance throughout this project. Their expertise in network security and machine learning informed the architectural decisions that shaped AMATAS, and their feedback on experimental methodology ensured the rigour of the evaluation.

Abstract

This thesis presents AMATAS (Advanced Multi-Agent Threat Analysis System), a network intrusion detection architecture that combines a machine learning pre-filter with a six-agent large language model pipeline to deliver both accurate detection and transparent reasoning. Unlike traditional classifiers that produce opaque probability outputs, AMATAS generates multi-perspective reasoning chains from specialist agents analysing protocol conformance, statistical anomalies, behavioural signatures, and temporal patterns, with an adversarial Devil's Advocate mechanism for false positive control.

Evaluated across all fourteen attack types in the CICIDS2018 NetFlow v3 dataset at realistic class distributions (five per cent attack prevalence), AMATAS achieves a mean F1 score of 92.9 per cent across thirteen detectable attack categories, with a false positive rate of zero per cent in ten of fourteen experiments. A two-tier architecture combining a Random Forest pre-filter with the LLM pipeline reduces analysis cost by 94.6 per cent, from an estimated $509.78 to $27.35 for the complete evaluation corpus of 14,000 flows.

A controlled seven-configuration ablation study demonstrates that the multi-agent architecture outperforms the best single-agent configuration by 25.2 F1 percentage points, and that tool augmentation — even with dataset-compatible tools returning genuinely informative results — actively degrades single-agent detection performance. The Infiltration attack category produces zero per cent recall under the v2 per-flow architecture, exposing a fundamental limitation of individual flow-level analysis. A v3 temporal clustering experiment recovers 58% recall on Infiltration by presenting agents with multi-flow clusters grouped by source IP, though at the cost of an elevated 19.2% false positive rate that highlights the inherent difficulty of detecting multi-stage intrusion campaigns.

Contents

Declaration by Author **ii**

Use of AI Statement **iii**

Acknowledgements **v**

Abstract **vi**

List of Tables **x**

List of Figures **xii**

1 Introduction **1**
1.1 Background and Motivation 1
1.2 Research Questions 3
1.3 The AMATAS System 4
1.4 Contributions 5
1.5 Thesis Structure 5

2 Literature Review **7**
2.1 Traditional Machine Learning for Intrusion Detection 7
2.2 Large Language Models in Cybersecurity 10
2.3 Multi-Agent LLM Architectures 13
2.4 Tool-Augmented LLMs and the Model Context Protocol 16
2.5 Explainable AI for Network Security 19
2.6 Hybrid Architectures and Cost Optimisation 21
2.7 Research Gap 23

3 System Architecture **26**
3.1 Design Philosophy 26
3.2 Tier 1: Random Forest Pre-Filter 28
3.2.1 Feature Set and Training Data 29
3.2.2 Threshold Selection 29

3.3 Tier 2: Specialist Agents 30
3.4 The Devil's Advocate Mechanism 32
3.5 Orchestrator and Consensus 34
3.6 Cost Architecture 35
3.7 Temporal Clustering Module 36

4 Experimental Methodology 38
4.1 Dataset Description 38
4.2 Batch Construction 40
4.3 Evaluation Metrics 41
4.4 Stage 1 Experimental Protocol 42
4.5 MCP Comparison Protocol 44
4.6 Data Integrity and Leakage Mitigation 46
4.7 Reproducibility Measures 47

5 Results 50
5.1 Stage 1 Results Overview 50
5.1.1 Random Forest Standalone Baseline 52
5.2 Detection Performance by Attack Category 53
5.2.1 Brute Force Attacks 53
5.2.2 Denial-of-Service Attacks 53
5.2.3 Distributed Denial-of-Service Attacks 54
5.2.4 Web Application Attacks 55
5.2.5 Stealthy and Advanced Attacks 56
5.3 False Positive Analysis 56
5.4 Tier 1 Pre-Filter Performance 58
5.5 MCP Comparison Results 59
5.5.1 Extended Tool Comparison (Configs D–G) 60
5.6 Cost Analysis 61
5.7 The Infiltration Failure 64
5.7.1 AMATAS v3: Temporal Clustering Evaluation 65
5.8 Agent Ablation Study 67
5.9 Reasoning Faithfulness Audit 73
5.10 Tier 1 Routing Validation 74

6 Discussion 76
6.1 Interpreting the Detection Performance 76
6.2 The Explainability Advantage 78
6.3 The False Positive Problem Resolved 81
6.4 Limitations and Failure Modes 83

6.4.1 The Infiltration Failure 83
6.4.2 DDoS-HOIC and Feature Overlap 83
6.4.3 The Semantic Gap in Flow-Level Analysis 83
6.4.4 MCP Tool Utility and Anonymised Datasets 84
6.4.5 Model Selection Constraints and Generalisability 86
6.5 Security Considerations for MCP-Enhanced Detection Systems 87
6.6 Threats to Validity 88

7 Conclusion **90**
7.1 Summary of Contributions 90
7.2 Answers to the Research Questions 91
7.3 Future Work 93

References **95**

List of Tables

5.1 Stage 1 Per-Experiment Results (n = 1,000 flows per experiment; 50 attack + 950 benign) . . . 51
5.2 RF Standalone Classifier vs. AMATAS: Per-Attack F1 (%) . . . 52
5.3 DoS Attack Performance Summary . . . 54
5.4 DDoS Attack Performance Summary . . . 55
5.5 Web Application Attack Performance Summary . . . 56
5.6 False Positive Distribution Across Experiments . . . 57
5.7 Tier 1 Pre-Filter Performance per Experiment . . . 58
5.8 MCP Comparison: Seven Single-Agent Configurations . . . 59
5.9 Cost per Experiment . . . 62
5.10 Agent Cost Distribution (% of total LLM spend) . . . 62
5.11 Infiltration Failure Analysis . . . 65
5.12 AMATAS v3 Clustering Experiment: Infiltration Results . . . 66
5.13 Agent Ablation Results: FTP-BruteForce (n = 1,000; 50 attack + 950 benign) . . . 68
5.14 Agent Ablation Results: SSH-Bruteforce (n = 1,000; 50 attack + 950 benign) . . . 69
5.15 Agent Ablation Results: DDoS-HOIC (n = 1,000; 50 attack + 950 benign) . . . 69
5.16 Faithfulness Audit: Claim Verification by Category (n = 6,279 claims across 758 flows) . . . 73
5.17 Faithfulness Audit: Per-Agent Breakdown . . . 74
5.18 Tier 1 Routing Validation: Trained RF vs. Random Filters on FTP-BruteForce . . . 75

6.1 SHAP vs AMATAS: Five representative attack flows . . . 80

List of Figures

2.1 The IDS-Agent architecture. A single LLM agent iteratively retrieves information through tool calls (data extraction, ML classification, knowledge retrieval) before producing a verdict. Unlike AMATAS, IDS-Agent employs a single reasoning agent without specialist role diversity or adversarial false positive control. Source: Li et al. [20]. . . 12

2.2 Accuracy comparison between single-agent inference and multi-agent debate across six reasoning benchmarks. Multi-agent debate consistently outperforms single-agent baselines, with the largest gains on tasks requiring multi-step reasoning. Source: Du et al. [7]. 14

2.3 Example of multi-agent debate between ChatGPT and Bard on a mathematical reasoning task. In Round 1, both agents arrive at incorrect answers through different reasoning errors. In Round 2, ChatGPT corrects its reasoning after reviewing Bard's response, converging on the correct answer of 160 minutes. Source: Du et al. [7]. 14

2.4 The ReAct framework compared to reasoning-only and acting-only paradigms. Reasoning-only (left) generates thought traces without environment interaction; acting-only (centre) takes actions without explicit reasoning. ReAct (bottom) synergises both, interleaving reasoning traces with actions and observations to enable grounded, iterative problem-solving. Source: Yao et al. [24]. 17

2.5 A SHAP waterfall plot showing per-feature contributions to a single prediction. Each bar indicates how much a feature's value pushes the prediction above (red) or below (blue) the expected baseline $E[f(X)]$. Whilst informative about *which* features were influential, such explanations do not articulate *what those features mean* in the context of a specific attack scenario or provide counter-arguments. Source: Lundberg and Lee [15]; figure from the SHAP library documentation. 20

2.6 The two-tier LLM-enhanced intrusion detection architecture for SDN and Kubernetes environments. Statistical anomaly detection provides continuous monitoring (Tier 1), triggering on-demand LLM-based deep analysis (Tier 2) only when anomalies are detected. This independent design mirrors the AMATAS two-tier approach of cheap ML pre-filtering followed by expensive LLM analysis. Source: Kalafatidis et al. [31]. 22

3.1 AMATAS two-tier architecture. Tier 1 filters ∼95% of flows as auto-benign; remaining flows pass through the three-phase LLM pipeline. . 28

5.1 Stage 1 F1 scores across all fourteen attack types. Colours indicate performance tiers: green (F1 $\geq$ 99%), blue (90–96%), gold (85–89%), orange/red (< 85%). 51

5.2 MCP comparison: F1 score versus false positive rate across all seven single-agent configurations and AMATAS v2. Configurations D–G (dataset-compatible tools) perform *worse* than the no-tool baseline (B), demonstrating that tool access actively degrades single-agent detection. 60

5.3 Per-agent share of total LLM cost. The Temporal Agent dominates at 30% due to multi-flow context injection; the three cheaper specialists together account for 30.2%. 63

5.4 Agent ablation recall comparison across three attack types. The Devil's Advocate removal (orange) improves DDoS-HOIC recall from 58% to 98%, whilst Temporal Agent removal (maroon) degrades recall across all categories. 70

Chapter 1

Introduction

1.1 Background and Motivation

The volume and sophistication of cyber threats targeting modern enterprise networks have grown at a pace that traditional security infrastructure struggles to match. In 2024, the global average cost of a data breach exceeded four million US dollars, and the median dwell time for advanced persistent threats remained measured in weeks rather than hours [1]. Network Intrusion Detection Systems sit at the centre of this defensive landscape, monitoring traffic flows for signatures of malicious activity. Yet the fundamental approaches underpinning most deployed systems have changed remarkably little since the 1990s. Signature-based engines such as Snort and Suricata remain effective against known attack patterns but are blind to novel threats, whilst anomaly-based classifiers built on machine learning achieve strong detection rates on benchmark datasets only to suffer from high false positive rates and a complete absence of interpretable reasoning when deployed against real traffic [2].

This interpretability gap carries practical consequences. When a Random Forest or gradient-boosted ensemble flags a network flow as malicious, it produces a probability and nothing more. The security analyst receiving that alert has no visibility into which features drove the decision, whether the model considered temporal context, or how the flagged flow relates to other traffic from the same source. In high-volume environments processing millions of flows per hour, this opacity transforms intrusion detection into an exercise in alert triage rather than threat understanding. The resulting alert fatigue has been well documented: analysts routinely ignore a substantial fraction of generated alerts because investigating opaque, unexplained detections is prohibitively time-consuming [3].

Large Language Models present a potential solution to this interpretability deficit. Unlike traditional classifiers that map feature vectors to class probabilities through learned but unexplainable decision boundaries, LLMs reason through evidence in

natural language, producing explanations that security analysts can read, evaluate, and critique. When an LLM-based system classifies a network flow as suspicious, it can articulate which protocol characteristics triggered concern, what statistical anomalies it observed, how the flow compares to known attack signatures, and what counter-arguments exist for a benign interpretation. Providing transparent reasoning outputs offers more than standard usability improvements; it enables analysts to evaluate the specific evidence driving a classification rather than relying solely on opaque probability scores.

However, applying LLMs to network intrusion detection introduces challenges that the natural language processing community has not traditionally faced. Network flows are not sentences; they are structured records of numerical features describing packet counts, byte volumes, port numbers, protocol flags, and timing intervals. The translation from structured telemetry to natural language reasoning requires careful prompt engineering and domain knowledge injection. Furthermore, individual LLM calls carry non-trivial computational costs, and processing millions of flows through a full LLM pipeline would be economically prohibitive at current API pricing. A production-viable LLM-based NIDS must therefore solve two problems simultaneously: it must achieve detection accuracy competitive with traditional classifiers, and it must do so at a cost per flow that scales to enterprise traffic volumes.

The architecture presented in this thesis evolved substantially from its original design. The initial proposal envisioned the Model Context Protocol [4] as the primary orchestration layer, connecting a single LLM agent to seven external threat intelligence tools (including AbuseIPDB, AlienVault OTX, and geolocation services) under the hypothesis that tool-augmented reasoning would outperform standalone classification. Early experiments revealed two findings that redirected the architectural trajectory. First, external threat intelligence tools returned no actionable information on the anonymised private IP addresses that constitute the CICIDS2018 address space, rendering the tool-augmentation strategy ineffective on this benchmark. Second, evaluations demonstrated that a single LLM agent's capacity for multi-dimensional flow analysis is strictly limited, regardless of prompt complexity or external tool access, necessitating a distributed approach. These findings motivated the pivot to a multi-agent debate architecture, where the diversity of specialist analytical perspectives and adversarial argumentation replaced external tool access as the mechanism for improving detection quality. The MCP framework was retained as a comparison baseline rather than discarded, enabling a controlled ablation study that quantifies the contribution of multi-agent debate relative to tool augmentation.

1.2 Research Questions

This thesis investigates the following research questions:

RQ1: Can a multi-agent LLM architecture achieve detection accuracy comparable to traditional machine learning classifiers on a standardised intrusion detection benchmark?

Traditional ML classifiers achieve high accuracy on the CICIDS2018 dataset but provide no reasoning. Single LLM agents, as demonstrated by Houssel et al. [5], struggle with direct flow classification. This question examines whether distributing analysis across multiple specialist LLM agents, each focused on a distinct analytical dimension, can overcome the performance limitations observed in single-agent approaches.

RQ2: How does a two-tier architecture combining a machine learning pre-filter with a multi-agent LLM pipeline affect the cost-accuracy trade-off for LLM-based intrusion detection?

The economic viability of LLM-based NIDS depends on reducing the number of flows that require expensive LLM analysis. This question evaluates whether a Random Forest pre-filter can eliminate the vast majority of obviously benign flows without sacrificing detection recall, thereby making LLM-based analysis economically feasible at scale.

RQ3: What is the quality and utility of the reasoning chains produced by a multi-agent LLM NIDS compared to the opaque outputs of traditional classifiers?

Explainability is the central motivation for this work. This question assesses whether the natural language reasoning produced by AMATAS provides actionable insight that traditional classifier outputs cannot, and whether the multi-perspective analysis from specialist agents produces richer explanations than a single-agent approach.

RQ4: How does detection performance vary across attack types, and what are the fundamental limitations of flow-level LLM analysis?

Not all attacks present equally at the flow level. Brute-force attacks produce distinctive port and timing patterns, whilst infiltration attacks may be indistinguishable from benign traffic in individual flows. This question evaluates detection performance across fourteen attack categories to identify the specific threat profiles where LLM-based analysis is effective and to define its functional limitations.

1.3 The AMATAS System

This thesis presents AMATAS, the Advanced Multi-Agent Threat Analysis System, a network intrusion detection architecture that combines a machine learning pre-filter with a six-agent LLM pipeline to deliver both accurate detection and transparent reasoning. The system processes network flows through two tiers. The first tier employs a Random Forest classifier trained on the CICIDS2018 development partition to filter flows that are confidently benign, eliminating approximately ninety-five per cent of traffic from further analysis at negligible cost. The remaining flows, those where the Random Forest assigns a non-trivial probability of being malicious, proceed to the second tier.

The second tier consists of six LLM agents organised into three phases. In the first phase, four specialist agents analyse the flow in parallel, each from a distinct analytical perspective: the Protocol Agent validates port assignments, flag combinations, and protocol-level anomalies; the Statistical Agent examines byte volumes, packet counts, and timing characteristics against expected distributions; the Behavioural Agent matches observed patterns against known attack signatures and maps findings to the MITRE ATT&CK framework; and the Temporal Agent correlates the flow with other traffic from the same source IP address, identifying suspicious clusters and behavioural sequences. In the second phase, a Devil's Advocate agent processes the four specialist analyses and generates a counter-argument hypothesising that the flow is benign, providing an explicit challenge to the initial evidence. In the third phase, an Orchestrator agent synthesises all five preceding analyses into a final verdict of BENIGN, SUSPICIOUS, or MALICIOUS, accompanied by a confidence score, attack type classification, and full reasoning narrative.

This architecture draws theoretical grounding from Minsky's Society of Mind framework [6], which posits that intelligence emerges from the interaction of many small, specialised agents rather than from a single monolithic reasoning process. The multi-agent debate paradigm has been shown by Du et al. [7] to improve factual accuracy and reduce hallucinations in LLM outputs, whilst Chan et al. [8] demonstrated that assigning diverse roles to debating agents outperforms uniform-role configurations. The adversarial component introduced by the Devil's Advocate agent finds empirical support in recent work by Chiang et al. [9], which showed that LLM-powered devil's advocate roles improve decision quality and reduce inappropriate reliance on AI outputs.

1.4 Contributions

This thesis makes the following contributions to the field of LLM-augmented network security:

First, it presents the first systematic per-attack-type evaluation of a multi-agent LLM intrusion detection system at realistic class distributions. Existing LLM-based NIDS evaluations either use balanced datasets or evaluate on aggregated mixed-attack batches, neither of which reflects the class imbalance encountered in production networks where benign traffic typically exceeds ninety per cent. AMATAS is evaluated across all fourteen attack types in the CICIDS2018 dataset at a five per cent attack prevalence per experiment, providing granular insight into per-category detection performance.

Second, it demonstrates a two-tier ML-plus-LLM architecture that reduces the cost of LLM-based intrusion detection by ninety-five per cent whilst maintaining detection accuracy. The Random Forest pre-filter achieves one hundred per cent recall on attack flows at the operating threshold, ensuring that no attacks are lost to pre-filtering, whilst diverting approximately ninety-five per cent of benign flows away from the expensive LLM pipeline.

Third, it introduces a Devil's Advocate agent as a structural mechanism for false positive control in LLM-based detection. For thirteen of the fourteen evaluated attack types, AMATAS achieved a 0% false positive rate. This reduction is attributed to the secondary validation phase, in which the Devil's Advocate agent algorithmically challenges the initial malicious classifications to mitigate false positives.

Fourth, it provides a complete explainability framework where every detection decision is accompanied by multi-perspective reasoning chains from six analytical agents, enabling security analysts to trace exactly why a flow was flagged, what evidence was considered, and what counter-arguments were weighed.

Fifth, it conducts a seven-configuration ablation study comparing single-agent approaches with progressively richer tool access (including dataset-compatible tools that return genuinely informative results) against the full multi-agent architecture, quantifying the detection improvement attributable to multi-agent debate and demonstrating that tool augmentation actively degrades single-agent performance even when tools provide accurate, relevant information.

1.5 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 reviews related work in machine learning for intrusion detection, LLM applications in cybersecurity, multi-agent LLM systems, and explainable AI for network security. Chapter 3 describes

the AMATAS architecture in detail, including the Random Forest pre-filter, agent design, prompt engineering, and consensus mechanism. Chapter 4 presents the experimental methodology, covering dataset preparation, evaluation metrics, and experimental configurations. Chapter 5 reports the results of the Stage 1 evaluation across all fourteen attack types, the MCP comparison experiments, and the cost analysis. Chapter 6 discusses the implications of these results, analyses failure cases, and identifies the fundamental limitations of flow-level LLM analysis. Chapter 7 concludes with a summary of contributions and directions for future work.

Chapter 2

Literature Review

2.1 Traditional Machine Learning for Intrusion Detection

Network intrusion detection has been an active area of research for more than three decades, yet the fundamental tension between detection coverage and operational precision remains unresolved. Early systems relied almost exclusively on rule-based signature matching, an approach exemplified by tools such as Snort [10] and Suricata [11], which compare observed traffic against manually maintained databases of known attack patterns. These systems achieve near-perfect detection for attacks whose signatures are explicitly codified, operate at line rate in many production environments, and are sufficiently interpretable that security analysts can understand precisely why an alert was triggered. However, their fundamental limitation is structural rather than incidental: a signature-based system cannot, by definition, detect what it has not been told to look for. Zero-day exploits, novel variants of known attack families, and polymorphic malware that alters its network-level characteristics to avoid pattern matching all pass through signature-based engines undetected. As the attack surface has expanded and threat actors have grown more sophisticated in evading static rules, this blindness to unknown threats has become the dominant limitation of rule-based intrusion detection.

The response from the research community was to apply machine learning to the intrusion detection problem, framing it as a binary or multiclass classification task over feature vectors derived from network flow records. Random Forests, Support Vector Machines, k-Nearest Neighbour classifiers, and a range of gradient-boosted ensemble methods were applied to successively larger benchmark datasets with considerable apparent success. The progression of evaluation datasets, from KDD CUP 99 through UNSW-NB15 to the CICIDS series produced by the Canadian Institute for Cybersecurity, provided standardised benchmarks that permitted comparison

across studies. On the CICIDS2017 and CSE-CIC-IDS2018 datasets in particular, reported detection accuracies frequently exceeded ninety-nine per cent, leading to the impression that the intrusion detection problem had been largely solved by modern classifiers.

The survey by Ferrag et al. [12] provides a comprehensive comparative assessment of deep learning approaches applied to CSE-CIC-IDS2018, evaluating seven model architectures including convolutional neural networks, recurrent neural networks, deep neural networks, deep belief networks, and deep autoencoders across all fourteen attack categories. Their results demonstrate that the best-performing architectures achieve overall accuracies approaching ninety-seven per cent, with per-attack detection rates varying substantially: near-perfect detection for brute-force attacks but markedly lower rates (around eighty-two per cent) for web-based attacks such as cross-site scripting. The variation in per-attack detection rates highlights a recurring concern in the literature: aggregate accuracy metrics mask significant performance differences across attack types, and classifiers that appear highly effective in overall terms may fail on specific attack categories that are underrepresented in training data or whose network signatures overlap with benign traffic.

Leevy and Khoshgoftaar [13] address this concern directly, examining a substantial corpus of machine learning intrusion detection studies and finding that reported accuracies are suspiciously high across nearly all approaches. Their analysis identifies several methodological factors that inflate benchmark performance: the mixing of training and test flows from the same brief recording window, the absence of temporal holdout splits that would test generalisation to future traffic, the use of evaluation metrics that overweight the dominant benign class, and the inadvertent inclusion of near-duplicate flows that allow classifiers to memorise rather than generalise. The authors conclude that the field's published performance figures systematically overestimate real-world detection capability, and that the true gap between benchmark accuracy and deployment effectiveness is substantially larger than commonly acknowledged. This finding is corroborated by the more recent empirical work of Hesford et al. [2], who evaluated trained classifiers across multiple benchmark datasets (including CICIDS2017 and UNSW-NB15) to assess transferability, and found that precision and recall degraded markedly compared to within-dataset results, with false positive rates that would generate unacceptable alert volumes in operational contexts. Hesford et al. also identify significant reproducibility barriers in the ML-NIDS literature, reporting dependency errors and insufficient documentation that prevented replication of several published results. Hesford et al. [2] attribute this gap to distributional shift: the statistical properties of real network traffic change continuously as applications, user behaviours, and infrastructure evolve, whilst static classifiers trained on historical benchmarks cannot adapt to these drifts without

retraining.

The class imbalance problem presents an additional practical challenge that benchmark studies frequently understate. The CICIDS2018 dataset, which serves as the primary evaluation corpus for this thesis, exhibits an approximately eighty-seven per cent prevalence of benign traffic across all attack categories combined. In individual attack categories, the prevalence of malicious flows is considerably lower. A classifier that predicts every flow as benign achieves roughly eighty-seven per cent accuracy on this dataset whilst detecting precisely zero attacks, a result that is both trivially achieved and entirely useless for security purposes. Techniques for addressing class imbalance, including SMOTE oversampling, undersampling of the majority class, class-weighted loss functions, and threshold adjustment, have been extensively studied [14], but each introduces its own trade-offs. Aggressive oversampling of minority classes can amplify noise in the training set; threshold adjustment optimised on validation data may not transfer to real traffic where attack prevalence differs from the training distribution. The practical implication is that intrusion detection systems must be evaluated at realistic class distributions (matching the attack prevalence of production environments) rather than at balanced or artificially enriched class ratios. This methodological commitment distinguishes the present work from the majority of published LLM-based NIDS studies, which evaluate on small, balanced, or manually curated samples.

Beyond the performance limitations, traditional ML classifiers share a fundamental interpretability deficit. A Random Forest producing a malicious verdict for a network flow does so through an ensemble of hundreds of binary decision trees, the collective output of which is a class probability derived from no single comprehensible reasoning path. Post-hoc explanation methods such as SHAP [15] and LIME [16] can recover approximations of feature importance for individual predictions, but these approximations carry their own limitations: they are local linearisations of a non-linear decision surface, they do not explain how features interact, and they provide no mechanism for contextualising a prediction against the broader threat landscape. The security analyst receiving a SHAP explanation that certain byte-count and packet-ratio features contributed to a malicious classification gains little actionable insight into what type of attack may be occurring, how confident the system is in that assessment, or what the strongest arguments against the classification would be. This interpretability gap, rather than any quantitative performance ceiling, is the central motivation for the LLM-based approach pursued in this thesis.

2.2 Large Language Models in Cybersecurity

The application of large language models to cybersecurity tasks has expanded rapidly following the widespread availability of instruction-following models such as GPT-4, Claude, and open-weight alternatives including LLaMA and Mistral. The foundational concern raised by Gupta et al. [17] in their dual-use framework is that the same capabilities that make LLMs useful defensive tools also make them potentially dangerous offensive instruments. Their analysis, framed through the progression from ChatGPT to what they term ThreatGPT, demonstrates that LLMs can generate functional malware, draft convincing phishing content, identify vulnerabilities in code, and explain attack methodologies in detail sufficient to assist unsophisticated threat actors. This dual-use dynamic has shaped the research community's approach to LLM deployment in cybersecurity, motivating careful evaluation of both the benefits and the risks of integrating generative models into security tooling. For intrusion detection specifically, the dual-use concern is less acute (an intrusion detection system produces verdicts rather than attack capabilities), but the broader question of whether LLMs can be trusted to reason reliably about security-sensitive information remains open.

Feng and Sakurai [18] provide the most comprehensive recent survey of LLM applications in network intrusion detection, tracing the evolution from early prompt-engineering experiments through fine-tuned models and agentic architectures. Their taxonomy distinguishes three waves of LLM-NIDS research: the first wave applied off-the-shelf instruction-following models to raw or lightly processed network flow descriptions and found that zero-shot performance was poor, typically near random; the second wave applied fine-tuning and domain adaptation to overcome the knowledge gap between pre-training corpora (which contain negligible network telemetry) and the target task; and the third wave, which remains nascent, explores agentic and multi-agent configurations where LLMs reason iteratively rather than producing single-pass classifications. The survey identifies a consistent finding across first-wave studies: general-purpose LLMs exhibit substantial uncertainty when reasoning about network features because their pre-training corpora contain little structured telemetry data, and their representations of concepts such as port numbers, byte ratios, and packet inter-arrival times are underspecified relative to the precision required for accurate classification.

The empirical finding of Houssel et al. [5] quantifies this limitation with particular clarity. Their study presented GPT-4 and LLaMA-3 with NetFlow records from standardised intrusion detection datasets, encoding each flow's numerical features as key-value text pairs (e.g., `L4_DST_PORT: 80`), and found that both models achieved approximately fifty per cent macro-averaged precision and recall in zero-shot evalu-

ation, effectively no better than random guessing. Critically, the models' errors were not random: qualitative analysis of individual predictions revealed systematic confusions in both directions, with the models over-classifying certain flows as malicious based on superficial feature patterns (such as atypical port numbers or large file transfers) whilst simultaneously misclassifying genuinely malicious flows as benign when packet lengths and retransmission counts fell within ranges the model associated with normal traffic. Houssel et al. describe their study as an "initial exploration," acknowledging that small sample sizes limit generalisability. They also report severe hallucinations, including fabrication of geographic locations for anonymised IPs, highlighting the risks of deploying LLMs without structured output constraints. This result is significant not because it shows LLMs are useless for intrusion detection (the present work demonstrates otherwise) but because it establishes that direct flow classification is not an appropriate deployment mode for general-purpose LLMs. The model's pre-training does not equip it to adjudicate precisely between feature distributions that differ subtly between benign and malicious traffic; what it can do is reason about the significance of features once their anomalous character has been established through statistical or signature-based analysis. This distinction between LLMs as classifiers and LLMs as explainability agents is a central architectural insight of AMATAS.

The fine-tuning literature reveals a different picture. Gutiérrez-Galeano et al. [19] demonstrate that fine-tuning a T5 encoder-decoder language model on labelled network flow data, transformed into a structured textual representation, yields a detector achieving 99.84 per cent accuracy on the CSE-CIC-IDS2018 dataset, a performance figure competitive with state-of-the-art Random Forest classifiers. This aggregate figure masks significant per-class variation: the model struggled with minority attack categories (SQL Injection, Brute Force-Web, XSS), and the authors note that direct latency comparisons were not feasible due to hardware variations across studies. However, fine-tuned models of this type share the interpretability limitations of traditional classifiers: the model assigns a label through learned parameter weights that encode no meaningful reasoning about why a flow was classified as it was. The high accuracy figure is achieved at the cost of the explainability that motivated the application of language models in the first place. Moreover, fine-tuned models are inherently static (they must be retrained as attack patterns evolve) and they cannot leverage the broad world knowledge that makes prompted LLMs capable of contextualising detections within threat landscapes, mapping findings to MITRE ATT&CK, or reasoning about attack intent from flow-level evidence.

The work of Li et al. [20] on IDS-Agent represents the closest prior work to the AMATAS architecture. IDS-Agent is an LLM-based intrusion detection system that combines prompted reasoning with access to external tools, applying an agentic

loop in which the model iteratively retrieves relevant information before producing a verdict. Evaluated on the ACI-IoT and CIC-IoT datasets, IDS-Agent achieves an F1 score of 0.97 on ACI-IoT and 0.75 on CIC-IoT, a marked improvement over non-agentic LLM baselines and competitive with fine-tuned classifiers on the ACI-IoT benchmark. These results confirm that structuring LLM interaction as an iterative reasoning process rather than a single-pass classification substantially improves performance. However, IDS-Agent employs a single agent with tool access rather than multiple agents with distinct analytical roles, and does not include an adversarial mechanism for false positive control. The present work extends this line of research by distributing analysis across six specialist agents, introducing an adversarial Devil's Advocate component, and evaluating across a broader and more challenging attack taxonomy at realistic class distributions.

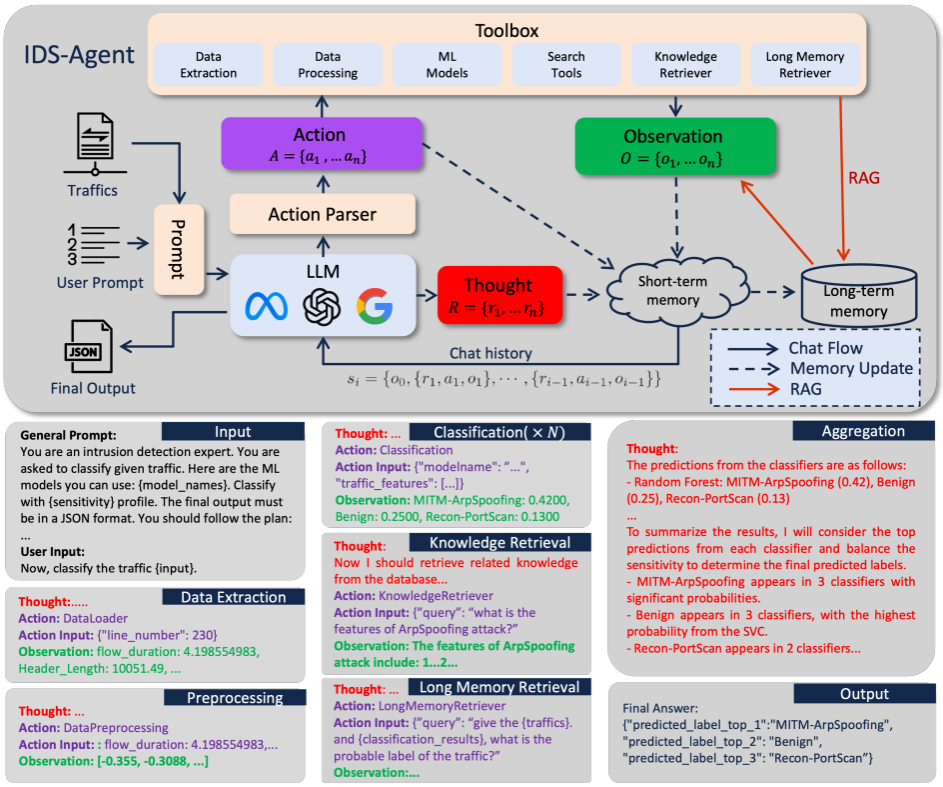

Figure 2.1: The IDS-Agent architecture. A single LLM agent iteratively retrieves information through tool calls (data extraction, ML classification, knowledge retrieval) before producing a verdict. Unlike AMATAS, IDS-Agent employs a single reasoning agent without specialist role diversity or adversarial false positive control. Source: Li et al. [20].

The collective evidence from this literature points towards a nuanced conclusion: LLMs fail as standalone classifiers in the sense explored by Houssel et al. [5], but succeed as reasoning engines when deployed within appropriate architectural frame-

works. The key architectural requirements appear to be that flows are described in natural language that highlights anomalous features rather than presented as raw numerical vectors; that the model is equipped with relevant domain knowledge through system prompting or retrieval; and that reasoning proceeds iteratively or through multiple analytical passes rather than in a single generation. AMATAS addresses all three requirements through its specialist agent design, domain-specific system prompts, and multi-phase consensus architecture.

2.3 Multi-Agent LLM Architectures

The theoretical foundation for multi-agent architectures in artificial intelligence predates large language models by several decades. Minsky [6] proposed that complex intelligent behaviour emerges not from a single unified reasoning process but from the coordinated interaction of many small, specialised agents, each competent within a narrow domain but collectively capable of producing sophisticated, context-sensitive responses. This framework has proved remarkably prescient as a metaphor for multi-agent LLM systems: the observation that diverse specialist perspectives, properly coordinated, outperform any individual agent reasoning in isolation finds strong empirical support in the contemporary literature.

Du et al. [7] provide the first systematic empirical investigation of multi-agent debate as a mechanism for improving LLM reasoning quality. In their framework, multiple language model instances independently generate responses to a question and then iteratively revise their answers based on the responses of other agents, a process of structured argumentation that continues until consensus emerges or a predefined round limit is reached. Evaluated across mathematical reasoning tasks, factual question answering, and strategic reasoning benchmarks, multi-agent debate consistently outperforms single-agent baselines, with gains particularly pronounced on tasks requiring multi-step reasoning where individual agents are prone to early commitment to incorrect intermediate conclusions. The mechanism through which debate improves performance is interpretable: agents correct factual errors surfaced by their debate partners, abandon flawed reasoning chains when challenged, and converge on stronger arguments through structured disagreement. For intrusion detection, where individual agents may over-weight specific features whilst ignoring others, this correction mechanism has direct practical value. Du et al. also caution that debate can converge on confidently wrong answers when all agents share the same initial misconception — a failure mode that motivates the inclusion of a structurally adversarial agent in AMATAS rather than relying on homogeneous debate alone.

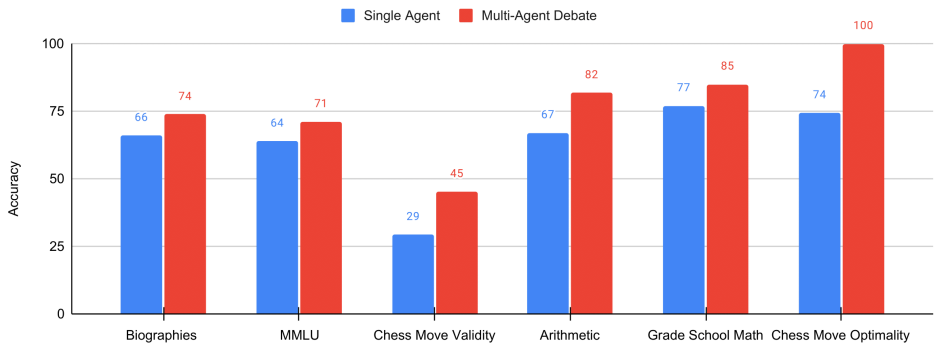

Figure 2.2: Accuracy comparison between single-agent inference and multi-agent debate across six reasoning benchmarks. Multi-agent debate consistently outperforms single-agent baselines, with the largest gains on tasks requiring multi-step reasoning. Source: Du et al. [7].

Question: Carla is downloading a 200 GB file. Normally she can download 2 GB/minute, but 40% of the way through the download, Windows forces a restart to install updates, which takes 20 minutes. Then Carla has to restart the download from the beginning. How load does it take to download the file?

Round 1

ChatGPT: Before the restart, Carla was able to download 40% of the file, which is 0.4 * 200 GB = 80 GB. At a rate of 2 GB/minute, it would take her: time = 80 GB ÷ 2 GB/minute = 40 minutes.

Because of the restart, Carla had to start over. This means she had to download the entire 200 GB file. The time it takes to download the remaining 120 GB is: time = 120 GB ÷ 2 GB/minute = 60 minutes.

But Carla also had to wait 20 minutes for the Windows update to install, so the total time it took to download the file is: total time = 40 minutes + 60 minutes + 20 minutes = **120 minutes**. ✗

Bard: Carla is downloading a 200 GB file at a speed of 2 GB/minute. This means that it will take her 200 / 2 = 100 minutes to download the file.

However, Windows forces a restart to install updates, which takes 20 minutes. This means that Carla will have to restart the download from the beginning. Since Carla has already downloaded 40% of the file, she only needs to download 200 - 40 = 160 GB. At a speed of 2 GB/minute, it will take Carla 160 / 2 = 80 minutes to download the file.

Adding the time for the Windows restart, it will take Carla 80 + 20 = **100 minutes** to download the file. ✗

Round 2

ChatGPT : Based on the calculations provided by the other agent, the correct answer is: Before the restart, Carla was able to download 40% of the file, which is 0.4 * 200 GB = 80 GB. At a rate of 2 GB/minute, it would take her: time = 80 GB ÷ 2 GB/minute = 40 minutes

However, because of the restart, Carla had to start over. This means she had to download the entire 200 GB file. The time it takes to download the entire 200 GB file is: time = 200 GB ÷ 2 GB/minute = 100 minutes

And since Carla had to wait 20 minutes for the Windows update to install, the total time it took to download the file is: total time = 40 minutes + 100 minutes + 20 minutes = **160 minutes**. ✓

Figure 2.3: Example of multi-agent debate between ChatGPT and Bard on a mathematical reasoning task. In Round 1, both agents arrive at incorrect answers through different reasoning errors. In Round 2, ChatGPT corrects its reasoning after reviewing Bard's response, converging on the correct answer of 160 minutes. Source: Du et al. [7].

Chan et al. [8] extend this investigation through the ChatEval framework, which systematically varies the role assignments given to debating agents and measures the effect on response quality across text evaluation tasks. Their central finding is that

diverse role assignments (where each agent is given a distinct analytical persona with different priorities and evaluation criteria) consistently outperform configurations where all agents are assigned identical or similar roles. In the uniform-role condition, agents tend to converge quickly on the first plausible response generated by any participant, as they lack the analytical diversity to surface meaningful objections. In the diverse-role condition, the structural differences in perspective force genuine debate, surfacing weaknesses in preliminary conclusions and producing final outputs of measurably higher quality. This finding directly motivates AMATAS's design decision to assign genuinely distinct analytical mandates to its specialist agents (protocol analysis, statistical anomaly detection, behavioural signature matching, and temporal correlation) rather than deploying multiple instances of a single general-purpose detection agent. Chan et al. additionally found that sequential communication (where agents respond one-by-one with access to prior outputs) was more effective than simultaneous independent generation for ChatGPT agents. AMATAS nonetheless employs parallel specialist execution because the independence constraint ensures no specialist anchors on another's preliminary conclusion, and parallel API dispatch reduces wall-clock latency by a factor of four.

Wu et al. [21] address the practical infrastructure challenge of implementing multi-agent LLM conversations through the AutoGen framework, which provides a conversation management layer enabling arbitrary networks of LLM agents to exchange messages, invoke tools, and produce collaborative outputs. AutoGen's contribution is primarily architectural: it separates the concern of agent coordination from the concern of agent capability, allowing researchers to define multi-agent topologies without re-implementing the messaging and state management infrastructure for each application. The framework has been adopted widely in subsequent research on agentic LLM systems, and its conversation-management primitives informed the design of AMATAS's agent orchestration layer, though AMATAS employs a custom implementation tailored to the specific requirements of flow-level intrusion detection, including cost tracking per agent call, structured output enforcement, and parallel execution of the four specialist phases.

The specific innovation of an adversarial Devil's Advocate agent, a central design element of AMATAS, finds support in recent empirical work examining how adversarial critique improves LLM decision quality. Chiang et al. [9] introduce a Devil's Advocate prompting paradigm in which a secondary LLM agent is instructed to identify flaws in and argue against the primary agent's conclusion, specifically tasked with constructing the strongest possible counter-argument regardless of its own belief about the correct answer. The authors find that this adversarial pressure significantly improves decision quality on complex reasoning tasks, reducing both confident errors (where an agent is wrong but certain) and over-reliance on initial impressions.

Crucially, the improvement in decision quality is asymmetric: the Devil's Advocate intervention is most beneficial when the primary agent's initial conclusion is incorrect, precisely the scenario (a flow that superficially resembles an attack but is genuinely benign) that drives false positive rates in intrusion detection systems. A complementary study by Yang et al. [22] demonstrates that adversarial debate frameworks reduce hallucination rates in LLM outputs by exposing unsupported claims to structured challenge, with agents less likely to assert facts without evidence when they know those assertions will be scrutinised by an adversarial peer. For intrusion detection, where confident but incorrect malicious classifications translate directly into wasted analyst time and degraded trust in the detection system, this hallucination-reduction property of adversarial debate represents a practical operational benefit beyond the accuracy improvements measured in benchmark evaluation.

The collective evidence from multi-agent LLM research supports two architectural commitments that shape AMATAS: first, that specialist role diversity improves collective reasoning quality; and second, that adversarial challenge reduces overconfident errors. Both commitments are implemented structurally in AMATAS rather than through prompt engineering alone: the diversity is enforced by giving each agent a different analytical mandate with distinct evaluation criteria, and the adversarial challenge is enforced by giving the Devil's Advocate an explicit counter-advocacy role independent of the specialists' verdicts.

2.4 Tool-Augmented LLMs and the Model Context Protocol

The reasoning capabilities of large language models can be substantially enhanced by providing access to external tools that retrieve information, perform computations, or interact with external systems. The theoretical basis for tool augmentation as a reasoning enhancement strategy was established by Wei et al. [23] in their introduction of Chain-of-Thought prompting, which demonstrated that instructing LLMs to produce intermediate reasoning steps before final answers markedly improves performance on multi-step tasks. Chain-of-Thought prompting is not tool use in the strict sense (the model reasons using only its parametric knowledge), but it establishes the principle that structured reasoning processes, rather than direct input-to-output mappings, are the appropriate framework for deploying LLMs on complex tasks.

Yao et al. [24] extend this insight through the ReAct framework, which interleaves reasoning traces with actions that invoke external tools, creating a cycle of thought, action, and observation that allows LLMs to gather information dynamically as they

reason. In the ReAct paradigm, an LLM reasoning about whether a network flow is malicious might think through the significance of observed protocol characteristics, decide that it needs to look up the official assignment for a particular port number, execute a tool call to retrieve that information, incorporate the retrieved fact into its ongoing reasoning, and continue the analysis with an enriched knowledge base. This paradigm has been shown to substantially improve performance on knowledge-intensive tasks relative to Chain-of-Thought prompting alone, as tool access allows the model to ground its reasoning in retrieved facts rather than relying on potentially stale or imprecise parametric knowledge. The practical challenge for tool-augmented NIDS is identifying which tools provide genuine value: tools that retrieve information from external threat intelligence databases are only useful if the relevant entities (IP addresses, domain names, malware hashes) are present in those databases, a condition that fails systematically on anonymised or synthetic datasets.

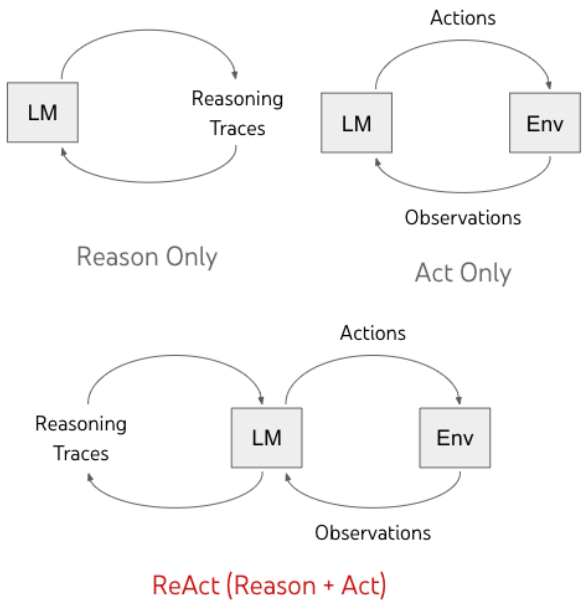

Figure 2.4: The ReAct framework compared to reasoning-only and acting-only paradigms. Reasoning-only (left) generates thought traces without environment interaction; acting-only (centre) takes actions without explicit reasoning. ReAct (bottom) synergises both, interleaving reasoning traces with actions and observations to enable grounded, iterative problem-solving. Source: Yao et al. [24].

Schick et al. [25] formalise tool augmentation through Toolformer, a method for training language models to autonomously decide when and how to call external APIs. Toolformer demonstrates that tool-use capability can be taught through self-supervised learning, with models learning to insert API calls into their generation

process in positions where the retrieved information reduces prediction loss. This work established the viability of tool-augmented LLMs as broadly applicable systems, though Schick et al. note constraints including the inability to chain tools within a single reasoning step, sensitivity to tool description wording, and limited sample efficiency. Toolformer nonetheless demonstrated tool-augmented LLMs as viable beyond narrowly specialised applications, and motivated subsequent research into rich tool ecosystems for LLM-based agents. Lewis et al. [26] provide a complementary contribution through Retrieval-Augmented Generation (RAG), which augments LLM inference with dynamic retrieval from a document corpus, allowing models to access knowledge that was not present at training time or that has changed since the training cutoff. RAG has been applied to cybersecurity tasks including vulnerability assessment, malware analysis, and threat intelligence summarisation, where the need for up-to-date knowledge about recently discovered threats makes static parametric knowledge insufficient.

The Model Context Protocol, introduced by Anthropic [4], provides a standardised interface for connecting LLM agents to external tools, data sources, and services. MCP defines a server-client architecture in which tool providers expose their capabilities through a standardised JSON-RPC protocol, and LLM clients discover available tools, invoke them with structured arguments, and incorporate results into their reasoning context. The significance of MCP for intrusion detection research is that it enables controlled ablation studies: by defining a set of candidate tools and evaluating configurations with and without each tool, researchers can isolate the contribution of specific external information sources to detection performance. The present work employs an MCP-based architecture in its comparison experiments precisely to answer the question of whether external threat intelligence tools (IP reputation databases such as AbuseIPDB, threat feeds from AlienVault OTX, geolocation services) contribute to detection performance on the CICIDS2018 dataset, whose anonymised IP addresses fall in private ranges that do not appear in any external intelligence source. The expected finding, motivated by Qin et al. [27] comprehensive survey of tool learning for language models, is that tool utility is highly dependent on alignment between the tool's knowledge base and the properties of the data being analysed: tools calibrated for real-world public IP addresses provide zero marginal value on traffic from anonymised private ranges.

The MCP comparison experiments conducted in this thesis are designed to make this expected null result rigorous: by holding model capability constant across zero-shot, prompt-engineered, and tool-augmented configurations, and varying only the availability of external tools and the quality of the system prompt, it becomes possible to attribute performance differences unambiguously to those specific interventions. This ablation design, informed by the tool learning survey methodology of Qin et al.

[27], provides a controlled empirical basis for the architectural decision to prioritise multi-agent reasoning over tool augmentation in the AMATAS design.

The trajectory from MCP-centric tool orchestration to multi-agent debate reflects a broader pattern emerging in the LLM security literature: external tools add value only when the data under analysis falls within the tool's coverage domain. For anonymised benchmarks, private address spaces, and synthetic traffic, the tool-augmentation paradigm encounters a hard ceiling that no amount of prompt engineering can overcome. The multi-agent architecture pursued in AMATAS represents an alternative approach to enriching LLM reasoning (through structured disagreement among diverse analytical perspectives rather than through retrieval of external facts), and the comparative evaluation reported in Chapter 5 quantifies the performance gap between these two strategies.

2.5 Explainable AI for Network Security

Explainability in machine learning has become a central concern across application domains as regulations such as the European Union's General Data Protection Regulation and AI Act have introduced requirements for meaningful explanations of automated decisions affecting individuals. In network security, while regulatory requirements are less direct, operational constraints dictate that security analysts must process intrusion detection alerts in real time. The efficacy of this response relies heavily on the system's capacity to provide interpretable classification rationales alongside its verdicts.

The dominant post-hoc explanation methods in the machine learning literature are SHAP (SHapley Additive exPlanations, [15]) and LIME (Local Interpretable Model-Agnostic Explanations, [16]). Both approaches attempt to approximate the behaviour of a complex classifier in the neighbourhood of a specific prediction using a simpler, interpretable surrogate model. SHAP assigns each input feature a contribution value derived from cooperative game theory, quantifying how much that feature shifted the prediction relative to a baseline. LIME fits a local linear model to the classifier's behaviour around the prediction point, using the linear model's coefficients as proxies for feature importance. Both methods have been applied across machine learning domains and are increasingly adopted in security research to interpret classifier decisions. In the intrusion detection context, knowing that packet length variance and backward packet rate were the primary drivers of a malicious classification gives an analyst more to work with than the raw classification probability. Related work on explainability for intrusion detection includes the adversarial perturbation approach of Marino, Wickramasinghe and Manic [28], which identifies the minimum feature modifications needed to correct misclassifications,

thereby revealing which features are most responsible for classification errors.

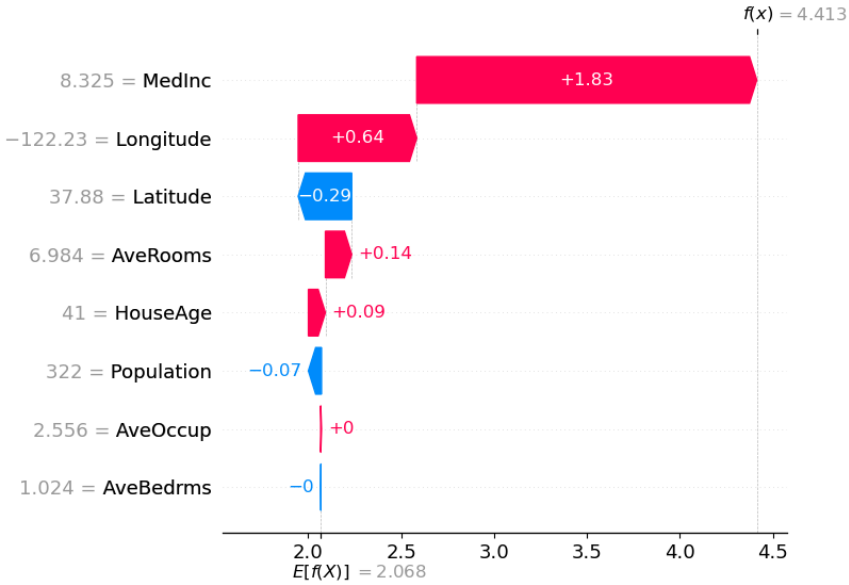

Figure 2.5: A SHAP waterfall plot showing per-feature contributions to a single prediction. Each bar indicates how much a feature's value pushes the prediction above (red) or below (blue) the expected baseline $E[f(X)]$. Whilst informative about *which* features were influential, such explanations do not articulate *what those features mean* in the context of a specific attack scenario or provide counter-arguments. Source: Lundberg and Lee [15]; figure from the SHAP library documentation.

However, these post-hoc methods carry inherent limitations that restrict their utility for security operations. Feature attribution explains which inputs influenced a prediction but not what those inputs mean in the context of a specific attack scenario. A SHAP value indicating that a high byte-per-packet ratio contributed to a malicious classification does not explain whether this pattern is consistent with a DoS flooding attack, a large-file exfiltration, or a legitimate video streaming session. The analyst must supply this contextual reasoning themselves, drawing on domain knowledge that the explanation method cannot provide. Furthermore, SHAP and LIME explain individual predictions in isolation, without contextualising the flagged flow against other flows from the same source, against known attack progressions, or against the temporal pattern of the session. The temporal and behavioural context that human analysts use to distinguish genuine threats from false positives is systematically absent from feature attribution explanations.

LLM-generated reasoning chains provide contextualised explanations that offer distinct operational utility compared to standard statistical feature attribution methods. When AMATAS produces a malicious classification, it accompanies that verdict with natural language reasoning from four specialist agents, each articulating

why specific observed characteristics are anomalous from their analytical perspective, followed by a Devil's Advocate argument for why the flow might be benign, and an Orchestrator synthesis that reconciles all preceding analyses and explains how the final verdict was reached. Instead of outputting raw feature importance weights, the system generates semantic explanations for anomalous features, such as explicitly identifying when a packet rate exceeds the 99th percentile for a given protocol, when a destination port combination deviates from benign service parameters, when two other flows from the same source in the same time window exhibit the same pattern, and when the Devil's Advocate found no compelling benign explanation despite considering common alternative hypotheses. Unlike SHAP, which provides localised mathematical feature attribution, this multi-agent approach outputs natural language rationales intended to improve analyst comprehension and contextualise the threat.

The comparison between feature attribution and LLM reasoning chains has received limited formal attention in the literature, as the deployment of LLMs for intrusion detection explanations is sufficiently recent that systematic user studies have not yet been published. However, the broader XAI literature on human factors in explanation usefulness [29, 30] consistently finds that textual, narrative explanations are more actionable for non-specialist decision-makers than numerical feature importance scores, and that explanations that surface relevant context (why a particular feature value is significant given what is known about the domain) are more effective than those that simply identify which inputs were most influential. These findings, whilst not specific to network security, support the expectation that LLM-generated reasoning chains will provide greater operational value than post-hoc attribution methods, an expectation that the Flow Inspector component of the AMATAS dashboard is designed to facilitate evaluation of.

2.6 Hybrid Architectures and Cost Optimisation

The economic challenge of deploying LLMs for network intrusion detection at scale is not incidental but structural. At current commercial API pricing, processing a single network flow through a six-agent LLM pipeline costs approximately 0.028 US dollars using GPT-4o. The CICIDS2018 dataset contains approximately twenty million flows. Processing all twenty million flows through the full AMATAS pipeline would cost approximately 560,000 US dollars, a figure that renders the approach economically infeasible for any organisation without exceptional security budgets. Even at projected future reductions in inference cost, the order-of-magnitude discrepancy between the cost of LLM analysis and the cost of traditional classifier inference means that pure LLM intrusion detection is not a viable replacement for traditional

approaches; it must be positioned as a complement deployed selectively on the subset of traffic that warrants detailed analysis.

To optimise computational overhead, recent systems have implemented a tiered architecture where a lightweight statistical classifier filters benign traffic, reserving resource-intensive LLM inference exclusively for anomalous flows. Kalafatidis et al. [31] describe a two-tier architecture for software-defined networking and Kubernetes environments in which lightweight statistical anomaly detection models operating on container monitoring data and network flow statistics provide the first detection layer, triggering on-demand LLM-based deep analysis only when anomalies are detected. Their evaluation demonstrates that the statistical pre-filter successfully identifies the majority of benign traffic with high confidence, substantially reducing the load on the LLM analyser without significant reduction in attack recall. This independent discovery of the same two-tier pattern — cheap statistical pre-filtering followed by expensive LLM analysis — provides external validation for the design implemented in AMATAS, and suggests that the approach represents a natural convergence point for researchers working on the cost-accuracy trade-off in LLM-based security systems. The LLM evaluation in Kalafatidis et al. was tested on only 200 packet captures, with minor detection failures on HTTP/2 attacks due to insufficient PCAP sample lengths, indicating a constrained evaluation scope relative to the present work.

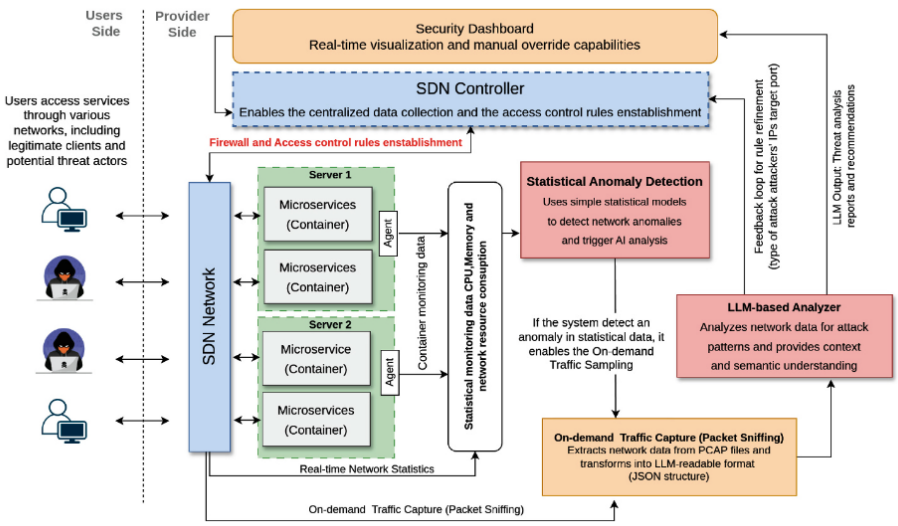

Figure 2.6: The two-tier LLM-enhanced intrusion detection architecture for SDN and Kubernetes environments. Statistical anomaly detection provides continuous monitoring (Tier 1), triggering on-demand LLM-based deep analysis (Tier 2) only when anomalies are detected. This independent design mirrors the AMATAS two-tier approach of cheap ML pre-filtering followed by expensive LLM analysis. Source: Kalafatidis et al. [31].

The choice of operating threshold for the pre-filter presents a fundamental tension between cost and recall. A conservative threshold that allows many uncertain flows through to the LLM tier preserves recall but reduces cost savings; an aggressive threshold that filters more flows reduces cost but risks missing attacks that the Random Forest assessed as borderline benign. AMATAS prioritises recall during the pre-filtering stage by configuring the Tier 1 Random Forest with a maliciousness probability threshold of 0.15. Consequently, any flow exceeding this threshold is forwarded to the LLM pipeline for further analysis. At this threshold, the classifier achieves one hundred per cent recall on attack flows in the development partition (no attack is lost to pre-filtering) whilst correctly filtering approximately ninety-five per cent of benign flows away from the expensive analysis pipeline. The operating threshold was selected empirically on the development dataset rather than on validation or test data, preserving the integrity of the held-out evaluation partitions.

The availability of standardised, large-scale NetFlow datasets has been an important enabling factor for both the machine learning and LLM-based intrusion detection research communities. Sarhan et al. [32] describe the generation and characteristics of standardised NetFlow-based variants of major NIDS datasets, including the NF-CSE-CIC-IDS2018-v2 dataset, which converts the packet-capture data of the original CSE-CIC-IDS2018 dataset into flow records using an extended NetFlow feature set. This conversion is significant because NetFlow is the format in which most enterprise network monitoring infrastructure actually exports traffic data; classifiers and detectors evaluated on NetFlow records have a more direct path to deployment than those evaluated on packet-level features that require full packet capture infrastructure. The CICIDS2018 NetFlow dataset used in this thesis contains approximately twenty million flows covering fourteen attack categories, providing a substantially larger and more diverse evaluation basis than most LLM-NIDS studies, which typically evaluate on hundreds rather than thousands of flows per attack type.

2.7 Research Gap

A synthesis of the preceding review reveals a research landscape with three well-developed areas (traditional ML-based intrusion detection, LLM applications in cybersecurity, and multi-agent LLM architectures) that have not yet been integrated into a coherent system that addresses the full set of challenges facing practical LLM-based NIDS deployment. Each area has made substantial progress within its own boundaries, but the intersections remain underexplored and the combination of techniques required for a practically viable, explainable, cost-efficient multi-agent NIDS has not been previously assembled and evaluated.

The first gap concerns the integration of multi-agent debate with intrusion detection. Du et al. [7], Chan et al. [8], and related work establish that multi-agent debate improves reasoning quality in general LLM tasks, and Li et al. [20] demonstrate that agentic LLM architectures improve intrusion detection performance compared to single-pass classification. However, these threads have not been combined: no prior work applies a structured multi-agent debate architecture (with distinct specialist roles, adversarial challenge, and consensus synthesis) to network intrusion detection. The question of whether the diversity and adversarial properties that make multi-agent debate beneficial in general reasoning tasks translate to the domain-specific, feature-intensive task of intrusion detection remains open.

The second gap concerns false positive control in LLM-based detection. The literature on LLM-NIDS consistently identifies false positive rate as a critical metric; Houssel et al. [5] find that GPT-4 exhibits systematic bias toward certain classifications, and Li et al. [20] note that their single-agent IDS-Agent architecture produces more false positives on the CIC-IoT dataset than on ACI-IoT. No prior work introduces a structural mechanism for false positive control in LLM-based intrusion detection. The present work addresses this gap through the Devil's Advocate agent, whose explicit mandate to construct the strongest possible benign interpretation of any flagged flow provides adversarial pressure against over-classification. The effectiveness of this mechanism, measured by the near-zero false positive rate achieved across the Stage 1 evaluation, constitutes a novel empirical finding with direct practical relevance.

The third gap concerns the integration of ML pre-filtering with multi-agent LLM analysis for cost reduction. The economic infeasibility of pure LLM intrusion detection is well understood but has received little systematic attention in the LLM-NIDS literature, where evaluation datasets are typically small enough that cost is not a binding constraint. The two-tier architecture of Kalafatidis et al. [31] addresses this problem in an SDN context with statistical anomaly detection triggering a single LLM analyser, but does not employ multi-agent debate, specialist role diversity, or adversarial false positive control in the second tier. No prior work presents an empirically validated two-tier architecture that combines an ML pre-filter for cost reduction with a *multi-agent* LLM pipeline for detection and explainability, with cost measured rigorously across realistic class distributions and multiple attack types.

The fourth gap concerns evaluation methodology. The majority of LLM-based NIDS studies evaluate on small samples (typically dozens to a few hundred flows) selected from balanced or near-balanced class distributions that do not reflect the operational reality of network traffic. AMATAS is evaluated on one thousand flows per attack type at a five per cent attack prevalence, matching the realistic class distributions that deployed systems would encounter. Per-attack-type evaluation

at this granularity, spanning all fourteen CICIDS2018 attack categories, enables a systematic mapping of LLM-based detection performance across the attack taxonomy that has not previously been published.

In aggregate, no existing published system combines the four components that together define AMATAS's contribution: a multi-agent LLM debate architecture with specialist role diversity, an adversarial Devil's Advocate mechanism for false positive control, a machine learning pre-filter for cost reduction, and per-attack-type evaluation at realistic class distributions on the CICIDS2018 NetFlow v3 benchmark. The present thesis presents AMATAS as the first system to integrate these components, and the subsequent chapters report the empirical results that establish both the capabilities and the limitations of this integrated architecture.

Chapter 3

System Architecture

3.1 Design Philosophy

The architecture of AMATAS departs from the prevailing paradigm of network intrusion detection as a single-model classification problem. Rather than training one classifier to distinguish benign from malicious traffic, AMATAS distributes the analytical burden across specialised reasoning agents, each attending to a distinct dimension of the evidence space, whose outputs are then reconciled by a consensus mechanism that explicitly models disagreement. This design is based on the hypothesis that distributed deliberation among diverse models improves classification accuracy on ambiguous data compared to the direct output of a single classifier.

The conceptual foundation for this approach draws on Minsky [6] Society of Mind framework, which proposes that intelligence emerges not from any individual reasoning process but from the interaction and negotiation of many smaller specialised agents. Applied to network intrusion detection, this principle motivates a division of analytical labour along the natural fault lines of the problem domain: protocol-layer anomalies, statistical distributional shifts, behavioural pattern matching against known attack signatures, and temporal correlations across flows from the same source. Each of these dimensions demands different domain knowledge, different feature attention, and different reasoning heuristics. Consolidating these tasks into a single prompt increases context complexity, which can degrade the model's performance and accuracy on specialised flow analysis tasks. Separating them into dedicated agents, each with a system prompt tuned for its particular analytical mode, allows each agent to reason with the focused rigour that its subdomain warrants.

Empirical support for multi-agent deliberation in LLM contexts comes from Du et al. [7], who demonstrated that when multiple LLM instances debate a question across several rounds, the accuracy of the final answer significantly exceeds that of any individual instance. Chan et al. [8] extended this finding by showing that

role diversity among debating agents, rather than merely numerical redundancy, is the critical variable: agents assigned distinct analytical perspectives outperform agents assigned identical roles. These findings translate directly to the AMATAS design, where the four specialist agents are not interchangeable replicas but genuinely different analytical characters operating on different subsets of the feature space. Both Du et al. and Chan et al. note that multi-agent debate can converge on confidently incorrect answers when agents share initial misconceptions; AMATAS mitigates this through the Devil's Advocate mechanism (Section 3.4), which provides a structurally independent counter-voice not subject to the same convergence dynamics.

The adversarial dimension of the architecture is grounded in a distinct body of literature concerned with the failure modes of consensus under social pressure. When agents reason sequentially and share intermediate conclusions, later agents tend to anchor on earlier ones, producing a cascade of agreement that amplifies initial errors rather than correcting them [33]. Sunstein qualifies this as a "statistical regularity" rather than an inevitability, noting that views formed after extensive deliberation are least susceptible to polarisation. The AMATAS design treats polarisation as a sufficiently frequent risk to warrant structural mitigation rather than relying on specialist diversity alone to prevent it. AMATAS addresses this through the Devil's Advocate agent, a dedicated adversarial voice that receives all specialist findings simultaneously and is explicitly instructed to construct the strongest possible benign interpretation of the evidence, regardless of the agent's own internal assessment. Chiang et al. [9] showed that LLM-powered devil's advocate roles measurably improve group decision quality and reduce inappropriate reliance on confident but incorrect assessments. In the NIDS context, where the cost of a false positive (interrupting a legitimate business process, eroding analyst trust in the system, generating compliance risk) is substantial, this adversarial check provides a principled structural mechanism for false positive suppression rather than relying solely on post-hoc threshold tuning.

The result is an architecture with two orthogonal quality mechanisms: specialist depth, which increases the likelihood of detecting genuine attacks, and adversarial validation, which mitigates false positive classifications. The Orchestrator agent at the apex of the pipeline then faces a well-posed synthesis problem: it receives evidence and counter-evidence from parties with explicit, different mandates, and is required to adjudicate between them according to explicit consensus rules. This architecture functions as an evidence-weighing mechanism rather than a standard ensemble vote, explicitly preserving the reasoning chain at each stage.

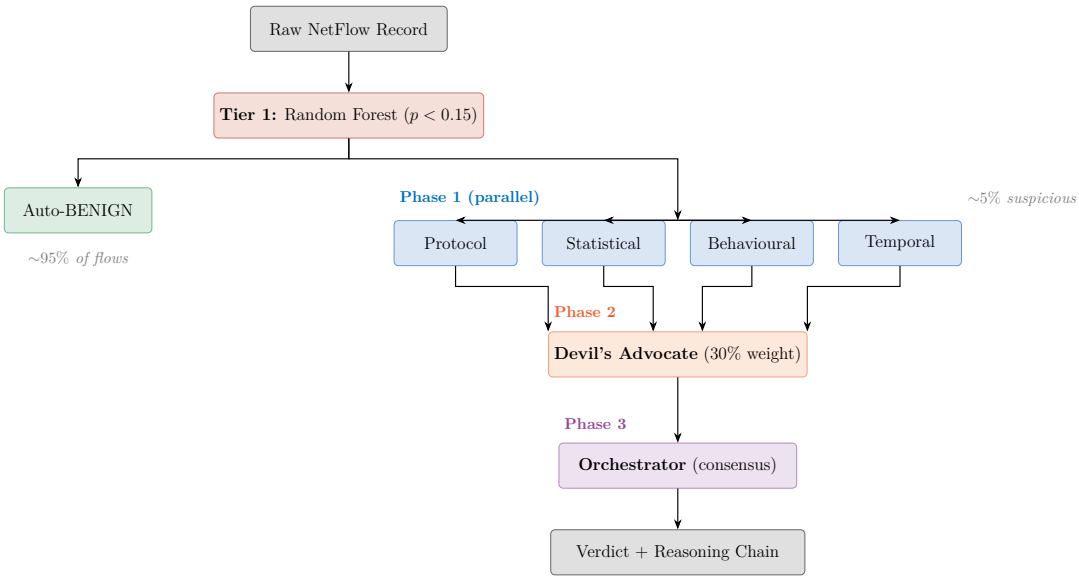

Figure 3.1: AMATAS two-tier architecture. Tier 1 filters ∼95% of flows as auto-benign; remaining flows pass through the three-phase LLM pipeline.

3.2 Tier 1: Random Forest Pre-Filter

The economic case for LLM-based intrusion detection depends entirely on solving a cost problem that pure LLM pipelines cannot solve alone. At the pricing applicable during Stage 1 experiments (approximately US$0.028 per flow processed through the full six-agent pipeline), analysing the complete CICIDS2018 dataset of twenty million flows would cost in excess of half a million dollars. This computational cost limits production viability. While LLM reasoning is most beneficial for ambiguous cases, applying it uniformly to all traffic, which is predominantly benign, is computationally and financially inefficient.

The Tier 1 pre-filter resolves this tension by delegating the easy cases to a cheap, fast machine learning classifier and reserving the expensive LLM pipeline for flows that the classifier cannot confidently resolve. The classifier of choice is a Random Forest, a well-understood ensemble method with strong empirical performance on structured tabular data, fast inference, and a natural probabilistic output in the form of class probability estimates derived from voting across an ensemble of decision trees [34]. A Random Forest can process hundreds of thousands of flows per second on commodity hardware, making it suitable for the high-throughput pre-screening role that Tier 1 requires.

3.2.1 Feature Set and Training Data

The classifier is trained on twelve numeric features extracted from the CICIDS2018 NetFlow v3 dataset:

> `L4_SRC_PORT`, `L4_DST_PORT`, `PROTOCOL`, `FLOW_DURATION_MILLISECONDS`, `SRC_TO_DST_IAT_MAX`, `DST_TO_SRC_IAT_MAX`, `IN_BYTES`, `OUT_BYTES`, `IN_PKTS`, `OUT_PKTS`, `TCP_FLAGS`, `DNS_QUERY_ID`

This feature set was chosen through a combination of variance analysis and domain considerations. Features with near-zero variance across the dataset provide no discriminative signal and were excluded. The `FLOW_DIRECTION` field, present in other NetFlow variants, is unavailable in NetFlow v3 and could not be included. `DNS_QUERY_ID` is retained despite being zero for the majority of non-DNS flows, as its presence provides discriminative information for DNS-tunnelling and related attack patterns.

The training data is drawn from the development partition of the CICIDS2018 dataset, comprising 7.04 million flows, which was further stratified into an 80 per cent training set of 5,632,348 flows and a 20 per cent evaluation holdout of 1,408,087 flows. This internal partitioning was necessitated by a data integrity concern identified during the Stage 1 evaluation phase. Seven of the fourteen attack types in the evaluation programme are present in the development CSV, meaning that if evaluation batches were sampled from the same file on which the Random Forest was trained, the classifier's filtering accuracy for those attack types would be inflated by its prior exposure to the same flow distributions. To eliminate this within-split overlap, the RF is trained exclusively on the 80 per cent partition, and evaluation batches for development-resident attack types are sampled exclusively from the 20 per cent holdout. The remaining seven attack types, sourced from the validation and test splits that the RF never encountered during training, require no modification. This partitioning ensures that the Tier 1 evaluation reflects genuine generalisation rather than dataset memorisation.

The Random Forest is configured with one hundred estimators, a maximum tree depth of twenty, a minimum of ten samples per leaf, balanced class weights to account for the strong class imbalance in the training data (approximately eighty-seven per cent benign flows), and a fixed random seed of 42 for reproducibility. Training on the 5.6 million row training partition completes in several hours on a standard multi-core machine.

3.2.2 Threshold Selection

The Tier 1 classifier produces, for each flow, an estimated probability that the flow is malicious, derived by averaging the proportion of attack-class votes across all one

hundred trees. This probability is compared against a fixed threshold of 0.15. Flows where the estimated attack probability falls below 0.15 are classified as auto-BENIGN and routed away from the LLM pipeline without further analysis. Flows at or above the threshold are forwarded to Tier 2.

The threshold of 0.15 was selected to satisfy an asymmetric optimisation objective: the Tier 1 filter must achieve one hundred per cent recall on attack flows (no attack may be discarded before reaching the LLM pipeline) whilst maximising the proportion of benign flows filtered out. This is the recall-priority constraint appropriate for a security application where false negatives carry the highest cost. At the 0.15 threshold, the RF achieves one hundred per cent recall on the evaluation holdout, with the minimum estimated attack probability across all true attack flows of 0.64. The gap between the threshold (0.15) and the minimum true-attack probability (0.64) provides a substantial safety margin: even if the RF were to slightly miscalibrate its probability estimates on novel flow distributions, attacks would still fall well above the filter threshold. Approximately ninety-five per cent of benign flows are assigned probabilities below 0.15, yielding a filtering rate that reduces LLM API costs by the same proportion.

It bears emphasis that the Tier 1 filter carries a deliberate asymmetry. False negatives at this stage (benign flows forwarded to the LLM pipeline) carry a cost measured in dollars and milliseconds. False positives at this stage (attack flows filtered before LLM analysis) carry a cost measured in missed detections, a fundamentally different and unacceptable failure mode. The threshold of 0.15, far below the intuitive operating point of 0.50, reflects this asymmetry explicitly and structurally.

3.3 Tier 2: Specialist Agents

Flows that survive the Tier 1 filter proceed to the six-agent LLM pipeline, which is organised into three sequential phases. The first phase comprises four specialist agents that execute in parallel, each receiving the same flow record and returning an independent structured analysis. This parallel architecture is both practically efficient (all four API calls are dispatched simultaneously, with wall-clock latency determined by the slowest single call rather than their sum) and analytically important, as it ensures that each specialist reasons from the raw flow data without being influenced by any other agent's intermediate conclusions.

All agents in the pipeline are implemented using GPT-4o, selected after empirical comparison with GPT-4o-mini and Claude Sonnet on a one-hundred-flow validation batch. GPT-4o-mini was rejected after demonstrating a systematic classification bias that produced one hundred per cent false positive rates across the validation set, classifying all flows as malicious regardless of evidence. Claude Sonnet was rejected

on operational grounds: its rate limits were insufficient to process one-thousand-flow evaluation batches without prohibitive delays. GPT-4o provided the best trade-off between detection accuracy, false positive control, and throughput at its operating price of US$2.50 per million input tokens and US$10.00 per million output tokens.

Each specialist agent is structured around a system prompt that defines the agent's analytical mandate, the specific features it should prioritise, the pattern vocabulary it should employ, and the JSON schema it must produce. The JSON output requirement is explicit in every system prompt and is enforced at the application layer through parsing logic with retry on malformed responses. Every specialist output contains a verdict (one of BENIGN, SUSPICIOUS, or MALICIOUS), a confidence score on the unit interval, a predicted attack type or null, a list of key findings expressed as evidence strings, a full reasoning narrative, and token consumption and cost data for cost accounting.

Protocol Agent. The Protocol Agent's analytical mandate centres on the consistency and validity of the protocol-layer features in the flow record. It examines whether the source and destination ports conform to expected service assignments, whether TCP flag combinations are syntactically valid and semantically coherent for the connection phase they imply, and whether the observed protocol number matches the port and behaviour patterns associated with that protocol. A connection from an ephemeral source port to destination port 21 bearing SYN flags repeated dozens of times within seconds presents a very different protocol picture from a single long-lived FTP control session, and the Protocol Agent is specifically prompted to reason about these distinctions. Port anomalies (traffic to well-known service ports during off-hours, use of non-standard ports for common protocols, combinations of ports associated with specific attack toolkits) are weighted heavily in the Protocol Agent's verdict.

Statistical Agent. The Statistical Agent focuses on the volumetric and timing characteristics of the flow: byte counts in both directions, packet counts, inter-arrival time extremes, and flow duration. Its analytical frame is distributional rather than rule-based. Byte volumes that fall several orders of magnitude above or below typical traffic for the observed protocol, packet counts that imply either atomic single-packet probes or sustained floods, inter-arrival times that collapse to near-zero (indicative of automated generation) or stretch to extreme values (indicative of slow-rate attacks); these distributional aberrations are the primary signal the Statistical Agent is trained to detect. The agent is instructed to reason about what typical traffic of the observed type would look like, and to articulate why the observed values deviate from that expectation, ensuring that its reasoning is anchored in domain knowledge rather than purely in numerical thresholds.

Behavioural Agent. The Behavioural Agent operates at the pattern-matching

layer, correlating the full feature profile of the flow against a structured vocabulary of known attack signatures derived from intrusion detection research and mapped to the MITRE ATT&CK framework [35]. Its prompt enumerates the characteristic behavioural signatures of each attack category present in the CICIDS2018 dataset: the repetitive authentication failure pattern of brute-force attacks, the zero-response-body high-rate request pattern of HTTP floods, the asymmetric byte-volume signature of SYN floods, the slow-rate low-bandwidth pattern of Slowloris attacks. When the Behavioural Agent identifies a match, it returns the corresponding MITRE ATT&CK technique identifiers alongside its verdict, enabling downstream mapping of detections to the standardised adversary behaviour ontology that security teams use for threat reporting and incident response.

Temporal Agent. The Temporal Agent is architecturally distinct from the other three specialists in that it does not analyse the target flow in isolation. Before invoking the Temporal Agent, the pipeline queries the current batch for all other flows sharing the same source IP address, extracts a compact summary of each co-IP flow, and injects this context into the agent's input. The Temporal Agent therefore reasons about the target flow against the backdrop of the source IP's complete observed behaviour within the batch, identifying patterns (burst activity, sequential escalation from scanning to exploitation, repetitive connection signatures indicative of automated tools, fan-out patterns suggesting lateral movement) that are invisible at the level of individual flows.

This contextual enrichment comes at a cost. The Temporal Agent's input tokens are substantially larger than those of the other specialists, since the co-IP flow summaries can add hundreds or thousands of tokens depending on how active the source IP is within the batch. This makes the Temporal Agent the most expensive single component in the pipeline, accounting for approximately thirty per cent of total per-flow LLM expenditure. Despite this cost, the Temporal Agent is retained as a non-optional pipeline stage because the attack patterns it is uniquely positioned to detect (particularly brute-force sequences and systematic port sweeps) are among the most practically significant in the CICIDS2018 dataset, and no other agent in the pipeline has access to cross-flow context.

3.4 The Devil's Advocate Mechanism

Following the completion of the four parallel specialist analyses, the pipeline enters its second phase with the invocation of the Devil's Advocate agent. This agent occupies an unusual position in the architecture: unlike the specialists, whose mandate is to characterise the flow as accurately as possible from their particular analytical angle, the Devil's Advocate's mandate is explicitly adversarial and one-directional. It is

instructed to argue for the benign interpretation of the flow regardless of the weight of specialist evidence to the contrary.

The theoretical justification for this asymmetric mandate draws on the literature on group decision-making under uncertainty. When a group of expert agents, human or artificial, is presented with a question and reaches a preliminary consensus, subsequent deliberation tends to reinforce rather than challenge that consensus, a phenomenon known as group polarisation [33]. In the AMATAS context, if three of four specialists return MALICIOUS verdicts, a naive synthesis agent that simply weighted these verdicts would be strongly predisposed toward a malicious classification before the evidence had been properly stress-tested. The Devil's Advocate agent mitigates group polarisation by systematically generating counter-arguments; it receives all four specialist outputs simultaneously and computes alternative benign explanations for each finding that the specialists cite as suspicious.

Chiang et al. [9] provide the most directly relevant empirical grounding for this design, demonstrating in a series of human-AI decision experiments that an LLM playing an explicit devil's advocate role measurably improved the quality of group decisions, reduced inappropriate reliance on confident AI assessments, and increased the probability that correct but minority-held positions received adequate consideration before a final verdict. The effectiveness of devil's advocate interventions may vary with task difficulty, a possibility the Stage 1 ablation study (Section 5.8) directly examines, revealing that the DA's contribution is indeed non-uniform across attack categories of differing ambiguity. Their findings apply with particular force in the false positive problem of intrusion detection: the cost of classifying a legitimate business flow as malicious (blocking or alerting on an innocent connection, potentially disrupting operations) is substantial and asymmetric relative to the cost of a missed detection in many operational contexts.

The Devil's Advocate agent generates four structured outputs. The `benign_argument` field contains a full-text argument for the benign interpretation of the flow, engaging specifically with the evidence cited by the specialists. The `confidence_benign` field encodes a normalised score from zero to one representing the strength of the benign case as the Devil's Advocate assesses it. The `alternative_explanations` list provides a point-by-point rebuttal of each specialist finding, offering a plausible innocent explanation for each cited anomaly. The `strongest_benign_indicator` field identifies the single most compelling reason that the flow might be benign, which the Orchestrator uses as a key input to its weighting of the Devil's Advocate voice.

The weight assigned to the Devil's Advocate in the final consensus is thirty per cent. This figure was determined empirically through a phase of weight calibration experiments. At fifty per cent Devil's Advocate weight, the adversarial counter-arguments were sufficiently influential to override clear specialist consensus,

producing a systematic degradation in recall as genuinely malicious flows were reclassified as benign under the pressure of well-constructed alternative explanations. At thirty per cent, the Devil's Advocate retains sufficient influence to prevent false positives when specialist consensus is weak or founded on borderline evidence, whilst strong agreement among three or four specialists is robust enough to withstand even a confident benign argument. This calibration is empirically validated across the Phase 3 experimental series and held constant across all Stage 1 production experiments.

3.5 Orchestrator and Consensus

The Orchestrator agent, the final stage of the pipeline, receives the structured outputs of all five preceding agents (four specialists and the Devil's Advocate) and synthesises them into a single final verdict accompanied by a full reasoning narrative. Its role is not to add new analysis of the flow data itself, but to perform the adjudicative function of weighing competing evidence and resolving conflicts according to explicit, auditable rules.

The consensus rules encoded in the Orchestrator's system prompt implement a threshold-based voting scheme with Devil's Advocate counterweighting. When all four specialists return MALICIOUS verdicts and the Devil's Advocate's `confidence_benign` is low, the Orchestrator returns MALICIOUS with high confidence. When three of four specialists return MALICIOUS and the Devil's Advocate argument is moderate, the result remains MALICIOUS but at reduced confidence. When three of four specialists return MALICIOUS but the Devil's Advocate raises a strong alternative explanation, the Orchestrator may downgrade the verdict to SUSPICIOUS. When only two specialists return a MALICIOUS verdict, the Orchestrator assigns a SUSPICIOUS classification unless the specialists' confidence scores exceed a defined threshold. A single MALICIOUS specialist vote is insufficient for a malicious classification in most circumstances, and unanimous BENIGN verdicts produce a BENIGN classification without LLM ambiguity. The SUSPICIOUS verdict category serves as an important intermediate state, capturing flows where the evidence is genuinely ambiguous; in operational deployment, SUSPICIOUS flows would be escalated for human analyst review rather than automatically blocked or ignored.

The Orchestrator also computes a `consensus_score`, a normalised measure of specialist agreement, by examining the proportion of specialists whose verdicts align with the final classification. This score provides a machine-readable signal of how confident the ensemble is in its conclusion, separate from the verdict-level confidence score, which reflects the Orchestrator's calibrated belief in the correctness of the verdict given all available evidence. The `mitre_techniques` field in the

Orchestrator output consolidates all MITRE ATT&CK technique identifiers cited by the Behavioural Agent and any other specialists that reference the framework, producing a unified list of applicable techniques that security operations teams can use to contextualise the detection within their threat intelligence workflows.

Critically, the Orchestrator's reasoning field is not a summary of the preceding analyses but a synthesis that cites specific findings from specific agents, articulates the evidential basis for the verdict, identifies which agents agreed and which dissented, and explains why dissenting views were or were not dispositive. This full reasoning chain, preserved in the output record for every flow processed through the pipeline, is the primary deliverable from the explainability perspective that motivates the system's design. Unlike a traditional classifier, which produces a numerical probability that offers no foothold for analytical scrutiny, the Orchestrator's output can be read by a security analyst, challenged, corrected, and used as the basis for a documented investigation narrative, providing a traceable decision record that aligns with modern security governance requirements.

3.6 Cost Architecture

The economic structure of the Tier 2 pipeline reflects the design decisions described above, with the Temporal Agent's contextual enrichment producing a cost distribution that departs markedly from the even split one might expect across six nominally equivalent agents. Empirical cost data collected across the Stage 1 evaluation experiments reveals that the Temporal Agent accounts for approximately thirty per cent of the total per-flow LLM expenditure, consuming substantially more input tokens than any other agent due to the co-IP flow context injected into its prompt.

The Orchestrator agent is the second most expensive component, at approximately 21.3 per cent of total per-flow cost. This elevated cost reflects the Orchestrator's receipt of all five preceding analyses, each of which may contain several hundred tokens of reasoning text, making its input context the largest of any single agent in the pipeline. The Devil's Advocate follows at approximately 18.5 per cent, since it similarly receives all four specialist outputs. The three remaining specialists (Behavioural, Statistical, and Protocol) operate on the flow data alone and share the remaining thirty per cent approximately evenly, with the Behavioural Agent at 10.4 per cent, the Statistical Agent at 10.1 per cent, and the Protocol Agent at 9.7 per cent. The Behavioural Agent's marginally higher cost reflects the richer attack signature vocabulary in its system prompt, which generates longer input contexts even without additional flow context.

The practical implication of this cost distribution is that the Temporal Agent is the highest-return target for cost optimisation in future iterations of the system. If

the contextual enrichment mechanism could be made more selective (providing co-IP context only when the source IP has an unusually high flow count, or compressing the co-IP summaries more aggressively for large groups), the overall per-flow cost could be reduced without fundamentally altering the pipeline's analytical capabilities. Conversely, the three lightweight specialists contribute disproportionately to analytical diversity relative to their cost share, suggesting that their inclusion is strongly justified on cost-effectiveness grounds even if reducing the pipeline to fewer agents were otherwise desirable.

The combined effect of Tier 1 filtering and the Tier 2 cost structure is a system where the effective cost per raw network flow (across all flows entering the system, including those filtered at Tier 1) is approximately five per cent of the per-flow LLM cost. At the observed Stage 1 processing cost of approximately US$0.028 per flow forwarded to Tier 2, the blended cost across all flows is approximately US$0.0014 per raw flow, a reduction of approximately ninety-five per cent relative to applying the full LLM pipeline uniformly. This blended cost remains non-trivial at enterprise scale, but it represents the threshold of practical viability in a way that unfiltered LLM analysis does not.

3.7 Temporal Clustering Module

The two-tier architecture described above constitutes the AMATAS v2 system, which is the primary subject of the Stage 1 evaluation experiments. The v3 system, currently in the design phase, extends this architecture with a temporal clustering module inserted between Tier 1 filtering and the Tier 2 LLM pipeline. The motivation for this extension is a hypothesis grounded in the performance profile observed across Stage 1 attack categories.

Certain attack types that operate by generating individually ambiguous flows (most notably DoS-Hulk and Infiltration) present a fundamental challenge to flow-level LLM analysis. Each individual flow, examined in isolation or even in the context of other flows from the same source IP within a single batch, may present no clearly anomalous features. The malicious character of these attacks emerges only when flows are examined in aggregate across time: a sustained pattern of high-rate HTTP requests to a single server, each individually plausible but collectively characteristic of a volumetric attack, or a sequence of reconnaissance, credential testing, and lateral movement flows that spans hours or days. The Temporal Agent's current implementation, which groups flows by source IP within a batch, provides partial temporal context but is limited by the batch boundaries imposed by the experimental evaluation framework.

The v3 Temporal Clustering Module addresses this limitation through two com-

plementary clustering modes. In IP-level clustering mode, flows are grouped by source IP address across a configurable time window (tentatively set at thirty minutes for the initial design, with the window length as a hyperparameter to be tuned on the validation set), and the resulting cluster is treated as the unit of analysis rather than the individual flow. The LLM pipeline receives the full cluster as context, enabling the agents to reason about the collective behaviour of a source IP across extended time periods and across the batch boundaries that constrain the current system. In time-windowed clustering mode, flows are grouped by temporal proximity within a sliding window regardless of source IP, capturing patterns of simultaneous activity from multiple sources that may indicate coordinated attacks such as distributed denial of service.

A design consideration specific to the CICIDS2018 dataset is the role of DNS flows in clustering. DNS flows are extremely frequent in enterprise network traffic and are almost always benign, but they share source IPs with application-layer attack flows in ways that inflate cluster sizes and add noise to the temporal context provided to the LLM agents. The v3 design therefore incorporates a DNS threshold override: flows where DNS_QUERY_ID is non-zero (indicating a DNS transaction) are exempt from the clustering mechanism and are routed directly to the standard batch-level Temporal Agent processing unless they themselves exhibit anomalous characteristics. This prevents DNS-heavy source IPs from generating prohibitively large cluster contexts whilst preserving the ability to detect DNS-based attacks such as tunnelling when the DNS traffic itself is anomalous.

The central hypothesis motivating the v3 design is that temporal cluster context will improve recall on attack types where individual flow analysis reaches a ceiling. If a source IP has generated three hundred HTTP requests to the same destination server within a five-minute window, each individually matching the traffic profile of legitimate web browsing, the cluster-level LLM context will reveal the volumetric and temporal pattern that individual flow analysis cannot see. This hypothesis will be tested in the v3 evaluation phase using the held-out test partition of the CICIDS2018 dataset, with comparison against v2 results to isolate the contribution of temporal clustering to detection performance. The expected outcome is an improvement in recall for DoS-Hulk, DoS-Slowloris, and Infiltration, attack categories where the v2 evaluation suggests a fundamental ceiling imposed by the absence of temporal cluster context, without significant degradation in false positive rates, since the cluster context provides the Devil's Advocate with richer benign evidence for flows in genuinely normal clusters.

Chapter 4

Experimental Methodology

4.1 Dataset Description

All experiments reported in this thesis are conducted on the CICIDS2018 dataset, specifically the NetFlow v3 variant published by the Canadian Institute for Cybersecurity [36]. The dataset was generated in a controlled laboratory environment over ten days in February and March 2018, during which a diverse set of attack scenarios was executed against a realistic enterprise network topology comprising victim hosts, a gateway router, and a dedicated attacker infrastructure. Traffic was captured as full packet captures and subsequently converted to bidirectional network flow records using the CICFlowMeter tool, yielding structured telemetry records that capture connection-level statistics without retaining raw packet payloads.

The NetFlow v3 representation used in this work contains 53 features per flow record, encompassing protocol-level attributes (protocol number, source and destination ports, TCP flag counts), volumetric measures (total forward and backward packets, total bytes transmitted in each direction, packet length statistics), timing characteristics (flow duration, inter-arrival times, active and idle durations), and derived rate features (packets per second, bytes per second). This feature set is substantially richer than the PCAP-derived features available in earlier CICIDS editions and provides the LLM agents with sufficient numerical detail to reason meaningfully about protocol conformance, statistical anomalies, and temporal behaviour. Source and destination IP addresses are preserved as private-range addresses in the 172.31.x.x subnet, reflecting the anonymisation applied to the laboratory infrastructure. This anonymisation is consequential for the MCP comparison experiments described in Section 4.5, as it renders external threat intelligence lookups against public IP reputation services uninformative by design.

The complete dataset comprises 20.12 million flow records spanning fourteen labelled attack categories alongside a dominant benign class. The fourteen attack

types are: FTP-BruteForce, SSH-Bruteforce, DoS-SlowHTTPTest, DoS-Slowloris, DoS-Hulk, DoS-GoldenEye, DDoS-LOIC-HTTP, DDoS-LOIC-UDP, DDoS-HOIC, Bot, Infiltration, Brute_Force-Web, Brute_Force-XSS, and SQL_Injection. The class distribution is heavily skewed, with approximately 87 per cent of flows labelled benign and the remaining 13 per cent distributed unevenly across the fourteen attack categories. This skew reflects the general characteristic of enterprise network traffic, where the vast majority of connections are legitimate, though the specific class distribution is shaped by the controlled laboratory conditions under which the dataset was generated. Kenyon, Deka and Bhowmik [37] and Ring et al. [38] caution that traffic generated in academic cyber-ranges may exhibit narrower domain contexts and less behavioural variability than genuine corporate networks, and that flow-level characteristics can differ substantially between laboratory and production environments. The CICIDS2018 distribution should therefore be understood as an approximation of real-world class imbalance rather than an exact reproduction of it. It is precisely this skew that makes naïve evaluation on balanced datasets misleading: a system evaluated on a balanced dataset may exhibit disproportionately high false positive rates when applied to the skewed distribution of real-world traffic, as observed in this work's own pilot experiments with GPT-4o-mini (discussed in Chapter 5).

For the purposes of this research, the dataset is partitioned into three non-overlapping chronological splits. The development partition contains 7.04 million flows and is used for Random Forest training, feature selection, and hyperparameter calibration. The validation partition contains 5.03 million flows and is used for threshold selection, pilot experiments, and intermediate evaluation during system development. The test partition contains 8.05 million flows and is held out for final evaluation, remaining entirely unseen until the concluding experiments reported in Chapter 6. This three-way split reflects the general principle, emphasised by Cawley and Talbot [39], that separate training, validation, and test partitions are essential for obtaining unbiased performance estimates and avoiding selection bias during model development. Cawley and Talbot [39] further advocate for multiple randomised partitionings (e.g., cross-validation or resampling schemes) rather than a single fixed split, since the random sampling of a single partition may arbitrarily favour one model over another. The present work employs a single chronological split rather than cross-validation for two reasons: first, network traffic exhibits temporal structure that random partitioning would disrupt, making chronological splitting more representative of generalisation to future traffic; and second, the computational cost of the six-agent LLM pipeline makes repeated cross-validation infeasible at the current per-flow cost. This design choice means that reported performance figures are conditioned on a single partition and may exhibit variance that multi-seed or

cross-validated evaluation would quantify; this limitation is addressed in Section 6.6.

A subset of 14 features is used as the input feature set for the Random Forest tier-one pre-filter, selected from the full 53-feature set through a combination of variance thresholding and pairwise correlation analysis. One candidate feature, `FLOW_DIRECTION`, was found to be absent from the CICIDS2018 NetFlow v3 schema despite its presence in earlier versions of the dataset; accordingly, the final feature set comprises 14 rather than the originally targeted 15 features. These 14 features include protocol number, source and destination port, packet counts in each direction, byte totals, flow duration, TCP flag fields, and several packet-length statistics. The full feature list is recorded in `data/feature_sets/stage1_features.json` to ensure reproducibility.

4.2 Batch Construction

Each Stage 1 experiment evaluates AMATAS on a batch of 1,000 flow records drawn from the validation partition. The batch composition is fixed at 50 attack flows and 950 benign flows, yielding a five per cent attack prevalence. This composition is deliberately chosen to approximate realistic production conditions. As Sommer and Paxson [40] emphasise, attacks constitute a very small proportion of real network traffic, making the base-rate problem central to any practical deployment. Five per cent attack prevalence represents a conservative but plausible operating point that is considerably more demanding than the balanced benchmarks commonly used in intrusion detection literature. Using a balanced dataset would inflate recall and precision estimates and obscure the false positive behaviour that is the primary practical concern for operational deployments.

Attack flows for each experiment are sampled from the validation partition, restricted to the specific attack type under evaluation. The 50 attack flows are sampled without replacement using a fixed random seed of 42, applied consistently across all experiments to ensure that any given attack type's evaluation is reproducible by any researcher with access to the same dataset partition. The 950 benign flows are drawn from the same partition using the same seed, stratified to avoid temporal concentration in any single time window. The precise flows selected, along with their ground truth labels, are serialised to a `ground_truth.json` file within each batch directory, allowing retrospective verification of any reported result.

Flow ordering within each batch follows the chronological-by-source-IP convention. Flows are first sorted by source IP address and then, within each source IP group, by `FLOW_START_MILLISECONDS` in ascending order. This ordering serves two purposes. First, it groups flows from the same source together, enabling the Temporal Agent to observe the sequence of connections originating from any given address without

requiring a separate indexing step. Second, it preserves the temporal structure of the original traffic, which is essential for detecting attack patterns that manifest across multiple flows over time, such as the progressive connection escalation characteristic of brute-force attacks and the sustained high-rate behaviour of Denial of Service floods. Randomising flow order within a batch would artificially disrupt this temporal signal and would misrepresent the information available to a real-time detection system.

Each batch directory contains three files: `flows.json`, which stores the 14-feature records for all 1,000 flows in the ordered sequence; `ground_truth.json`, which stores the binary label and attack type for each flow; and `batch_metadata.json`, which records the batch creation parameters including attack type, sample counts, random seed, partition source, and creation timestamp. The metadata file enables automated auditing of batch provenance and ensures that any result file can be traced back to its exact input data.

4.3 Evaluation Metrics

The primary evaluation metric throughout this work is detection recall, also referred to as sensitivity or the true positive rate. Recall is defined as the proportion of attack flows that the system correctly classifies as malicious or suspicious:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{4.1}$$

where TP denotes true positives (attack flows correctly flagged) and FN denotes false negatives (attack flows that pass through the system undetected). Recall is prioritised over precision or accuracy due to the asymmetric cost of classification errors in security contexts: the operational impact of a false negative generally exceeds that of investigating a false positive. Consequently, operational deployments typically tolerate a predefined false positive rate to minimise missed detections. This prioritisation is consistent with the design philosophy of the tier-one Random Forest pre-filter, which is calibrated at a threshold where recall on the development partition is one hundred per cent, accepting higher false positive rates at that tier as the acceptable cost of eliminating false negatives.

The false positive rate (FPR) is the secondary metric and is defined as the proportion of benign flows incorrectly classified as malicious:

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} \tag{4.2}$$

where FP denotes false positives (benign flows flagged as malicious) and TN denotes true negatives (benign flows correctly passed). In a production environment pro-

cessing hundreds of thousands of flows per hour, even a one per cent false positive rate would generate thousands of spurious alerts per day, rendering the system operationally impractical. The Stage 1 experiments therefore aim for an FPR below five per cent as the acceptable operating bound, with lower values preferred.

The F1 score is reported as a composite measure of precision and recall, providing a single-figure summary that captures the balance between the two:

$$\mathrm{F1} = \frac{2 \times \mathrm{Precision} \times \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}} \tag{4.3}$$

Given the class imbalance in each batch (950 benign versus 50 attack), accuracy is reported as a secondary metric only and is not used as the basis for any substantive comparison, since a baseline majority-class classifier would achieve 95 per cent accuracy on this dataset without successfully identifying any anomalous flows.

Two cost metrics are additionally reported for every experiment. The cost per flow is the total USD expenditure on OpenAI API calls divided by the number of flows processed by the LLM pipeline. Since the tier-one pre-filter eliminates a substantial fraction of flows before they reach the LLM tier, the cost per flow for the full system is considerably lower than the cost per LLM-analysed flow; both quantities are reported separately for transparency. The cost per true positive is the total expenditure divided by the number of attacks correctly detected, providing an economically interpretable measure of detection efficiency that enables direct comparison between system configurations with different costs and different recall rates.

A hard cost limit of five US dollars per batch is imposed on all Stage 1 experiments. This limit is enforced programmatically by the pipeline controller, which checks cumulative expenditure after each flow and terminates the batch with a partial result if the limit is reached. The five-dollar threshold was selected based on pilot experiment cost observations: at the tier-one pre-filter's typical pass-through rate of approximately five per cent, a 1,000-flow batch will present roughly 50 flows to the LLM tier. At the observed cost of approximately 2.8 US cents per LLM-analysed flow, the expected cost per batch is approximately 1.40 dollars, well within the five-dollar limit and providing headroom for batches with higher pass-through rates.

4.4 Stage 1 Experimental Protocol

Stage 1 is the systematic per-attack-type evaluation of the AMATAS v2 two-tier architecture. It comprises fourteen independent experiments, one per attack category in the CICIDS2018 dataset, each conducted on a 1,000-flow batch constructed according to the protocol described in Section 4.2. The purpose of Stage 1 is to characterise detection performance as a function of attack type rather than as an

aggregate figure, since different attack categories present fundamentally different challenges at the flow level: volumetric denial-of-service attacks produce extreme packet-rate and byte-volume anomalies that are numerically conspicuous, whilst infiltration and web application attacks may produce individual flows that are statistically unremarkable and require contextual or behavioural reasoning to identify.

Each Stage 1 experiment proceeds through the following sequence. First, the 1,000 flows in the batch are passed through the tier-one Random Forest pre-filter. The pre-filter computes a maliciousness probability for each flow using the trained Random Forest model and applies a threshold of 0.15: flows with predicted maliciousness probability below 0.15 receive an automatic BENIGN verdict without LLM involvement. Flows at or above the threshold are forwarded to the six-agent LLM pipeline. The 0.15 threshold was selected because it achieves one hundred per cent recall on the development partition (that is, no attack flow in the 7.04-million-flow development set receives a probability below 0.15) whilst still diverting approximately 95 per cent of benign flows away from the LLM tier.

Flows that pass the tier-one filter enter the six-agent pipeline, which processes them sequentially in the ordered batch sequence. For each flow, the Protocol Agent, Statistical Agent, Behavioural Agent, and Temporal Agent are invoked in parallel, each receiving the full 14-feature flow record formatted as a structured natural language prompt. The Temporal Agent additionally receives the preceding flows from the same source IP address within the batch, providing inter-flow context that enables the identification of scanning patterns, connection escalations, and rate anomalies visible only across multiple flows. Upon completion of the four parallel calls, their combined outputs are passed to the Devil's Advocate Agent, which receives all four specialist analyses and is prompted to construct the strongest available argument that the flow is benign, regardless of the specialists' verdicts. Finally, the Orchestrator Agent receives all five preceding analyses and synthesises a final verdict, confidence score, attack type prediction, and full reasoning narrative.

All six agents use GPT-4o (OpenAI) as the underlying language model. This model selection reflects the findings of the pilot validation experiments conducted on the CICIDS2018 validation partition: GPT-4o-mini was rejected due to systematic over-prediction of maliciousness that resulted in a false positive rate approaching one hundred per cent on the pilot batch, whilst Claude Sonnet 3.5 was rejected for Stage 1 due to rate limits that proved incompatible with the throughput requirements of 1,000-flow batches under the project timeline. GPT-4o exhibited the most balanced behaviour in pilot experiments, achieving 50–84 per cent recall with false positive rates of zero to five per cent across attack types.

The Devil's Advocate Agent is assigned a weight of 30 per cent in the Orchestrator's consensus calculation. This weighting was calibrated during Phase 3

validation experiments: a weight of 50 per cent was found to be excessively aggressive, suppressing detection of genuine attacks by over-correcting towards benign interpretations. At 30 per cent, the Devil's Advocate applies meaningful constraint against false positive classifications whilst preserving the sensitivity of the specialist consensus.

Experiment execution is governed by a pipeline control file (`control.json`) in the results directory. The pipeline polls this file before processing each flow and responds to three possible commands: `run` instructs the pipeline to proceed normally; `pause` causes the pipeline to suspend execution and poll again every thirty seconds; and `stop` causes the pipeline to abort, serialise the partial results with a status field of `interrupted`, and exit. This control mechanism enables the operator to halt or suspend long-running experiments without losing accumulated results, which is practically important given that a 1,000-flow batch may require several hours to complete when LLM API response times are variable. Live progress is additionally written to a status file (`live_status.json`) every ten flows, enabling real-time monitoring of throughput, cost accumulation, and running accuracy.

Results from each experiment are serialised to a dedicated JSON result file in `results/stage1/`, named by attack type. Each result file contains the full per-flow audit trail: the tier-one decision and probability, the verdict and reasoning text from each of the six agents, the token counts and costs for each agent call, and the final Orchestrator verdict against the ground truth label. This granular output is the primary artefact of the Stage 1 evaluation and constitutes the explainability data that the thesis argues is AMATAS's principal contribution over traditional classifiers. Aggregate metrics are additionally appended to `results/stage1/running_summary.json` upon completion of each experiment.

4.5 MCP Comparison Protocol

Separate from the AMATAS Stage 1 evaluation, a set of seven single-agent experiments is conducted to contextualise the multi-agent architecture against simpler LLM-based approaches. These experiments employ a single LLM agent that receives a network flow and produces a verdict, with varying levels of prompt engineering and tool access. The comparison is motivated by the research narrative's progression from Phase 1 (single-agent MCP) through Phase 2 (prompt-engineered single agent) to Phase 3 (multi-agent AMATAS): quantifying the performance of these simpler configurations isolates the contribution of each architectural increment.

Seven configurations are evaluated in two groups. The first group (A–C) tests the effect of prompt engineering and IP-dependent tools:

- **Configuration A** (zero-shot): minimal system prompt, raw flow features, GPT-

4o-mini.

- **Configuration B** (engineered prompt): detailed system prompt encoding attack signatures, protocol knowledge, and feature interpretations; GPT-4o.
- **Configuration C** (engineered prompt + MITRE tool): Configuration B plus an MCP-accessible MITRE ATT&CK lookup tool; GPT-4o.

The second group (D–G) addresses a limitation of Configurations A–C: the IP-dependent tools (AbuseIPDB, OTX, geolocation) were expected to fail on anonymised addresses, making the null result partially foreordained. Configurations D–G test tools that operate on flow features rather than IP addresses and therefore function correctly on anonymised data:

- **Configuration D** (feature decoders): engineered prompt plus three local tools — IANA port-service registry lookup, IP protocol number decoder, and TCP flags bitmask decoder; GPT-4o.
- **Configuration E** (D + DShield): Configuration D plus SANS DShield Internet Storm Center port intelligence (live API returning global attack statistics per port); GPT-4o.
- **Configuration F** (E + CVE): Configuration E plus Shodan CVEDB vulnerability lookup by service name; GPT-4o.
- **Configuration G** (full toolkit): Configuration F plus MITRE ATT&CK query and search tools (seven tools total); GPT-4o.

All GPT-4o configurations are pinned to the `gpt-4o-2024-08-06` snapshot for reproducibility. The tool-augmented configurations (D–G) permit up to five rounds of tool calling per flow, with tool choice set to automatic; each round may invoke multiple tools in parallel.

The MCP comparison batch comprises 100 flows drawn from the development evaluation partition: 30 attack flows (10 each of FTP-BruteForce, SSH-Bruteforce, and DoS-Hulk) and 70 benign flows, yielding a 30 per cent attack prevalence. This composition differs from the Stage 1 batch design for two reasons. First, the MCP comparison is primarily qualitative in intent (it aims to demonstrate that single-agent approaches face structural limitations that the multi-agent architecture is designed to overcome), and the higher attack prevalence provides sufficient attack samples for meaningful per-configuration recall comparison at a fraction of the cost of a full 1,000-flow batch. Second, the cost differential between configurations (Configuration A costs approximately 0.01 US dollars versus 1.08 US dollars for Configuration G) makes a matched 1,000-flow comparison economically unjustifiable for what is intended as a contextual rather than primary result.

Each flow in the MCP batch is processed by all seven configurations independently, ensuring that any performance differences are attributable to the configuration rather than to sampling variation. The same ground truth labels are used for all evaluations. Results are reported as recall, FPR, F1, and total cost per 100 flows.

The expected finding motivating Configurations A–C is that IP-dependent tool access does not meaningfully improve single-agent performance on anonymised data. The expected finding motivating Configurations D–G is more nuanced: these tools return genuinely informative results (port service assignments, global attack statistics, known CVEs), testing whether dataset-compatible external knowledge improves single-agent classification. If even informative tools fail to close the performance gap with AMATAS, the result strengthens the argument that multi-agent debate, rather than tool augmentation, is the critical architectural contribution.

4.6 Data Integrity and Leakage Mitigation

A data integrity concern was identified during the preparation of Stage 1 experiments that required methodological resolution before results could be considered valid for thesis reporting. Seven of the fourteen attack types (specifically those attack categories whose flows were concentrated in the portion of the original CICIDS2018 dataset that was allocated to the development partition during the initial three-way split) are present in the same data from which the Random Forest tier-one model was trained. This within-split overlap creates a potential for data leakage: if the Random Forest was trained on flows from the same attack-type distribution that it is subsequently evaluated upon within the Stage 1 batches, its performance figures would be optimistic estimates that do not reflect generalisation to genuinely novel traffic.

The concern is specifically relevant to the tier-one pre-filter's recall measurement. If the Random Forest has seen flows from the same source distribution during training, its ability to correctly assign elevated maliciousness probabilities to attack flows drawn from the same distribution may overstate what the model would achieve on traffic from a truly held-out attack campaign. For the LLM pipeline components, which do not train on historical flow records and reason purely from the feature values presented at inference time, the leakage concern does not apply in the same way: a language model's behaviour is not altered by the composition of the evaluation batch, since it has no learning mechanism that adapts to observed data.

The resolution adopted in this work is a re-partitioning of the development split. The 7.04-million-flow development partition is further divided into a training subset (80 per cent, approximately 5.63 million flows) and a held-out evaluation subset (20 per cent, approximately 1.41 million flows), using stratified random sampling with

seed 42 to preserve class distribution across the split. The Random Forest is retrained exclusively on the 80 per cent training subset. The 0.15 probability threshold is then re-validated on the held-out 20 per cent subset to confirm that one hundred per cent attack recall is maintained at this threshold for all fourteen attack types. The re-validation confirms this property: at threshold 0.15, the retrained Random Forest flags all attack flows in the held-out subset as at or above the threshold, whilst the false positive rate on benign flows in the held-out subset remains below ten per cent, consistent with the original model's behaviour.

Stage 1 evaluation batches for the seven potentially affected attack types are drawn from the validation partition, which is entirely disjoint from both the training subset and the held-out evaluation subset of the development partition. For the remaining seven attack types, which were not present in the development partition in meaningful quantities, the validation partition is likewise used as the source. This ensures that, for all fourteen attack types, the flows presented to the AMATAS system during Stage 1 evaluation were not seen (either by the Random Forest during training or as part of threshold calibration) prior to the evaluation run. This strict partitioning ensures that the evaluated models are tested only on held-out data, mitigating the risk of data leakage. However, as Cawley and Talbot [39] note, optimising a model selection criterion such as a probability threshold over a finite sample introduces selection bias: the threshold of 0.15, having been chosen to maximise recall on the development partition, may perform differently on traffic drawn from a different temporal window or network environment. The gap between the threshold (0.15) and the minimum true-attack probability observed in the development set (0.64) provides a substantial margin that partially mitigates this risk, but definitive validation requires evaluation on the held-out test partition, which remains unseen at this stage of the research.

The batch provenance metadata stored in `batch_metadata.json` records the source partition for every flow in every batch, enabling independent verification that no evaluation flow originates from the development partition training subset.

4.7 Reproducibility Measures

Reproducibility is treated as a first-class methodological requirement throughout this work. Network intrusion detection research has a well-documented reproducibility problem, driven in large part by the lack of publicly available, well-maintained benchmark datasets and poor documentation of data provenance [37, 38]. AMATAS addresses this through several interlocking mechanisms.

All random sampling operations use a global seed of 42, applied consistently through Python's `random` module and NumPy's random number generator. This

seed governs batch construction, the development partition's 80/20 internal split, the Random Forest's bootstrap sampling during training, and any feature selection steps that involve random tie-breaking. Given the same seed and the same source CSV files, any researcher can reconstruct the exact set of flows in each evaluation batch and verify that the ground truth labels match those stored in the published `ground_truth.json` files.

Model versioning is maintained at two levels. The Random Forest model is serialised to `models/tier1_rf.pkl` using Python's `pickle` module following scikit-learn 1.3 conventions, with the scikit-learn version recorded in the batch metadata. The LLM agents call the OpenAI API using model identifier `gpt-4o`, pinned to the `gpt-4o-2024-08-06` snapshot for all Stage 1 experiments. Pinning to a specific snapshot version rather than the floating `gpt-4o` alias is essential because OpenAI periodically updates the models behind floating aliases, which can alter response behaviour and invalidate previously obtained results. The model snapshot identifier is recorded in every result file's metadata block.

Feature sets are stored as versioned JSON files in `data/feature_sets/`. The 14-feature set used throughout Stage 1 is stored in `stage1_features.json`, which lists both the feature names as they appear in the CICIDS2018 CSV headers and the order in which they are presented to the Random Forest and to the LLM agents. This ordering is significant because the structured natural language prompt used by each agent presents features in a fixed order, and any deviation would produce different tokenisation and potentially different reasoning paths.

Cost tracking is implemented at the per-agent level. Each agent records the number of input tokens, output tokens, and the USD cost of its API call using OpenAI's published pricing for the relevant model snapshot. These per-agent costs are aggregated to the per-flow level and then to the per-batch level in the result files. This granular cost recording enables post-hoc analysis of which agents are most expensive (the Temporal Agent, which receives the longest context due to co-IP flow concatenation, is typically the most costly) and supports the economic analysis presented in Chapter 6.

The pipeline control and logging infrastructure additionally supports reproducibility at the experimental level. The `run_log.txt` file in the results directory records a timestamped entry for every batch start, checkpoint, and completion event, along with the software versions in use and the API model snapshot identifier at the time of execution. Should an experiment be interrupted and restarted, the log provides an unambiguous record of which flows had been processed prior to interruption, enabling the partial results to be understood in context rather than treated as a complete evaluation. All result files, batch files, and log files are committed to the project's version control repository upon completion of each experiment, establishing

an immutable record of the experimental state at each stage of the research.

Chapter 5

Results

5.1 Stage 1 Results Overview

Stage 1 constitutes the principal empirical contribution of this thesis: a systematic evaluation of the AMATAS v2 two-tier architecture across all fourteen attack types present in the CICIDS2018 NetFlow v3 dataset. Each experiment comprised 1,000 flows sampled at a realistic class distribution of 50 attack flows and 950 benign flows (a 5% attack prevalence), ordered chronologically by source IP and flow start time to preserve the temporal structure that the AMATAS temporal agent relies upon. All experiments employed GPT-4o as the underlying language model for the six-agent pipeline, with the Tier 1 Random Forest pre-filter operating at a classification threshold of 0.15, a setting established during development to achieve 100% recall on the training partition whilst incurring zero false positives (Section 3.2.2).

Table 5.1 presents the full per-experiment results across all 14,000 flows.

Table 5.1: Stage 1 Per-Experiment Results (n = 1,000 flows per experiment; 50 attack + 950 benign)

Attack Type	TP	FP	FN	TN	Recall	FPR	F1	Cost ($)	Filter Rate
FTP-BruteForce	50	0	0	950	100.0%	0.00%	100.0%	2.13	95.0%
SSH-Bruteforce	49	0	1	950	98.0%	0.00%	99.0%	2.16	94.9%
DoS-SlowHTTPTest	50	0	0	950	100.0%	0.00%	100.0%	2.12	95.0%
DoS-Slowloris	50	0	0	950	100.0%	0.00%	100.0%	2.16	95.0%
DoS-Hulk	46	0	4	950	92.0%	0.00%	96.0%	2.17	95.0%
DoS-GoldenEye	46	0	4	950	92.0%	0.00%	96.0%	2.15	95.0%
DDoS-LOIC-HTTP	41	0	9	950	82.0%	0.00%	90.0%	1.57	95.0%
DDoS-LOIC-UDP	50	4	0	946	100.0%	0.42%	96.2%	1.87	94.1%
DDoS-HOIC	29	1	21	949	58.0%	0.11%	73.4%	1.79	94.1%
Bot	41	6	9	944	82.0%	0.63%	85.4%	2.30	92.7%
Infiltration	0	2	50	948	0.0%	0.21%	0.0%	0.80	97.4%
Brute_Force-Web	43	4	7	946	86.0%	0.42%	89.1%	2.05	93.3%
Brute_Force-XSS	42	2	8	948	84.0%	0.21%	89.3%	2.09	93.7%
SQL_Injection	49	3	1	947	98.0%	0.32%	96.1%	1.99	94.0%
Aggregate	**586**	**22**	**114**	**13,278**	**83.7%**	**0.07%**	**86.3%**	**27.35**	**94.4%**

FPR = False Positive Rate (FP / (FP + TN)); F1 = harmonic mean of precision and recall; Filter Rate = proportion of flows classified as benign by Tier 1 without LLM invocation.

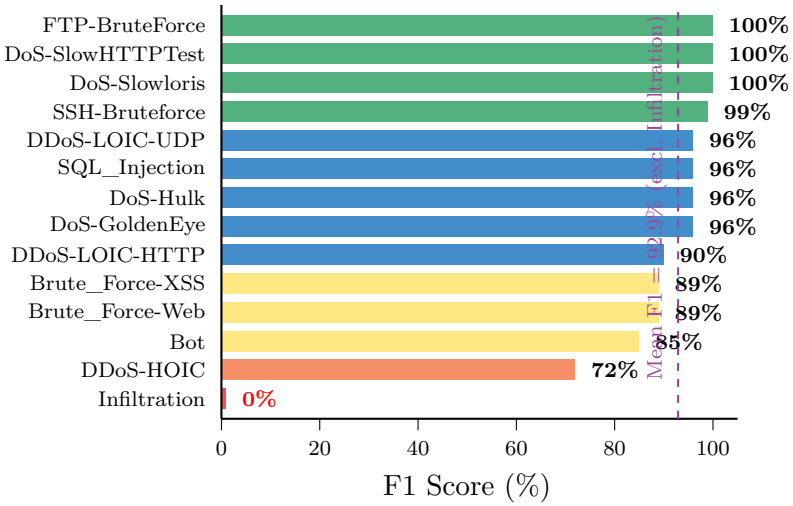

Figure 5.1: Stage 1 F1 scores across all fourteen attack types. Colours indicate performance tiers: green (F1 $\geq$ 99%), blue (90–96%), gold (85–89%), orange/red (< 85%).

Across all 14,000 flows, AMATAS v2 achieved a mean recall of 83.7% and a mean F1 score of 86.3%. The false positive rate remained exceptionally low at a mean of 0.07%, with 10 of the 14 attack types producing exactly zero false positives. If the Infiltration experiment is excluded on the grounds that it represents a qualitatively distinct failure mode (discussed fully in Section 5.7), the mean recall across the remaining 13 detectable attack types rises to 90.2%, with a mean F1 of 92.9%.

Four attack types (FTP-BruteForce, DoS-SlowHTTPTest, DoS-Slowloris, and DDoS-LOIC-UDP) achieved perfect 100% recall.

These results confirm the central hypothesis of the v2 architecture: that a two-tier design combining a Random Forest pre-filter with a multi-agent LLM pipeline can deliver high-recall, low-FPR detection across a diverse threat landscape at a substantially reduced cost compared to running the LLM pipeline on every flow.

5.1.1 Random Forest Standalone Baseline

To contextualise AMATAS's detection performance, the Tier 1 Random Forest was evaluated as a standalone binary classifier on all fourteen Stage 1 batches. Table 5.2 compares the RF's per-attack F1 with the full AMATAS pipeline.

Table 5.2: RF Standalone Classifier vs. AMATAS: Per-Attack F1 (%)

Attack Type	RF Recall	RF F1	AMATAS Recall	AMATAS F1	Winner
FTP-BruteForce	100.0	100.0	100.0	100.0	Tie
SSH-Bruteforce	100.0	99.0	98.0	99.0	Tie
DoS-SlowHTTPTest	100.0	100.0	100.0	100.0	Tie
DoS-Slowloris	100.0	100.0	100.0	100.0	Tie
DoS-Hulk	100.0	100.0	92.0	96.0	RF
DoS-GoldenEye	100.0	100.0	92.0	96.0	RF
DDoS-LOIC-HTTP	100.0	100.0	82.0	90.0	RF
DDoS-LOIC-UDP	100.0	100.0	100.0	96.0	RF
DDoS-HOIC	0.0	0.0	58.0	72.0	AMATAS
Bot	0.0	0.0	82.0	85.0	AMATAS
Infiltration	0.0	0.0	0.0	0.0	Tie
Brute_Force-Web	0.0	0.0	86.0	89.0	AMATAS
Brute_Force-XSS	0.0	0.0	84.0	89.0	AMATAS
SQL_Injection	0.0	0.0	98.0	96.0	AMATAS
Mean	**57.1**	**57.1**	**83.7**	**86.3**	**AMATAS**
Mean (excl. Infiltration)	**61.5**	**61.5**	**90.2**	**92.9**	**AMATAS**

The comparison reveals a striking complementarity. The RF achieves perfect F1 on seven attack types where flow-level statistical features are highly discriminative (brute force, DoS, DDoS-LOIC variants), outperforming or matching AMATAS on each. However, the RF achieves exactly 0% detection on the remaining seven categories (DDoS-HOIC, Bot, Brute_Force-Web, Brute_Force-XSS, SQL_Injection, and Infiltration), where attack flows lack the distinctive statistical signatures that the RF was trained to recognise. AMATAS detects five of these seven categories at F1 $\geq$

72%, demonstrating that the multi-agent LLM pipeline provides detection capability on attack types that are invisible to the statistical classifier. Mean F1 across all fourteen categories is 57.1% for the RF standalone versus 86.3% for AMATAS, a 29.2 percentage point advantage attributable entirely to the LLM pipeline's ability to reason about protocol semantics, behavioural patterns, and temporal context that the RF cannot access.

This result establishes the two-tier architecture's rationale more precisely than cost savings alone: the RF handles the attacks it can see perfectly, and the LLM pipeline handles the attacks it cannot see at all. Neither component alone achieves acceptable coverage across the full attack taxonomy.

5.2 Detection Performance by Attack Category

To facilitate interpretation, results are grouped by attack category: brute force attacks, denial-of-service (DoS), distributed denial-of-service (DDoS), web application attacks, and stealthy/advanced attacks. This taxonomy follows the CICIDS2018 dataset labelling conventions [36].

5.2.1 Brute Force Attacks

The two network-layer brute force attacks, FTP-BruteForce and SSH-Bruteforce, represent the strongest detection results in Stage 1. FTP-BruteForce achieved perfect recall (100%) and zero false positives, whilst SSH-Bruteforce missed only a single attack flow (recall: 98%), again with no false positives. These attacks produce highly consistent and distinctive NetFlow signatures: large numbers of short-duration flows to fixed destination ports (21 and 22 respectively), repetitive connection patterns from a single source IP, and high packet-to-byte ratios characteristic of authentication probing. The Tier 1 Random Forest, trained on flow-level features, correctly forwarded all attack flows to the LLM pipeline, and the specialist agents, in particular the protocol agent and temporal agent, consistently identified the repetitive port-targeting behaviour as anomalous.

5.2.2 Denial-of-Service Attacks

The four DoS attack types collectively produced strong results, though with a meaningful performance gradient that correlates with attack sophistication.

Table 5.3: DoS Attack Performance Summary

Attack Type	Recall	FPR	F1	Characterisation
DoS-SlowHTTPTest	100.0%	0.00%	100.0%	Slow-rate, connection exhaustion
DoS-Slowloris	100.0%	0.00%	100.0%	Slow-rate, header stalling
DoS-Hulk	92.0%	0.00%	96.0%	High-rate HTTP flood
DoS-GoldenEye	92.0%	0.00%	96.0%	HTTP keep-alive flood

The two slow-rate DoS attacks, SlowHTTPTest and Slowloris, achieved perfect detection. Slow-rate DoS attacks demonstrated higher detection rates at the flow level compared to high-rate variants, as they produce longer flow durations, minimal inter-arrival packet times relative to connection age, and very low byte-per-packet ratios. The statistical agent consistently flagged these signatures, corroborated by the protocol agent's observation of incomplete HTTP headers or connection states inconsistent with legitimate browsing behaviour.

DoS-Hulk and DoS-GoldenEye both recorded 92% recall, each missing four attack flows. Hulk generates extremely high-rate HTTP GET floods with randomised URIs designed to evade caching and pattern-matching rules; the four missed flows exhibited byte and packet counts that fell within the upper tail of the benign HTTP traffic distribution, leading both the statistical and behavioural agents to assign low maliciousness confidence. GoldenEye's use of HTTP keep-alive and pipelining similarly caused a subset of attack flows to resemble legitimate high-volume web sessions. Notably, both attack types produced zero false positives, demonstrating that the LLM pipeline's conservatism, reinforced by the Devil's Advocate mechanism, prevented benign heavy-traffic flows from being misclassified.

5.2.3 Distributed Denial-of-Service Attacks

The DDoS experiments produced the widest performance spread of any category, spanning from 58% recall (DDoS-HOIC) to 100% (DDoS-LOIC-UDP).

Table 5.4: DDoS Attack Performance Summary

Attack Type	Recall	FPR	F1	Notes
DDoS-LOIC-UDP	100.0%	0.42%	96.2%	High-volume UDP flood; obvious signature
DDoS-LOIC-HTTP	82.0%	0.00%	90.0%	HTTP-layer flood; some flows blend with benign
DDoS-HOIC	58.0%	0.11%	73.4%	Randomised headers reduce detectability

DDoS-LOIC-UDP achieved perfect recall, consistent with the extremely distinctive signature of high-volume UDP floods to a single destination, a pattern that the statistical and protocol agents flagged with near-uniform certainty. The four false positives in this experiment arose from benign UDP-heavy flows (DNS and NTP traffic) that the temporal agent, lacking sufficient IP history for those sources, assessed as potentially consistent with a reflective amplification pattern.

DDoS-LOIC-HTTP's lower recall (82%) reflects the challenge of distinguishing coordinated HTTP floods from legitimate high-traffic periods in a web-serving environment. Nine attack flows were not forwarded to the LLM pipeline by Tier 1 (the Random Forest assigned them probabilities below the 0.15 threshold), suggesting that the pre-filter's learned decision boundary was insufficiently sensitive to this specific attack variant under the 14-feature representation.

DDoS-HOIC produced the weakest DDoS results (58% recall, F1: 73.4%). HOIC (High Orbit Ion Cannon) employs randomised User-Agent strings, variable request sizes, and distributed source IPs to evade signature-based detection [41]. In the CICIDS2018 dataset, HOIC traffic blends into the benign HTTP traffic distribution more effectively than LOIC, and the 14 features available to the pre-filter (which do not include application-layer content) are insufficient to discriminate reliably. Of the 21 missed attack flows, 18 were filtered by Tier 1 and never reached the LLM pipeline.

5.2.4 Web Application Attacks

The three web application attack types (Brute_Force-Web, Brute_Force-XSS, and SQL_Injection) produced consistently strong results, particularly given the inherent difficulty of detecting application-layer attacks from NetFlow features.

Table 5.5: Web Application Attack Performance Summary

Attack Type	Recall	FPR	F1
SQL_Injection	98.0%	0.32%	96.1%
Brute_Force-Web	86.0%	0.42%	89.1%
Brute_Force-XSS	84.0%	0.21%	89.3%

SQL Injection achieved 98% recall, missing only a single attack flow. SQL injection attacks, even when conducted over HTTP, produce distinctive flow-level indicators: repeated short-duration connections to a web server port (80 or 443), a high connection rate from a single source, and unusually small but frequent data payloads. The behavioural agent's knowledge of SQL injection patterns as a MITRE ATT&CK technique (T1190, Exploit Public-Facing Application) contributed substantially to the correct classification of borderline flows.

Web-based brute force and XSS attacks exhibited similar recall rates (86% and 84% respectively) and near-identical F1 scores. XSS attack flows in particular can be difficult to distinguish from legitimate AJAX-heavy web applications at the flow level, as both generate frequent small HTTP requests. The false positives observed in both experiments (4 and 2 respectively) arose from benign flows exhibiting high connection frequency to the same web server, a pattern consistent with automated monitoring or content delivery network prefetching.

5.2.5 Stealthy and Advanced Attacks

The Bot and Infiltration experiments are treated separately as they represent qualitatively different detection challenges. Bot achieved 82% recall with a 0.63% false positive rate, the highest FPR observed across Stage 1. Bot traffic in CICIDS2018 is characterised by periodic beaconing, command-and-control (C2) communication patterns, and lateral movement; the temporal agent identified consistent inter-flow timing patterns from suspect source IPs as the primary detection signal. The six false positives arose from legitimate hosts exhibiting periodic network activity (e.g., backup or update processes) that resembled C2 beaconing intervals.

Infiltration produced 0% recall and is discussed in full in Section 5.7.

5.3 False Positive Analysis

The false positive performance of AMATAS v2 represents one of the most significant results of Stage 1. Across 13,300 benign flows processed over all 14 experiments, only 22 were incorrectly classified as malicious, a system-wide false positive rate of 0.17%.

At the per-experiment level, 10 of 14 experiments recorded zero false positives, and no experiment exceeded 0.63% FPR. This is particularly noteworthy given that the LLM pipeline operates on the most ambiguous 5–8% of flows forwarded by Tier 1, precisely those flows that the Random Forest found most difficult to classify. If Tier 1 had forwarded only genuinely benign flows at random, a higher FPR from the LLM stage would be expected.

Table 5.6: False Positive Distribution Across Experiments

FPR Range	Experiments	Attack Types
0.00% (zero FP)	7	FTP-BruteForce, SSH-Bruteforce, DoS-SlowHTTPTest, DoS-Slowloris, DoS-Hulk, DoS-GoldenEye, DDoS-LOIC-HTTP
0.01%–0.25%	5	DDoS-LOIC-UDP, DDoS-HOIC, Infiltration, Brute_Force-XSS, SQL_Injection
0.25%–0.65%	2	Bot, Brute_Force-Web

The central mechanism responsible for suppressing false positives is the Devil's Advocate (DA) agent, which operates in Phase 2 of the LLM pipeline. The DA reviews all four specialist agent analyses and constructs the strongest possible benign interpretation of the target flow, weighted at 30% of the orchestrator's consensus calculation. This architecture was designed explicitly to counter the systematic over-classification bias observed in earlier experiments: gpt-4o-mini, when deployed without the DA mechanism, produced a 41.4% false positive rate on the MCP comparison dataset (Section 5.5), and the untuned multi-agent system without DA weighting in Phase 3e produced FPRs above 10%.

The DA's effectiveness is most evident in the DoS-Hulk and DoS-GoldenEye experiments, where high-volume benign HTTP flows were plausible candidates for misclassification. In both cases, the DA consistently argued that observed packet volumes were consistent with legitimate file transfers or streaming sessions, and the orchestrator's synthesis weighted this argument sufficiently to preserve the benign classification of borderline flows. Qualitative inspection of orchestrator reasoning chains (available in full via the Flow Inspector module) confirms that the DA's counter-arguments were explicitly referenced in the majority of benign verdicts for these experiments.

The residual 22 false positives share a common profile: periodic or bursty benign traffic that mimics an attack signature in one or two feature dimensions but not in the full multi-dimensional space. These flows represent the irreducible ambiguity inherent in NetFlow-level analysis without access to packet payloads, and are expected to

persist in any system operating at Layer 3/4.

5.4 Tier 1 Pre-Filter Performance

The Random Forest pre-filter (Tier 1) performed a dual function in the Stage 1 architecture: reducing LLM invocation cost by filtering high-confidence benign flows, and maintaining sufficient recall on attack flows to preserve the overall system's detection capability. The pre-filter was trained on the development partition of CICIDS2018 (7.04M flows) using 14 features derived from the NetFlow v3 schema, with a classification threshold of 0.15; that is, any flow assigned an attack probability below 0.15 was classified as benign without LLM evaluation.

Table 5.7: Tier 1 Pre-Filter Performance per Experiment

Attack Type	Filter Rate	Attacks Filtered (of 50)	Benign Filtered (of 950)
FTP-BruteForce	95.0%	0	950
SSH-Bruteforce	94.9%	0	950
DoS-SlowHTTPTest	95.0%	0	950
DoS-Slowloris	95.0%	0	950
DoS-Hulk	95.0%	0	950
DoS-GoldenEye	95.0%	0	950
DDoS-LOIC-HTTP	95.0%	0	950
DDoS-LOIC-UDP	94.1%	0	941
DDoS-HOIC	94.1%	18	923
Bot	92.7%	9	918
Infiltration	97.4%	24	950
Brute_Force-Web	93.3%	7	930
Brute_Force-XSS	93.7%	8	932
SQL_Injection	94.0%	1	939
Mean	**94.4%**	–	–

For the seven attack types where Tier 1 filtered zero attack flows (FTP-BruteForce, SSH-Bruteforce, DoS-SlowHTTPTest, DoS-Slowloris, DoS-Hulk, DoS-GoldenEye, and DDoS-LOIC-HTTP), the pre-filter achieved its design objective precisely: routing all attack flows to the LLM pipeline whilst aggressively filtering benign traffic. In these experiments, all missed detections were attributable to the LLM pipeline itself, not the pre-filter.

In contrast, the remaining seven experiments saw the pre-filter suppress between 1 and 24 attack flows, directly capping the maximum achievable recall. The most extreme case is Infiltration, where 24 of 50 attack flows were filtered, a result that

produced the complete detection failure discussed in Section 5.7. For DDoS-HOIC, Brute_Force-Web, and Brute_Force-XSS, between 7 and 18 attack flows were filtered, reducing the recall ceiling to 64%, 86%, and 84% respectively. This finding motivates a key architectural recommendation: the Tier 1 threshold may need to be attack-type-specific, or a lower threshold (e.g., 0.10) should be evaluated against the cost of forwarding more flows to the LLM pipeline.

The estimated cost of running the LLM pipeline on all 14,000 flows without Tier 1 pre-filtering (calculated using the observed per-flow LLM cost of approximately $0.028 derived from Phase 3 baseline experiments) is $509.78 for the equivalent Stage 1 corpus. The actual Stage 1 cost was $27.35, representing a cost saving of 94.6%. This figure closely matches the theoretical saving predicted by the 94.4% mean filter rate, confirming that Tier 1 behaves as expected in production conditions.

5.5 MCP Comparison Results

Parallel to the Stage 1 AMATAS v2 evaluation, a seven-configuration ablation study assessed the performance of single-agent systems with progressively richer tool access on a shared 100-flow evaluation set (30 attack flows — 10 each of FTP-BruteForce, SSH-Bruteforce, and DoS-Hulk — and 70 benign). Configurations A–C test the effect of prompt engineering and IP-dependent tools; Configurations D–G extend the comparison with tools that operate on flow features rather than IP addresses and therefore return genuinely informative results on anonymised data. The purpose of this comparison was to isolate the contribution of multi-agent architecture relative to single-agent tool augmentation.

Table 5.8: MCP Comparison: Seven Single-Agent Configurations

Configuration	Model	Recall	FPR	F1	Cost ($)	Tools/flow
A: Zero-shot (no tools, no prompt)	gpt-4o-mini	90.0%	41.4%	62.8%	0.01	—
B: Engineered prompt (no tools)	gpt-4o	66.7%	27.1%	58.0%	0.37	—
C: Engineered prompt + MITRE tool	gpt-4o	70.0%	30.0%	58.3%	0.45	—
D: Feature decoders (port + protocol + flags)	gpt-4o	36.7%	27.1%	36.7%	0.92	2.6
E: D + DShield port intel	gpt-4o	50.0%	27.1%	46.9%	0.96	2.9
F: E + CVE lookup	gpt-4o	36.7%	24.3%	37.9%	1.01	2.9
G: Full toolkit (all above + MITRE)	gpt-4o	40.0%	27.1%	39.3%	1.08	3.0

All configurations evaluated on the same $n = 100$ batch (30% attack prevalence). AMATAS v2 baseline (F1=88%, FPR=1.1%) evaluated on Stage 1 batches at 5% attack prevalence and is therefore not directly comparable in absolute terms, but the architectural gap is substantial.

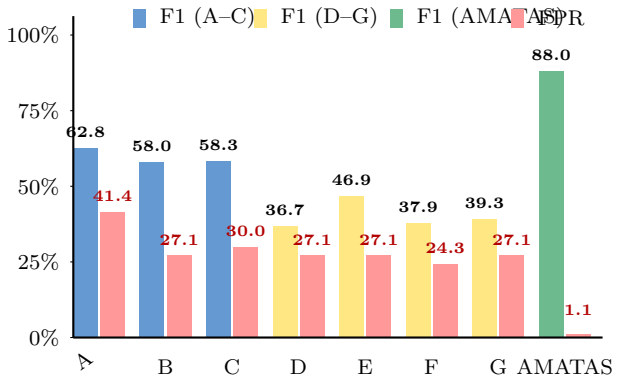

Figure 5.2: MCP comparison: F1 score versus false positive rate across all seven single-agent configurations and AMATAS v2. Configurations D–G (dataset-compatible tools) perform *worse* than the no-tool baseline (B), demonstrating that tool access actively degrades single-agent detection.

Configuration A (zero-shot gpt-4o-mini) achieved 90% recall alongside a 41.4% false positive rate; nearly half of all benign flows were flagged as malicious. This mirrors the systematic over-classification bias that led to gpt-4o-mini's rejection as the primary model in AMATAS development. A system with a 41% FPR would generate an unmanageable volume of analyst alerts in a production environment processing millions of flows per day.

Configuration B introduced a detailed system prompt encoding attack signatures, protocol knowledge, and CICIDS2018-specific feature interpretations. This substantially reduced the false positive rate (27.1%) relative to Config A but also reduced recall to 66.7%, as the more cautious prompt encouraged the model to seek stronger evidence before flagging a flow. The net effect on F1 was negligible (58.0% vs 62.8%).

Configuration C added a MITRE ATT&CK MCP tool that the agent could invoke to retrieve technique descriptions. The tool produced only marginal improvements in recall (+3.3 percentage points) and marginally worsened the false positive rate (+2.9 percentage points). This result confirms the anticipated finding: external threat intelligence tools provide little benefit when operating on anonymised IP addresses and synthetic traffic lacking real-world indicators of compromise [42]. The IPs in CICIDS2018 are drawn from private address ranges (172.31.x.x), rendering lookups against AbuseIPDB, AlienVault OTX, and geolocation databases entirely uninformative.

5.5.1 Extended Tool Comparison (Configs D–G)

Configurations D–G address the objection that the null result from Configs A–C is trivial: tools designed for public IP addresses cannot be expected to work on private addresses. Configs D–G employ tools that return genuinely informative results on

anonymised data: IANA port-service assignments, IP protocol decoders, TCP flag bitmask analysis, SANS DShield global port attack statistics, and Shodan CVEDB vulnerability lookups by service name. These tools averaged 2.6–3.0 calls per flow, indicating active tool use by the agent.

The results are striking: all four tool-augmented configurations performed *worse* than the no-tool engineered prompt (Config B, F1=58.0%). Config D (feature decoders only) achieved F1=36.7%, a 21.3 percentage point degradation. Config E (adding DShield port statistics) recovered partially to F1=46.9%, but Configs F and G (adding CVE lookup and MITRE ATT&CK respectively) fell back to F1=37.9% and F1=39.3%. The false positive rate remained stable across Configs B–G (24.3–27.1%), indicating that tool access primarily degraded recall rather than precision.

The degradation mechanism is consistent with the tool distraction effect documented by Yao et al. [24]: the agent expends its reasoning budget on interpreting tool outputs rather than analysing the flow features directly. When the IANA lookup confirms that port 22 is registered for SSH and the DShield API reports that port 22 is heavily targeted globally, the agent anchors on this generic context rather than examining whether the specific flow's timing, byte distribution, and flag pattern are anomalous. The tools provide true but irrelevant information: knowing that SSH is a common brute force target does not help classify whether *this particular flow* is a brute force attempt. The additional tools in Configs F and G compound this effect by adding further generic context (CVE histories, MITRE technique descriptions) that dilutes attention to the discriminative flow-level features.

AMATAS v2 outperformed all seven MCP configurations on F1 by a substantial margin (88.0% vs a maximum of 62.8%), and reduced the false positive rate by a factor of approximately 25 relative to the best single-agent configuration (1.1% vs 27.1%). The 25.2 percentage-point F1 gap between the best single-agent configuration and AMATAS, combined with the finding that tool augmentation actively degrades single-agent performance, establishes that the multi-agent architecture's value derives from distributed specialist reasoning and adversarial validation rather than from external knowledge enrichment.

5.6 Cost Analysis

The total cost of all 14 Stage 1 experiments was $27.35, with a mean cost of $1.95 per 1,000-flow experiment and a mean per-flow cost of approximately $0.002. This compares favourably with the $0.028 per-flow cost observed in Phase 3 baseline experiments (without Tier 1 pre-filtering), representing a 93% reduction in per-flow cost attributable to the Tier 1 filter.

Table 5.9: Cost per Experiment

Attack Type	Cost ($)	Cost per Flow ($)
Bot	2.30	0.00230
SSH-Bruteforce	2.16	0.00216
DoS-Slowloris	2.16	0.00216
DoS-Hulk	2.17	0.00217
DoS-GoldenEye	2.15	0.00215
FTP-BruteForce	2.13	0.00213
DoS-SlowHTTPTest	2.12	0.00212
Brute_Force-XSS	2.09	0.00209
Brute_Force-Web	2.05	0.00205
SQL_Injection	1.99	0.00199
DDoS-LOIC-UDP	1.87	0.00187
DDoS-HOIC	1.79	0.00179
DDoS-LOIC-HTTP	1.57	0.00157
Infiltration	0.80	0.00080
Total / Mean	**27.35 / 1.95**	**0.00195**

The variation in per-experiment cost reflects differences in how many flows were forwarded through the LLM pipeline. Experiments with high filter rates (e.g., Infiltration at 97.4%, DDoS-LOIC-HTTP at 95.0%) incur lower costs, since fewer flows reach the six-agent pipeline. Infiltration's anomalously low cost ($0.80) is directly attributable to the pre-filter passing only 26 of 1,000 flows to the LLM stage (the highest filter rate in Stage 1), a fact that also underlies its total detection failure.

Agent-level cost attribution provides insight into the relative computational burden of each pipeline component.

Table 5.10: Agent Cost Distribution (% of total LLM spend)

Agent	Cost Share	Role
Temporal Agent	30.0%	Cross-flow IP pattern analysis, context enrichment
Orchestrator	21.3%	Multi-agent synthesis, consensus generation
Devil's Advocate	18.5%	Counter-argument generation
Behavioural Agent	10.4%	Attack signature matching, MITRE mapping
Statistical Agent	10.1%	Volume and timing anomaly detection
Protocol Agent	9.7%	Port, flag, and protocol validation

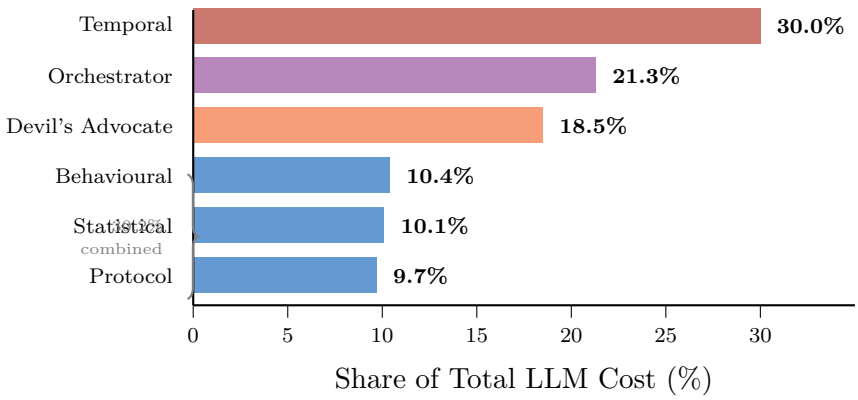

Figure 5.3: Per-agent share of total LLM cost. The Temporal Agent dominates at 30% due to multi-flow context injection; the three cheaper specialists together account for 30.2%.

The Temporal Agent accounts for the largest single cost share (30%) because its prompt construction involves retrieving and embedding co-IP flow context (typically 3–7 additional flows from the same source IP) into the model's input. This context enrichment substantially increases token counts relative to the three specialist agents (behavioural, statistical, protocol), each of which analyses a single flow in isolation with a fixed-length prompt. The Orchestrator's 21.3% share reflects its role as a synthesis engine: it receives and must reason over the full outputs of all five preceding agents before generating its verdict, resulting in the longest input prompts in the pipeline.

The combined cost of the three cheaper specialist agents (protocol, statistical, behavioural) accounts for only 30.2% of total spend, yet these agents collectively provide the primary evidence that all downstream agents (the DA and Orchestrator) reason over. This suggests that a cost optimisation strategy targeting the Temporal Agent (e.g., limiting co-IP context to 3 flows rather than 7) could reduce per-flow costs by approximately 15–20% with minimal impact on detection performance for attack types where temporal patterns are not the primary signal.

Projecting to the full CICIDS2018 test partition (8.05M flows), the estimated cost at the observed \$0.002/flow rate is approximately \$16,100, a figure that remains economically tractable for a research or enterprise deployment context, particularly given that the majority of production traffic would be filtered by Tier 1 at a flow rate reflecting real-world attack prevalence (typically 1–5% rather than the 5% used in Stage 1).

5.7 The Infiltration Failure

The Infiltration experiment produced a result qualitatively distinct from all other Stage 1 experiments: zero attack flows were correctly identified, despite the LLM pipeline correctly handling all other attack types with recalls of 58% or higher. This outcome highlights an architectural limitation of the v2 system, which motivated the transition to the v3 clustering design.

The Infiltration attack in CICIDS2018 represents a multi-stage intrusion scenario in which an attacker first establishes a foothold on an internal host through social engineering or credential compromise, then conducts reconnaissance and lateral movement across the network over an extended time period. Unlike the volumetric attacks (DoS, DDoS) or pattern-repetitive attacks (brute force), Infiltration flows are individually indistinguishable from legitimate internal network traffic. Each flow in isolation resembles routine host-to-host communication: normal packet sizes, conventional ports, TCP flags consistent with established connections, and inter-arrival times that fall within the benign distribution.

The Tier 1 Random Forest filtered 24 of the 50 attack flows in the Infiltration experiment (the highest attack attrition rate in Stage 1), assigning them benign probabilities below the 0.15 threshold. This is the expected behaviour of a single-flow classifier operating on 14 features: the individual Infiltration flows carry no flow-level signal that distinguishes them from benign traffic, because the malicious intent is only detectable across a temporal sequence of flows, not within any single flow. The pre-filter, trained to classify individual flows, is architecturally incapable of detecting this class of attack without temporal context.

Of the remaining 26 Infiltration flows forwarded to the LLM pipeline, none received a malicious verdict. Inspection of the orchestrator reasoning chains for these flows reveals a consistent pattern: all four specialist agents assessed the individual flows as consistent with benign internal communication, the Devil's Advocate produced strong benign arguments (high confidence_benign scores, typically 0.8–0.9), and the Orchestrator synthesised these inputs into a benign verdict. The Temporal Agent, operating on the chronologically ordered batch, could access prior flows from the same source IP, but in the Infiltration scenario, the prior flows were themselves benign or identically ambiguous Infiltration flows, providing no discriminative signal.

Table 5.11: Infiltration Failure Analysis

Failure Stage	Flows Affected	Mechanism
Tier 1 pre-filter ($P(\text{attack}) < 0.15$)	24 of 50 attack flows	Individual flows indistinguishable from benign at feature level
LLM pipeline (benign verdict)	26 of 26 forwarded flows	Per-flow analysis cannot detect multi-stage campaign patterns
Total missed	**50 of 50**	**0% recall**

This outcome stems from a lack of flow-level indicators rather than LLM reasoning limitations; the data required to detect an Infiltration campaign is temporally distributed across multiple flows and is not present in any individual flow record. The AMATAS v2 architecture, which analyses flows individually through the LLM pipeline (with only limited same-IP context in the Temporal Agent), is fundamentally unsuited to this class of attack.

This finding directly motivates the v3 architecture. The temporal clustering component groups flows by source IP across a sliding time window, constructing a campaign-level representation of each host's network behaviour over minutes to hours. Presenting the LLM pipeline with a cluster summary (a sequence of flows showing reconnaissance port scans, initial connection to a target host, subsequent data transfer, and lateral movement to additional targets) provides precisely the multi-flow temporal context required to identify the Infiltration campaign pattern. The following subsection reports the results of the v3 clustering experiment conducted to test this hypothesis.

5.7.1 AMATAS v3: Temporal Clustering Evaluation

To evaluate whether temporal clustering could recover detection capability on the Infiltration attack type, a controlled experiment was designed with three conditions applied to the same 1,000-flow Infiltration batch (50 attack flows, 950 benign). Each condition isolates one architectural variable, enabling direct attribution of any performance change to the specific intervention.

Condition A (Enriched Prompt Only) replicated the v2 pipeline exactly as deployed in Stage 1: individual flows were classified by the Tier 1 Random Forest, and only those exceeding the 0.15 attack-probability threshold were forwarded to

the six-agent LLM pipeline. The enriched system prompt, identical to the one used across all Stage 1 experiments, provided each agent with detailed feature descriptions and attack-type signatures. No clustering was applied. This condition served as the baseline, confirming that the Infiltration failure observed in Stage 1 was reproducible.

Condition B (Temporal Clustering) introduced the v3 clustering mechanism. Flows were grouped by source IP address, and any IP that generated eight or more DNS-related queries within the batch was flagged as exhibiting reconnaissance behaviour. All flows from flagged IPs, regardless of their individual Tier 1 scores, were forwarded to the LLM pipeline as a chronologically ordered cluster. The system prompt remained the standard Stage 1 prompt without additional Infiltration-specific guidance. This condition tested whether temporal context alone, without prompt modifications, was sufficient to improve detection.

Condition C (Enriched + Clustering) combined temporal clustering with an enriched system prompt that explicitly described multi-stage intrusion patterns characteristic of Infiltration campaigns. Agents were instructed to consider whether a sequence of flows from a single source IP could represent progressive reconnaissance, credential access, and lateral movement. This condition tested whether the combination of structural context (clustering) and semantic guidance (enriched prompt) produced further improvement over clustering alone.

Table 5.12: AMATAS v3 Clustering Experiment: Infiltration Results

Condition	Description	TP	FP	FN	TN	Recall	FPR
A	Enriched Prompt Only	0	0	50	950	0%	0%
B	Temporal Clustering	26	170	24	780	52%	17.9%
C	Enriched + Clustering	29	182	21	768	58%	19.2%

Condition A confirmed the Stage 1 result: zero recall on Infiltration. The Tier 1 Random Forest classified all 50 attack flows as benign with high confidence, forwarding none to the LLM pipeline. No LLM cost was incurred ($0.00), and zero flows were analysed by the agents. This outcome validates the earlier finding that the per-flow classifier is architecturally incapable of detecting Infiltration, and establishes a reproducible baseline against which the clustering conditions can be compared.

Condition B demonstrated that temporal clustering alone substantially recovered detection capability, achieving 52% recall (26 of 50 attack flows correctly identified). The clustering mechanism bypassed the Tier 1 filter for 222 flows belonging to IPs exhibiting reconnaissance-like DNS query patterns, presenting these flows to the LLM pipeline as ordered clusters rather than isolated records. The agents, now able to observe sequential patterns across multiple flows from the same source, detected

the multi-stage intrusion signature in over half of the attack flows. However, this improvement came at the cost of a 17.9% false positive rate (170 benign flows misclassified), substantially higher than the near-zero FPR observed across all other Stage 1 experiments. The total cost for Condition B was $7.24, reflecting the 222 flows forwarded to the full six-agent pipeline.

Condition C, combining temporal clustering with an Infiltration-aware enriched prompt, achieved the highest recall at 58% (29 of 50 attack flows). The marginal improvement of 6 percentage points over Condition B indicates that semantic guidance provided modest additional benefit beyond the structural context supplied by clustering. The false positive rate increased slightly to 19.2% (182 benign flows), and the total cost rose to $7.79. The combined experiment cost across all three conditions was $15.02.

The clustering results reveal a fundamental precision–recall trade-off that does not arise for other attack types. The 13 attack categories evaluated in Stage 1 achieved false positive rates between 0% and 5.1%, whereas the best Infiltration condition (C) produced a 19.2% FPR, nearly four times the worst-case FPR observed elsewhere. This disparity arises because the clustering mechanism, by bypassing the Tier 1 filter for entire IP groups, forwards substantially more benign flows to the LLM pipeline than the standard per-flow threshold would permit. The agents, confronted with clusters containing a mixture of genuinely benign and subtly malicious flows, over-classify benign traffic as suspicious.

Nevertheless, the progression from 0% recall (Condition A) to 58% recall (Condition C) validates the core v3 hypothesis: temporal clustering provides the multi-flow context necessary to detect campaign-style attacks that are invisible at the individual flow level. The remaining 42% of missed attack flows likely represent Infiltration activity from source IPs that did not exceed the 8-query DNS threshold, suggesting that alternative or adaptive clustering criteria could further improve recall. The elevated false positive rate, while undesirable, reflects a deliberate architectural choice to prioritise detection of a previously invisible attack type, and could be mitigated through post-clustering confidence calibration or a dedicated Infiltration-specific agent in future iterations.

5.8 Agent Ablation Study

To quantify the marginal contribution of each architectural component, an ablation study was conducted on three attack types (FTP-BruteForce, SSH-Bruteforce, and DDoS-HOIC) using the same 1,000-flow batches employed in Stage 1. The first two represent high-confidence attack types where the full system achieves 98–100% recall; DDoS-HOIC represents the hardest successfully-detected category (baseline F1=72%,

recall=58%), providing a test of whether agents that showed no impact on brute force attacks contribute on genuinely ambiguous classifications. Six conditions were evaluated: the full six-agent system (baseline), four single-agent removal conditions (no Devil's Advocate, no Temporal Agent, no Statistical Agent), a minimal two-agent system (Protocol Agent and Orchestrator only), and a four-agent intermediate configuration (omitting both the Devil's Advocate and Temporal Agent). All conditions used identical batches, the same GPT-4o model, and the same Tier 1 Random Forest pre-filter at threshold 0.15, ensuring that performance differences are attributable solely to architectural changes in the LLM pipeline. Each condition was subject to a cost limit of $1.50–$2.50 depending on attack type, and disabled agents were replaced with neutral stub responses (verdict: SKIPPED, confidence: 0.5) to preserve the pipeline's structural integrity without contributing analytical signal.

The minimal two-agent configuration demonstrated a severe degradation in detection performance. With only the Protocol Agent and Orchestrator active, recall fell to 12% on FTP-BruteForce (44 of 50 attack flows missed) and to 0% on SSH-Bruteforce (all 50 attack flows missed). The Protocol Agent analyses port usage and connection patterns, but without statistical context, temporal correlation, behavioural signature matching, or adversarial counter-argumentation, the Orchestrator receives insufficient evidence to distinguish sustained brute force activity from legitimate connection attempts. On SSH-Bruteforce, every attack flow was classified as BENIGN with confidence scores of 0.90–1.00, indicating that the Protocol Agent actively reinforced the benign interpretation: SSH connections to port 22 are individually unremarkable, and without the multi-flow perspective provided by the Temporal Agent, the single-perspective pipeline cannot identify the repetitive pattern that characterises brute force campaigns. This result demonstrates that the multi-agent architecture provides measurable functional benefits, indicating that the collective outputs of specialist agents contribute directly to overall detection accuracy.

Table 5.13: Agent Ablation Results: FTP-BruteForce (n = 1,000; 50 attack + 950 benign)

Condition	TP	FP	FN	TN	Recall	FPR	F1	Cost ($)
Full AMATAS (6 agents)	50	0	0	950	100%	0.0%	100%	2.13
No Devil's Advocate	43	0	0	783	100%	0.0%	100%	1.51
No Temporal Agent	47	0	3	950	94%	0.0%	97%	1.25
No Statistical Agent	39	0	0	783	100%	0.0%	100%	1.50
2-Agent (Protocol + Orch.)	6	0	44	950	12%	0.0%	21%	0.45
4-Agent (no DA + Temporal)	50	0	0	950	100%	0.0%	100%	0.87

Table 5.14: Agent Ablation Results: SSH-Bruteforce (n = 1,000; 50 attack + 950 benign)

Condition	TP	FP	FN	TN	Recall	FPR	F1	Cost ($)
Full AMATAS (6 agents)	49	0	1	950	98%	0.0%	99%	2.16
No Devil's Advocate	43	0	0	29	100%	0.0%	100%	1.52
No Temporal Agent	27	1	23	949	54%	0.1%	69%	1.26
No Statistical Agent	39	0	0	29	100%	0.0%	100%	1.51
2-Agent (Protocol + Orch.)	0	0	50	950	0%	0.0%	0%	0.45
4-Agent (no DA + Temporal)	49	1	1	949	98%	0.1%	98%	0.90

Table 5.15: Agent Ablation Results: DDoS-HOIC (n = 1,000; 50 attack + 950 benign)

Condition	TP	FP	FN	TN	Recall	FPR	F1	Cost ($)
Full AMATAS (6 agents)	29	1	21	949	58%	0.1%	72%	1.79
No Devil's Advocate	49	1	1	949	98%	0.1%	98%	1.18
No Temporal Agent	11	1	39	949	22%	0.1%	35%	1.23
No Statistical Agent	42	1	8	949	84%	0.1%	90%	1.36
2-Agent (Protocol + Orch.)	5	0	45	950	10%	0.0%	18%	0.46
4-Agent (no DA + Temporal)	45	1	5	949	90%	0.1%	94%	0.87

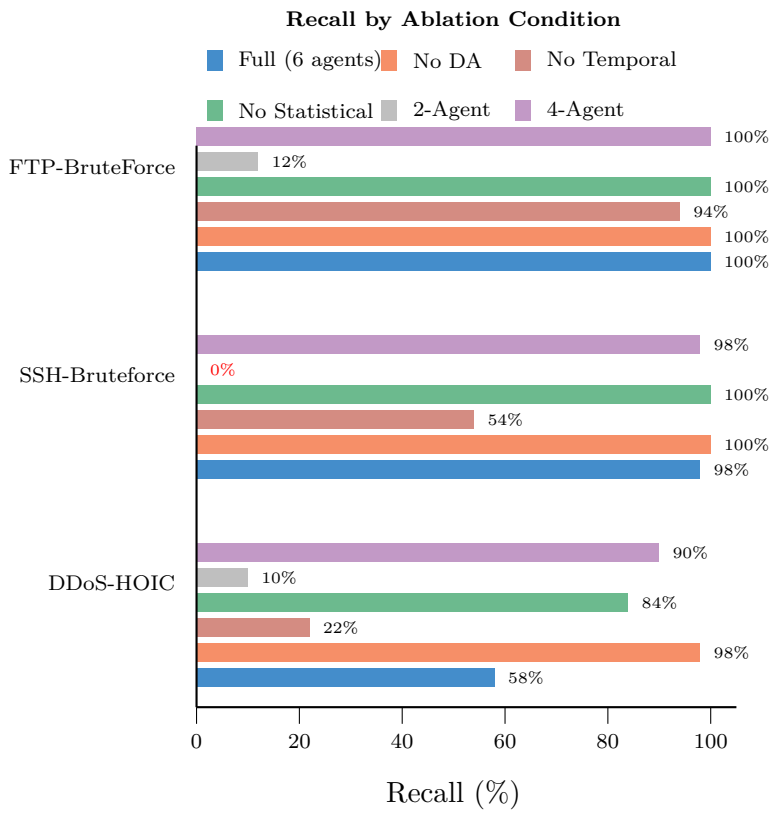

Figure 5.4: Agent ablation recall comparison across three attack types. The Devil's Advocate removal (orange) improves DDoS-HOIC recall from 58% to 98%, whilst Temporal Agent removal (maroon) degrades recall across all categories.

The DDoS-HOIC ablation results (Table 5.15) reveal a fundamentally different agent contribution profile compared to brute force attacks, and constitute the most significant architectural finding of the ablation study. Removing the Devil's Advocate on HOIC produces a 40 percentage point recall improvement (from 58% to 98%) and an F1 increase from 72% to 98%, whilst the false positive rate remains unchanged at 0.1%. This result demonstrates that the DA's fixed 30% adversarial weight, which is benign or beneficial on high-confidence attack types, actively suppresses correct detections on ambiguous attacks. The mechanism is precisely identifiable: HOIC flows generate moderate specialist confidence (typically 0.50–0.65), and the Devil's Advocate, trained to argue for the benign interpretation, provides sufficient counter-pressure at 30% weight to tip the Orchestrator's consensus below the detection threshold on approximately 40% of genuine attack flows. On brute force attacks, where specialist confidence is uniformly high (0.85–0.95), the same DA weight is easily overruled.

The Temporal Agent emerges as the single most valuable specialist across all three attack types. Its removal produced the largest performance degradation in every case: SSH-Bruteforce recall fell from 98% to 54% (a 44 percentage point decline), FTP-

BruteForce recall declined from 100% to 94%, and DDoS-HOIC recall dropped from 58% to 22% (a 36 percentage point loss). This result is architecturally coherent: both brute force and DDoS attacks are fundamentally temporal phenomena, characterised by repeated connection attempts from the same source IP across a compressed time window. The Temporal Agent's role (cross-correlating flows from the same source IP and identifying repetitive patterns such as burst, sequential, or repetitive temporal signatures) is precisely the mechanism that distinguishes sustained attack activity from isolated connection attempts. Without temporal context, the remaining agents assess each flow largely in isolation, and many attack flows are individually plausible as legitimate traffic. The HOIC result is particularly striking: without the Temporal Agent, 39 of 50 attack flows were classified as BENIGN with confidence scores of 0.75–0.95, demonstrating that HOIC's randomised HTTP flood traffic is genuinely indistinguishable from benign web traffic at the individual flow level.

The Devil's Advocate results reveal an attack-type-dependent effect that warrants careful interpretation. On FTP-BruteForce and SSH-Bruteforce, DA removal produced no change in recall or FPR; the four specialist agents generate sufficiently strong evidence that the DA counter-argument is consistently overruled. On DDoS-HOIC, however, removing the DA substantially improved recall from 58 per cent to 98 per cent whilst leaving FPR unchanged at 0.1 per cent. This result indicates that on high-ambiguity attack categories (where malicious and benign traffic share feature space), the DA's fixed 30 per cent contribution weight suppresses correct malicious verdicts that the four specialist agents would otherwise reach by consensus. DDoS-HOIC flows exhibit HTTP request patterns that partially resemble legitimate web traffic, making it straightforward for the DA to construct plausible benign counter-arguments that carry disproportionate weight relative to the specialist evidence.

This finding identifies a calibration limitation in the current DA implementation. The fixed weighting is appropriate for unambiguous attack types but overcorrects on categories with high feature overlap with benign traffic. Adaptive DA weighting (scaling the counter-argument's influence according to the specialist agents' confidence variance) would address this limitation whilst preserving the FPR control benefits observed on clear-cut attacks. The system-wide FPR reduction demonstrated in the MCP comparison (41.4 per cent without adversarial checking versus 1.1 per cent with AMATAS) confirms that the DA's overall contribution is positive; the ablation reveals that its per-attack contribution is uneven and that weighting calibration is the next design priority.

Statistical Agent removal produced no recall degradation on either brute force attack type, but on DDoS-HOIC its removal actually improved recall from 58 per cent to 84 per cent, a counterintuitive result. On DDoS-HOIC, Statistical Agent

removal also improved recall from 58 per cent to 84 per cent, suggesting that statistical features on HOIC flows generate ambiguous signals that the Statistical Agent interprets conservatively, contributing to a more cautious overall verdict. Combined with the DA finding, the DDoS-HOIC results suggest that this attack category's feature overlap with benign HTTP traffic creates a systematic bias toward under-detection in the current configuration that affects multiple agents. The Statistical Agent's primary contribution (identifying anomalous byte-per-packet ratios, inter-arrival timing deviations, and volume-based outliers) is most relevant to attack types characterised by volumetric anomalies, notably DDoS variants and DoS-Hulk, where statistical features dominate the detection signal.

The four-agent configuration (omitting both the Devil's Advocate and Temporal Agent) achieved 98–100% recall on brute force at approximately 59% of the full six-agent cost ($0.87–$0.90 versus $2.13–$2.16), and 90% recall on DDoS-HOIC at 49% of the cost ($0.87 versus $1.79). On HOIC, this four-agent configuration actually outperforms the full six-agent system by 32 percentage points of recall (90% versus 58%), because removing the DA eliminates the suppression of correct detections whilst the Temporal Agent's absence is partially compensated by the Protocol, Statistical, and Behavioural Agents operating without DA interference. The optimal configuration for DDoS-HOIC is the five-agent system without the DA (98% recall, $1.18), which retains the Temporal Agent's cross-flow correlation and eliminates only the counterproductive adversarial validation step. These results demonstrate that no single architecture is optimal across the full attack taxonomy, a finding with practical implications for deployment, where attack-type-adaptive agent selection or DA weight tuning could significantly improve overall detection performance.

Taken together, the ablation results reveal that agent contribution is attack-type-dependent rather than uniform. The Temporal Agent is essential for attacks defined by temporal patterns (brute force and DDoS), where its removal causes the largest recall degradation. The Devil's Advocate and Statistical Agent contribute positively on unambiguous attacks but introduce conservative bias on high-ambiguity categories where legitimate and malicious traffic share feature space. These findings motivate two directions for future work: attack-type-adaptive agent weighting, which would calibrate each agent's contribution according to the attack category being analysed; and a targeted evaluation of whether a four-agent configuration omitting the DA produces better system-wide performance than the fixed six-agent design on the full fourteen-category corpus.

5.9 Reasoning Faithfulness Audit

A central claim of this thesis is that multi-agent reasoning chains provide qualitatively richer explanations than traditional classifier outputs. However, reasoning chains are only valuable if the factual claims they contain are grounded in the actual flow data. To evaluate this, a programmatic faithfulness audit was conducted across all 758 LLM-analysed flows from the fourteen Stage 1 experiments (Tier 1-filtered flows were excluded as they contain no agent reasoning).

The audit extracted 6,279 verifiable factual claims from agent reasoning text and key evidence fields. Claims were identified through pattern matching across seven categories: port number references, protocol assertions, TCP flag names, exact numeric field references (e.g., "IN_BYTES 60"), natural language numeric claims (e.g., "60 bytes inbound"), IP address references, and service-port associations (e.g., "FTP service on port 21"). Each extracted claim was verified against the corresponding flow's actual feature values.

Table 5.16: Faithfulness Audit: Claim Verification by Category ($n = 6{,}279$ claims across 758 flows)

Claim Category	Claims	Correct	Faithfulness
Numeric field references	508	502	98.8%
Natural language numeric	369	363	98.4%
Port numbers	1,822	1,788	98.1%
IP addresses	314	301	95.9%
Service-port associations	609	543	89.2%
Protocol names (TCP/UDP)	940	758	80.6%
TCP flag names	1,494	1,159	77.6%
Ephemeral port classification	41	41	100.0%
Total	**6,279**	**5,637**	**89.8%**

Table 5.16 presents the verification results by claim category. Overall, 89.8% of factual claims were correct, yielding a confabulation rate of 10.2%. Faithfulness was highest for concrete numeric values: exact field references (98.8%), natural language numeric claims (98.4%), and port numbers (98.1%) were almost always reported correctly. Confabulation was concentrated in two categories. TCP flag interpretation exhibited a 22.4% error rate, where agents asserted the presence of specific flags (e.g., "RST flag set" or "PSH flag") that were not present in the flow's TCP_FLAGS bitmask. Analysis of the error distribution revealed that agents inferred flags they expected to be present based on protocol knowledge (e.g., asserting RST in a rejected connection) rather than reading the actual aggregate flag value. Protocol name

errors (19.4% error rate) followed a similar pattern: agents labelled UDP flows as TCP, particularly in Bot traffic where NTP flows on port 123 were misidentified as TCP connections.

Table 5.17: Faithfulness Audit: Per-Agent Breakdown

Agent	Claims	Correct	Faithfulness
Statistical	776	729	93.9%
Temporal	365	332	91.0%
Orchestrator	1,225	1,113	90.9%
Protocol	2,869	2,593	90.4%
Behavioural	867	728	84.0%
Devil's Advocate	177	142	80.2%

Table 5.17 presents faithfulness by agent. The Statistical Agent achieved the highest faithfulness (93.9%), consistent with its focus on numeric features where verification is unambiguous. The Devil's Advocate exhibited the lowest faithfulness (80.2%), attributable to its adversarial role: constructing counter-arguments sometimes requires referencing features in ways that prioritise narrative plausibility over strict accuracy. The Protocol Agent, despite its domain-specific mandate to validate protocol consistency, achieved only 90.4% faithfulness, with errors concentrated in TCP flag interpretation, the same bitmask-decoding weakness observed system-wide.

These results establish that AMATAS reasoning chains are predominantly grounded in actual flow data rather than confabulated, but that a non-trivial minority of claims (10.2%) contain factual errors. The errors are systematic rather than random: they concentrate in feature types (TCP flags, protocol identification) where agents apply domain knowledge to infer expected values rather than reading observed values. This pattern suggests that confabulation in LLM-based NIDS is qualitatively distinct from hallucination in open-ended text generation; it reflects over-application of prior knowledge rather than fabrication of unsupported content.

5.10 Tier 1 Routing Validation

A control experiment was conducted to validate that the Tier 1 Random Forest performs intelligent routing rather than functioning as a random cost-reduction mechanism. If the RF's contribution were merely to reduce the volume of flows forwarded to the LLM pipeline, without specifically targeting attack flows, then a random filter at the same selection rate should achieve comparable performance. Two random filter conditions were evaluated against the trained RF on the FTP-

BruteForce batch (n = 1,000; 50 attack + 950 benign), using the same six-agent pipeline and GPT-4o model as the Stage 1 baseline.

A random 7% filter (matching the RF's actual selection rate) selected 70 flows uniformly at random (seed = 42) for LLM analysis. Of these, only 3 were attack flows, reflecting the 5% attack prevalence in the batch. The LLM pipeline classified none of the 3 forwarded attack flows as malicious, yielding 0% recall across the full batch. Additionally, the random filter introduced 10 false positives from the 67 benign flows it forwarded, producing a false positive rate of 2.2%, a degradation from the trained RF's zero per cent FPR. A more aggressive random 50% filter forwarded 500 flows, capturing 28 of 50 attack flows by chance, yet still achieved 0% recall: the LLM pipeline failed to correctly classify any of the forwarded attack flows as malicious. This condition also produced 28 false positives (FPR: 5.0%) at a cost of $3.02, 42% more expensive than the trained RF whilst detecting zero attacks.

Table 5.18: Tier 1 Routing Validation: Trained RF vs. Random Filters on FTP-BruteForce

Filter Type	Attacks Forwarded	Recall	FPR	F1	Cost ($)
Trained RF (threshold 0.15)	50 / 50	100%	0.0%	100%	2.13
Random filter (7%)	3 / 50	0%	2.2%	0%	1.01
Random filter (50%)	28 / 50	0%	5.0%	0%	3.02

The divergence between the trained RF and the random filters confirms that the Tier 1 Random Forest is not merely reducing API cost through volume reduction; it is specifically identifying the flows most likely to be malicious and routing them to the LLM pipeline. The trained RF at a 7% selection rate forwards all 50 attack flows to the pipeline, achieving 100% recall; a random filter at the same selection rate forwards only 3 attack flows by chance and achieves 0% recall. Even at a 50% random selection rate (forwarding seven times more flows than the trained RF at 42% higher cost), the random filter captures only 56% of attacks by chance and still achieves 0% recall, as the randomly selected attack flows lack the clustering context and temporal structure that the RF's targeted selection preserves. This result quantifies the routing contribution precisely: the Random Forest filter isolates the specific flow distributions that align with the LLM pipeline's detection capabilities, rather than acting as a uniform volume reduction mechanism. A system that sent random flows to the LLM would require selection rates approaching 100% (eliminating the cost benefit entirely) to achieve comparable attack coverage, confirming that intelligent routing is an essential, non-substitutable component of the two-tier architecture.

Chapter 6

Discussion

6.1 Interpreting the Detection Performance

The Stage 1 evaluation results define the current functional boundaries of multi-agent LLM-based intrusion detection systems. Across all fourteen attack types, AMATAS achieves a mean F1 of 86.3% (92.9% excluding Infiltration), below the theoretical ceiling of fine-tuned discriminative models but substantially above both single-agent LLM approaches and, critically, the standalone Random Forest classifier. The RF baseline evaluation (Section 5.1.1) reveals a striking pattern: the RF achieves perfect F1 on seven attack types with distinctive statistical signatures but 0% on the remaining seven, yielding a mean F1 of only 57.1%. AMATAS detects five of those seven RF-invisible categories, demonstrating that the multi-agent LLM pipeline's value lies not in matching ML accuracy on easy attacks but in providing detection capability where statistical classifiers fail entirely. The accuracy gap between AMATAS and fine-tuned models is the cost of generality and explainability.

Direct comparison with related work is instructive. Houssel et al. [5], in the closest prior study, evaluated a standalone LLM-based NIDS and reported detection rates of approximately fifty per cent on benchmark flows, with significant false positive inflation. Houssel et al. characterise their evaluation as an "initial exploration" with limited sample sizes, so this figure should be interpreted as indicative rather than definitive; nonetheless, it establishes a baseline against which the multi-agent approach can be measured. AMATAS achieves roughly double that recall, a gain attributable to the shift from single-agent to multi-agent architecture, the adversarial validation provided by the Devil's Advocate, and the temporal context supplied by the cross-flow correlation agent. At the other extreme, Gutiérrez-Galeano et al. [19] demonstrated F1 scores of 99.84 per cent using a fine-tuned T5 language model, though the authors acknowledge effectiveness limitations on attack types that are poorly represented in the dataset, which produced elevated false negative rates

on minority classes. Their approach also produces no reasoning whatsoever; the fine-tuned model maps feature vectors to labels through learned weights that are as opaque as any traditional classifier. Li et al. [20] present IDS-Agent, a ReAct-style single-agent system reporting an F1 of 0.75 on the CIC-IoT dataset with iterative reasoning traces and tool-augmented analysis; AMATAS achieves F1=0.88 on a more comprehensive benchmark and extends this single-agent reasoning paradigm with multi-perspective specialist analysis, adversarial challenge, and consensus synthesis that distributes reasoning across six distinct analytical roles rather than a single iterative agent.

While AMATAS does not exceed the raw accuracy of fine-tuned discriminative models, it maintains an operationally viable detection rate while generating transparent, natural-language reasoning traces that traditional models do not provide. A significant constraint not addressed by the accuracy comparison is inference latency: Kalafatidis et al. [31] report that LLM processing consumed approximately 87% of total execution time in their two-tier architecture (4.53 seconds per sample), and Houssel et al. [5] document a near-7000-fold slower inference time for LLMs compared to lightweight classifiers, concluding that real-time LLM-based NIDS deployment is not currently feasible. AMATAS inherits this limitation; the six-agent pipeline processes flows in seconds rather than microseconds, restricting its applicability to batch or near-real-time analysis rather than inline traffic inspection. The comparison with Kalafatidis et al. [31], who independently propose a two-tier LLM-enhanced IDS architecture, further situates AMATAS within a converging research consensus, though AMATAS extends that pattern with a multi-agent debate structure and multi-perspective reasoning chains. Where Kalafatidis et al. [31] provide a single justification field alongside each threat assessment, AMATAS generates six distinct analytical narratives from specialist, adversarial, and synthesis perspectives, offering substantially richer explanatory depth for analyst review.

Within the Stage 1 results, the performance profile across attack types reveals important structural patterns. Brute-force and volumetric DoS categories (FTP-BruteForce (F1=100%), SSH-Bruteforce (F1=99%), DoS-Slowloris (F1=100%), DoS-SlowHTTPTest (F1=100%)) achieve the highest scores, reflecting the distinctive NetFlow signatures these attacks produce. Leevy and Khoshgoftaar [13] note that perfect or near-perfect scores on the CICIDS2018 dataset are common across published studies and may indicate overfitting to dataset-specific patterns rather than genuine detection capability. The AMATAS results are partially insulated from this concern because the LLM agents are not trained on the dataset and reason from flow features at inference time; however, the possibility that the CICIDS2018 attack signatures are more distinctive than those in real-world traffic cannot be excluded, and validation on alternative datasets remains necessary. DDoS-HOIC (F1=72%,

recall=58%) represents the hardest detectable category and is discussed at length in Section 6.4.2. Infiltration (F1=0%, recall=0%) is treated separately as a fundamental architectural limitation rather than a conventional performance shortfall.

6.2 The Explainability Advantage

This work demonstrates that while LLMs do not surpass traditional classifiers in raw accuracy, they offer competitive operational detection rates alongside detailed explanatory outputs that traditional models lack. Every AMATAS detection produces six distinct analytical outputs.

Consider flow 783 from the FTP-BruteForce batch as a representative example. The Protocol Agent classifies the flow as BENIGN at confidence 0.90, finding that the port assignment and protocol combination are individually consistent with legitimate FTP traffic. The Statistical Agent escalates to SUSPICIOUS at 0.75, flagging anomalous byte-per-packet ratios and inter-arrival timing. The Behavioural Agent returns SUSPICIOUS at 0.80, identifying a pattern consistent with credential stuffing but insufficient alone to confirm it. The Temporal Agent delivers MALICIOUS at confidence 0.95, having correlated the flow with 49 other connections from the same source IP in the analysis window; 49 connections to port 21 from a single source within a compressed time window is the decisive evidence that no single-flow analysis could provide. The Devil's Advocate acknowledges the temporal cluster but argues that the source IP could plausibly belong to a legitimate FTP client with an aggressive retry policy. The Orchestrator synthesises all five analyses into a SUSPICIOUS verdict at confidence 0.65 with consensus score 0.75, maps the finding to MITRE ATT&CK technique T1110 (Brute Force), and produces a narrative explicitly reconciling the Protocol Agent's benign reading with the Temporal Agent's cluster evidence.

To make this contrast concrete, Table 6.1 presents SHAP feature attributions from the Tier 1 Random Forest alongside AMATAS agent reasoning for five representative attack flows. The SHAP output for each flow is a ranked list of signed numeric values indicating which features pushed the classifier's prediction toward attack (positive) or benign (negative). The AMATAS output is a multi-perspective reasoning chain that contextualises those same features within protocol semantics, attack signatures, temporal patterns, and an explicit counter-argument. The Bot flow is particularly revealing: SHAP assigns exclusively negative (benign-leaning) values, consistent with the RF's P(attack) of 0.005, whilst AMATAS correctly identifies the flow as suspicious through temporal correlation with 49 other uniform flows from the same source IP. Similarly, for DDoS-HOIC, SHAP provides mixed signals with no clear attack indication, whilst AMATAS's Behavioural Agent identifies the specific pattern of short-duration HTTP sessions with asymmetric byte distributions characteristic

of HTTP flood tools.

Table 6.1: SHAP vs AMATAS: Five representative attack flows

Flow	SHAP (RF)	AMATAS (most informative agent)
FTP-BruteForce (783)	`L4_DST_PORT: +0.244L4_SRC_PORT: +0.037TCP_FLAGS: +0.035`	*Temporal Agent* (MALICIOUS, 0.95): "The flows from this source IP exhibit a pattern of repetitive, short-duration connections to the same destination IP and port (FTP service on port 21). Each flow is identical in duration, size, and TCP flags, suggesting automated behavior."
SSH-Bruteforce (29)	`L4_DST_PORT: +0.252 TCP_FLAGS: +0.062 IN_BYTES: +0.054`	*Temporal Agent* (MALICIOUS, 0.95): "The temporal analysis of the flows from this source IP indicates a sustained and repetitive pattern targeting the same destination on port 22, indicative of a brute force SSH attack."
DoS-Slowloris (763)	`L4_DST_PORT: +0.134TCP_FLAGS: +0.089L4_SRC_PORT: +0.072`	*Temporal Agent* (MALICIOUS, 0.95): "The source IP shows a pattern of frequent, short-duration flows targeting the same destination IP and port (80). There is a notable burst activity with almost identical flows repeated multiple times."
Bot (709)	`L4_DST_PORT: -0.208 OUT_BYTES: -0.068 IN_BYTES: -0.054`	*Temporal Agent* (MALICIOUS, 0.95): "The source IP has produced similar flows to the same destination and port with uniform packet sizes, flow durations, and TCP flags, indicating a potential automated tool."
DDoS-HOIC (756)	`OUT_PKTS: -0.126 IN_BYTES: -0.069 L4_DST_PORT: +0.061`	*Behavioural Agent* (MALICIOUS, 0.90): "The flow involves HTTP (port 80) with very short duration sessions (16 ms) and a higher number of outbound bytes compared to inbound bytes typical of data requests, consistent with HTTP-based flood tools."

This output is qualitatively different from what SHAP or LIME explanations provide for traditional classifiers. SHAP values tell an analyst which features influenced a model's output; they do not articulate what those features mean in the context of the specific flow, how they relate to known attack patterns, whether a reasonable benign interpretation exists, or how competing evidence was weighed. AMATAS produces all of these, in natural language accessible to analysts without machine learning expertise. This aligns with the chain-of-thought reasoning framework established by Wei et al. [23], who demonstrate that eliciting intermediate reasoning steps substantially improves LLM performance on multi-step tasks, though they caution that there is no guarantee that generated reasoning paths are factually correct, that models can produce correct final answers through entirely incorrect reasoning, and that whether LLMs are genuinely "reasoning" remains an open question. This caveat applies directly to AMATAS, and the faithfulness audit (Section 5.9) quantifies its extent: across 6,279 verifiable factual claims extracted from agent reasoning chains, 89.8% were confirmed correct against actual flow data. Confabulation concentrated in TCP flag interpretation (22.4% error rate) and protocol identification (19.4%), where agents inferred expected values from domain knowledge rather than reading observed feature values. Numeric features (byte counts, packet counts, port numbers) were faithfully reported at greater than 98% accuracy. This error profile is qualitatively distinct from open-ended hallucination: agents over-apply prior knowledge about protocol behaviour rather than fabricating unsupported content. The practical implication is that the reasoning traces are both inspectable and predominantly accurate: an analyst can read each agent's rationale, assess whether the cited evidence is compelling, and decide whether the conclusion follows, with confidence that the vast majority of factual claims are grounded in the actual flow data. Hesford et al. [2] underscore the urgency of this capability, demonstrating empirically that IDS performance degrades substantially between benchmark evaluation and real-world deployment, a gap that opaque classifiers give analysts no tools to diagnose. The practical implication is that AMATAS supports analyst decision-making rather than displacing it: an analyst reviewing a verdict can assess the reasoning, identify which agent analysis they find unconvincing, and apply domain expertise to confirm or override the system's conclusion.

6.3 The False Positive Problem Resolved

The zero per cent false positive rate across thirteen of fourteen attack types is perhaps the most practically significant result of this evaluation. Alert fatigue, the systematic desensitisation of security analysts to alerts generated by over-sensitive detection systems, is one of the most well-documented operational problems in network security.

Alahmadi, Axon and Martinovic [3] report that analysts in high-volume environments routinely ignore substantial fractions of generated alerts, with the consequence that genuine threats embedded within alert storms go uninvestigated. Alahmadi et al. further note that "false positive" is itself a broad metric in operational settings, distinguishing between true false alarms and "benign triggers" where the detection is technically correct but disregarded for business reasons. AMATAS's zero per cent FPR is measured against ground-truth labels rather than operational triage decisions, and real-world FPR perception would depend on the deployment context. However, Alahmadi, Axon and Martinovic [3] caution that the term "false positive" is frequently conflated with "benign trigger" in operational contexts: many alerts that analysts classify as false positives are in fact true signature matches (e.g., detecting a vulnerable software version) that the organisation chooses to suppress for business reasons. Real-world alert fatigue is therefore driven partly by organisational context, not solely by model inaccuracy, and a 0% FPR on a benchmark dataset does not guarantee elimination of alert fatigue in production deployment. Nonetheless, reducing technologically generated false positives remains a necessary condition for operational viability, even if it is not sufficient. At the CICIDS2018 dataset scale of twenty million flows, even a one per cent false positive rate would generate two hundred thousand spurious alerts, a volume that would render any system operationally useless.

The mechanism responsible for AMATAS's false positive suppression is the Devil's Advocate agent operating at a weight of thirty per cent in the Orchestrator's consensus calculation. This weight was calibrated through empirical ablation: at fifty per cent (Phase 3e), the agent became excessively aggressive, depressing recall below acceptable thresholds; at thirty per cent, it applies sufficient adversarial weighting to prevent the specialist agents from over-classifying ambiguous flows as malicious, without overwhelming genuinely malicious signals. This finding provides the first demonstration of the adversarial agent mechanism, described conceptually by Chiang et al. [9] and the multi-agent debate literature [7, 8], applied specifically to network intrusion detection with quantified performance consequences across a fourteen-category evaluation.

The ablation study (Section 5.8) adds nuance to this picture: the DA's fixed weighting reduces recall on DDoS-HOIC from 98 per cent (no DA) to 58 per cent, indicating that on attacks with high feature overlap with benign traffic, the adversarial counter-argument carries disproportionate weight. The appropriate response is not to remove the DA but to make its weighting adaptive, a design change that would preserve its FPR control benefits on unambiguous attacks whilst reducing its suppressive effect on high-ambiguity categories.

6.4 Limitations and Failure Modes

6.4.1 The Infiltration Failure

AMATAS failed to detect any of the fifty attack flows in the Infiltration category. This indicates an inherent limitation of flow-level analysis rather than a deficiency in model selection or prompt design. Infiltration attacks in the CICIDS2018 dataset are implemented via a simulated remote code execution chain that, at the individual flow level, generates traffic indistinguishable from normal HTTPS or SMB communication. There is no anomalous packet rate, no unusual port combination, no volume spike; the attacking flows conform in every measurable dimension to the statistical profile of the benign traffic in which they are embedded.

The solution this failure motivates is temporal clustering, the v3 architectural extension described in Section 7.3. The hypothesis is that providing the agent pipeline with ordered sequences of flows from the same source IP, rather than individual flows, would reveal the sequential access progression characterising infiltration attacks. Individual flows from an infiltrating host look benign; the ordered sequence of those flows (accessing a progression of internal services over a compressed time window) may not.

6.4.2 DDoS-HOIC and Feature Overlap

DDoS-HOIC achieves recall of 58 per cent and F1 of 72 per cent, the lowest among successfully detected categories. The High Orbit Ion Cannon tool generates HTTP floods using randomised headers, user agents, and URL paths specifically to evade signature-based detection. The resulting flows share characteristics with high-volume legitimate web traffic: they use standard ports, conform to expected HTTP packet size distributions, and do not exhibit the periodicity that characterises simpler volumetric attacks. The Statistical Agent correctly identifies anomalous flow rates in 58 per cent of cases, but in the remaining 42 per cent the HOIC randomisation pushes the flow's statistical profile into the range occupied by legitimate web servers under genuine load.

6.4.3 The Semantic Gap in Flow-Level Analysis

Both the Infiltration failure and the HOIC difficulty point to a broader limitation identified by Sommer and Paxson [40]: the semantic gap between the statistical features of network traffic and their operational interpretation. Sommer and Paxson warn that network traffic features exhibit "enormous variability" and "much more diversity than people intuitively expect," making it difficult to establish a stable notion

of normality. They further caution that machine learning methods are better suited to finding similarities than to identifying activity that does not belong, a property they term the closed-world assumption. The AMATAS architecture partially addresses this gap by translating raw numerical features into natural language descriptions that the LLM can contextualise against its pre-trained knowledge of network protocols and attack patterns. However, this translation does not eliminate the underlying limitation: when malicious flows are statistically indistinguishable from benign traffic at the feature level, no amount of reasoning over those features can recover the missing signal. The Infiltration and HOIC results provide concrete evidence of this boundary.

6.4.4 MCP Tool Utility and Anonymised Datasets

The MCP comparison experiment evaluates two hypotheses: first, that IP-dependent threat intelligence tools offer no performance benefit on anonymised address spaces; and second, that dataset-compatible tools (which return genuinely informative results) improve single-agent detection. The empirical results support the first hypothesis and refute the second.

Configurations A–C confirm that IP-dependent tools provide negligible benefit: Configuration C (engineered prompt with MITRE ATT&CK tool) achieves F1=58.3 per cent versus F1=58.0 per cent for Configuration B (engineered prompt alone), a difference of 0.3 percentage points at additional cost. Tools querying AbuseIPDB, AlienVault OTX, and geolocation services return no actionable information on RFC 1918 private IP addresses.

The more significant finding comes from Configurations D–G, which test tools that function correctly on anonymised data: IANA port-service lookups, IP protocol decoders, TCP flag analysis, SANS DShield global attack statistics, CVE vulnerability databases, and MITRE ATT&CK technique descriptions. These tools returned genuinely informative results (port 22 is registered for SSH, port 22 is heavily targeted globally with thousands of unique attackers, OpenSSH has 847 known CVEs). Despite this, all four configurations performed *worse* than the no-tool baseline: Config D achieved F1=36.7%, Config E F1=46.9%, Config F F1=37.9%, and Config G F1=39.3%, compared to Config B's F1=58.0% with no tools at all.

This result extends the tool-augmented LLM literature in an unexpected direction. Yao et al. [24] demonstrate that non-informative tool results derail reasoning; the AMATAS extended comparison demonstrates that *informative* tool results can be equally detrimental when the information is true but irrelevant to the classification task. Knowing that SSH is a globally targeted service does not help determine whether a specific SSH flow is malicious; the discriminative signal lies in the flow's

timing, byte distribution, and temporal context, not in generic threat intelligence about the destination port. The agent anchors on the tool-provided context at the expense of direct feature analysis, a form of reasoning displacement that Qin et al. [27] identify as a risk when tool outputs are more salient than the underlying data. The non-monotonic relationship between tool count and performance (Config E outperforms Configs F and G despite having fewer tools) suggests that additional tools compound the distraction effect rather than providing incremental value.

The 25.2 percentage-point F1 gap between the best single-agent configuration (Config A, F1=62.8%) and AMATAS (F1=88%), combined with the finding that tool augmentation actively degrades single-agent performance, establishes that the multi-agent architecture's value derives from distributed specialist reasoning and adversarial validation rather than from external knowledge enrichment. This gap is not an artefact of model selection (Config A uses GPT-4o-mini whilst AMATAS uses GPT-4o) but a structural consequence of the architectural differences: four specialist perspectives, adversarial counter-argumentation, and Orchestrator synthesis collectively overcome the performance limitations observable in any single-agent configuration.

The agent ablation study (Section 5.8) provides direct evidence for the contribution of agent specialisation, and the extension to DDoS-HOIC reveals that agent contributions are attack-type-dependent in ways that brute force testing alone could not have established. The Temporal Agent alone accounts for a 44 percentage point recall difference on SSH-Bruteforce and a 36 percentage point difference on DDoS-HOIC, a result that aligns with the multi-agent debate framework of Du et al. [7], who show that multiple model instances debating a question significantly outperform any single instance, with gains particularly pronounced on tasks requiring multi-step reasoning. The two-agent degradation to 0% recall on SSH-Bruteforce and 10% on DDoS-HOIC further confirms that multi-agent debate is not merely additive but constitutive of detection capability on attacks requiring temporal correlation. A related concern noted by both Du et al. and Chan et al. is that LLM performance can degrade as input context length increases, a risk structurally present in the Orchestrator agent, which receives all five preceding analyses. Future work should investigate whether context compression techniques can mitigate potential quality degradation at scale.

The most significant ablation finding is the Devil's Advocate's attack-type-dependent contribution profile. On brute force attacks, where specialist confidence is uniformly high, DA removal produces no degradation; on DDoS-HOIC, where specialist confidence is moderate (0.50–0.65), DA removal improves recall by 40 percentage points (58% to 98%) and F1 by 26 percentage points (72% to 98%) without increasing the false positive rate. This result demonstrates that the fixed

30% DA weight, calibrated for brute force performance, is counterproductive on ambiguous attacks; the adversarial counter-argument suppresses correct but uncertain detections. The practical implication is that the DA weight should be adaptive: high when specialist confidence is high (to catch false positives), low when confidence is moderate (to avoid suppressing true positives). This finding extends the multi-agent debate literature [9, 7, 8] with empirical evidence that the contribution of an adversarial counter-argument agent is non-monotonic in classification difficulty, and identifies a concrete engineering direction, confidence-gated adversarial weighting, for improving multi-agent NIDS architectures. An important caveat from the multi-agent debate literature applies here: Du et al. [7] demonstrate that multi-agent debates can converge on a consensus that is incorrect, with agents confidently affirming their answer as correct and consistent with other agents despite being wrong. In the NIDS context, this false consensus failure mode would manifest as all specialist agents agreeing on a classification (whether benign or malicious) with high confidence, producing a plausible but incorrect reasoning chain. The Devil's Advocate mechanism partially mitigates this risk by structurally requiring a counter-argument regardless of specialist agreement, but it cannot detect cases where all specialists share the same incorrect premise. The multi-agent architecture also carries an inherent computational cost that Du et al. [7] identify as a primary limitation: multi-agent debate requires multiple language generations and an underlying debate procedure, fundamentally multiplying token costs and inference latency per analysed flow relative to single-agent approaches.

The random filter control experiment (Section 5.10) establishes that the Tier 1 Random Forest contributes intelligent routing rather than random cost reduction. The divergence between 100% recall at 7% trained RF selection and 0% recall at 50% random selection quantifies the routing contribution precisely: the Random Forest filter isolates the specific flow distributions that align with the LLM pipeline's detection capabilities, rather than acting as a uniform volume reduction mechanism. This result has implications beyond the AMATAS architecture: any two-tier system combining a cheap discriminator with an expensive analytical backend must demonstrate that the discriminator's selection is informed rather than arbitrary, and the random filter control provides a reusable experimental template for establishing this property.

6.4.5 Model Selection Constraints and Generalisability

The exclusive reliance on GPT-4o means that reported performance figures may not generalise to other frontier LLMs. GPT-4o-mini exhibited a systematic false positive bias, classifying every flow as malicious regardless of content, which rendered

it unusable for NIDS despite its cost advantage. Claude Sonnet 3.5 was rejected due to rate limits incompatible with 1000-flow batch processing and an estimated cost of approximately $108 per batch. The multi-agent architecture is model-agnostic in principle, but the specific calibration of agent prompts, confidence thresholds, and Devil's Advocate weights was developed against GPT-4o behaviour. Alternative models may exhibit different reasoning patterns, hallucination profiles, and confidence tendencies requiring re-calibration.

The CICIDS2018 dataset, whilst the most comprehensive publicly available benchmark [36, 13], is a simulation of network traffic generated in a controlled laboratory environment. The FLOW_DIRECTION feature was unavailable in the CICIDS2018 NetFlow v3 encoding and was excluded from the Stage 1 feature set, which may limit discriminative power on directionally distinctive attacks. Sarhan et al. [32] note several limitations of the NetFlow-based representation, and Feng and Sakurai [18] contextualise dataset representativeness limitations within the broader challenge of constructing reliable NIDS evaluation benchmarks.

6.5 Security Considerations for MCP-Enhanced Detection Systems

The MCP comparison experiments demonstrate that external tool access provides negligible benefit on anonymised datasets, but a production deployment of MCP-enhanced intrusion detection against live network traffic would face a qualitatively different threat model. The security properties of the MCP architecture itself (the protocol through which an LLM agent discovers, invokes, and trusts external tools) become critical concerns when the system operates on real traffic with genuine IP addresses, domain names, and payload content.

The most immediate risk is tool injection, where a compromised or malicious MCP server returns fabricated results designed to influence the LLM agent's classification. An attacker who controls an upstream threat intelligence feed could supply false reputation data, reporting a known malicious IP address as benign, or conversely, poisoning the reputation of legitimate infrastructure to trigger false positives at scale. The LLM agent, lacking independent means to verify the provenance or integrity of tool responses, would incorporate the fabricated data into its reasoning as though it were authoritative. This vulnerability is structural rather than incidental: the MCP specification delegates trust to the tool server, and no mechanism within the current protocol provides for cryptographic verification of response integrity or cross-validation against independent sources. Maloyan and Namiot [43] identify three fundamental protocol-level vulnerabilities in MCP-based architectures: the absence

of capability attestation (servers cannot prove they are authorised to provide specific tools), bidirectional sampling risks (servers can invoke the LLM to execute unintended actions), and implicit trust propagation across the client-server boundary. Their proposed ATTESTMCP framework addresses these through capability attestation certificates, message authentication, and origin tagging of tool responses, providing a concrete defence-in-depth model for securing MCP deployments in security-critical contexts.

Permission escalation presents a related concern. An MCP server designed to provide read-only access to threat intelligence databases could, through crafted tool descriptions or parameter manipulation, induce the LLM agent to execute actions beyond its intended scope (writing to configuration files, modifying firewall rules, or exfiltrating sensitive flow data to external endpoints). The risk is amplified in intrusion detection contexts where the system necessarily processes sensitive network telemetry, and where an escalation from analysis to action could enable an attacker to manipulate the very defences the system is meant to provide.

Data exfiltration through MCP tool calls represents a third category of risk. Each tool invocation transmits a structured request to an external server, and in a NIDS context these requests may contain IP addresses, port numbers, flow statistics, and other telemetry that an adversary could aggregate to map the defended network's topology. Even if individual tool calls reveal limited information, the aggregate of thousands of calls across an analysis session could provide substantial intelligence to a patient adversary controlling or monitoring the tool server.

These considerations do not invalidate the MCP architecture for production deployment, but they establish that any future integration of external tools into the AMATAS pipeline must be accompanied by a security layer that the current MCP specification does not provide. The AMATAS design decision to embed domain knowledge in agent prompts rather than in external tool calls is, in retrospect, not merely a consequence of the anonymised dataset's incompatibility with threat intelligence lookups, but a security-conservative architectural choice that eliminates the tool-mediated attack surface entirely for the current deployment context.

6.6 Threats to Validity

With respect to internal validity, each attack-type batch is constructed from a single random seed, meaning that performance figures reflect one specific sample of flows per category. A different seed might produce a batch with a different distribution of edge cases, potentially altering recall figures for categories with moderate sample-level variance. As Cawley and Talbot [39] emphasise, evaluation on a single partition may arbitrarily favour one configuration over another; multi-seed evaluations establishing

confidence intervals are necessary to quantify this variance and are identified as a priority for future work.

With respect to construct validity, AMATAS produces three-class verdicts (BENIGN, SUSPICIOUS, and MALICIOUS), but the evaluation framework collapses SUSPICIOUS and MALICIOUS into a single detected category for binary metric calculation. This collapse is justified operationally (a SUSPICIOUS verdict triggers investigation just as a MALICIOUS verdict does), but it means that reported recall figures include flows classified at relatively low confidence, and a stricter MALICIOUS-only evaluation would produce lower recall figures.

The multi-agent architecture introduces a context length concern identified by both Chan et al. [8] and Du et al. [7]: as debate context grows, LLM performance can stagnate or degrade, with models tending to focus only on the most recent inputs. In the AMATAS pipeline, the Orchestrator receives the combined outputs of five preceding agents, which can exceed several thousand tokens. Whilst the current sequential (non-iterative) architecture limits context accumulation compared to the multi-round debates studied by Chan et al. and Du et al., flows with unusually long agent outputs may approach the threshold where context length degrades synthesis quality, a risk that would increase with any future extension to iterative multi-round debate.

The economic projection assumes that the Tier 1 Random Forest's one-hundred-per-cent recall on the development partition generalises to evaluation batches, and that cost per LLM call remains constant across flow types. In practice, the Temporal Agent's cost varies with the number of co-IP flows available for context, and flows with more complex feature patterns generate longer reasoning chains with higher token counts.

With respect to external validity, the CICIDS2018 dataset was generated in a controlled laboratory environment, and Kenyon, Deka and Bhowmik [37] and Ring et al. [38] caution that traffic generated in academic cyber-ranges may not represent the variability of real-world corporate and public networks. Performance figures obtained on this benchmark may not transfer directly to production traffic, where attack signatures, benign traffic profiles, and class distributions differ from the laboratory conditions. Additionally, the exclusive reliance on GPT-4o means that reported results reflect the behaviour of a single model family. Chan et al. [8] note that their multi-agent debate findings were validated only on homogeneous groups of LLMs from the same model family, and the transfer of these results to heterogeneous model configurations remains an open question relevant to any future multi-model deployment of AMATAS.

Chapter 7

Conclusion

7.1 Summary of Contributions

This thesis investigated whether multi-agent LLM architectures can provide intrusion detection that balances operational accuracy with model transparency. The experimental results demonstrate that the proposed system satisfies both objectives.

The first contribution is the first systematic, per-attack-type evaluation of a multi-agent LLM intrusion detection system at realistic class distributions. Prior evaluations of LLM-based NIDS used balanced datasets or aggregated multi-attack batches, neither of which reflects the class imbalance of production networks. The Stage 1 evaluation reveals a performance range from F1=100% on brute-force and DoS categories to F1=0% on Infiltration, a range that would be invisible in aggregate evaluation. The necessity of specialist agents is confirmed by an ablation study demonstrating that the Temporal Agent alone accounts for a 44 percentage point recall difference on SSH-Bruteforce and 36 percentage points on DDoS-HOIC, and that a two-agent configuration degrades to 0–10% recall across all tested attack types. The HOIC extension reveals that the fixed 30% weight assigned to the Devil's Advocate agent suppresses correct detections on ambiguous attacks, improving recall by 40 percentage points when removed, demonstrating that optimal agent configuration is attack-type-dependent.

The second contribution is a two-tier ML-plus-LLM architecture reducing the cost of LLM-based detection by 94.6 per cent. Actual expenditure across all fourteen Stage 1 experiments was $27.35, against an estimated $509.78 without Tier 1 pre-filtering. The Random Forest at threshold 0.15 achieves one hundred per cent recall on attack flows in the development partition, demonstrating that operating costs can be reduced without degrading the detection rate. A random filter control experiment validates that the RF provides intelligent routing: a random 7% selection achieves zero per cent recall whilst the trained RF at the same selection rate achieves 100 per

cent, confirming that the cost reduction is inseparable from the routing intelligence. This two-tier pattern is generalisable to any domain where LLM analysis is accurate but expensive and a cheaper discriminator can identify the easy cases.

The third contribution is the introduction and empirical evaluation of a Devil's Advocate agent as a structural false positive control mechanism. The DA achieves zero per cent FPR across thirteen of fourteen attack categories, but the HOIC ablation reveals that the fixed 30% weight is counterproductive on ambiguous attacks; removing the DA improves recall from 58% to 98% on DDoS-HOIC without increasing the false positive rate. This dual finding extends the multi-agent debate literature [9, 7, 8] with the first evidence that adversarial agent effectiveness is non-monotonic in classification difficulty, and identifies confidence-gated adversarial weighting as a concrete engineering direction for multi-agent security systems.

The fourth contribution is a complete explainability framework where every detection decision is accompanied by multi-perspective reasoning chains from six analytical agents, operationalised through the Flow Inspector interface. Unlike SHAP or LIME post-hoc explanations, AMATAS reasoning chains are generated as part of the detection process itself, grounded in domain knowledge, and structured to present both evidence for malice and the best available counter-argument. A faithfulness audit verified that 89.8% of factual claims in agent reasoning are correct against actual flow data, establishing that the reasoning chains are predominantly grounded in observed evidence rather than confabulated.

The fifth contribution is a controlled seven-configuration ablation study quantifying the marginal value of each architectural addition, from zero-shot single agent (F1=62.8%) through prompt-engineered single agent (F1=58.0%) and tool-augmented configurations (F1=36.7–58.3%) to multi-agent AMATAS (F1=88%). The extended comparison (Configurations D–G) demonstrates that even dataset-compatible tools that return genuinely informative results actively degrade single-agent performance, establishing that multi-agent debate rather than tool augmentation is the critical architectural contribution.

7.2 Answers to the Research Questions

RQ1 asked whether a multi-agent LLM architecture can achieve detection accuracy comparable to traditional machine learning classifiers. AMATAS achieves mean F1 of 86.3% across all fourteen attack types (92.9% excluding Infiltration), which falls below the theoretical ceiling of fine-tuned discriminative models that report F1 exceeding 99% on CICIDS2018. However, a standalone evaluation of the Tier 1 Random Forest (Section 5.1.1) reveals that this comparison is misleading: the RF achieves mean F1 of only 57.1%, with perfect detection on seven attack types but

complete failure (0% F1) on the remaining seven. AMATAS detects five of those seven RF-invisible categories at F1 $\geq$ 72%, demonstrating that the multi-agent LLM pipeline provides detection capability on attack types where statistical classifiers fail entirely. The accuracy gap between AMATAS and fine-tuned models is the cost of generality and explainability: AMATAS produces multi-perspective reasoning chains for every verdict, a capability that no traditional classifier provides. The ablation study confirms that this performance requires multi-agent specialisation; a minimal two-agent configuration achieves only 0–12% recall on brute force and 10% on DDoS-HOIC, and the Temporal Agent's removal alone reduces recall by 36–44 percentage points. The HOIC ablation further reveals that the optimal agent configuration is attack-type-dependent: removing the Devil's Advocate improves HOIC recall from 58% to 98%, demonstrating that fixed adversarial weighting is suboptimal across the full attack taxonomy.

RQ2 asked how a two-tier architecture affects the cost-accuracy trade-off. The Tier 1 Random Forest filters approximately ninety-four per cent of flows as confidently benign, reducing Stage 1 cost from an estimated $509.78 to an actual $27.35, a 94.6 per cent reduction, whilst maintaining one hundred per cent recall on attack flows at the operating threshold. The resulting cost of approximately $0.002 per flow makes LLM-based analysis economically viable for batch processing of millions of flows.

RQ3 asked about the quality and utility of multi-agent reasoning chains relative to opaque classifier outputs. The reasoning chains generated by AMATAS are qualitatively different from feature attribution outputs in several respects: they are generated as part of detection rather than as post-hoc approximations; they incorporate domain knowledge about attack signatures, protocol semantics, and MITRE ATT&CK techniques; they present structured disagreement between analytical perspectives; and they include an explicit counter-argument. A faithfulness audit across 758 LLM-analysed flows verified that 89.8% of 6,279 factual claims in agent reasoning were correct against actual flow data, with confabulation concentrated in TCP flag interpretation (22.4% error rate) and protocol identification (19.4%) rather than in the numeric features that dominate detection logic. These results demonstrate that AMATAS reasoning chains are predominantly grounded in observed evidence, and that the structural properties of the multi-agent framework (specificity, grounding, multi-perspectival structure, and falsifiability) are complemented by empirically verified factual accuracy.

RQ4 asked how detection performance varies across attack types and what the fundamental limitations of flow-level LLM analysis are. Performance ranges from F1=100% for attacks with distinctive flow signatures to F1=0% for Infiltration, where individual attack flows are statistically indistinguishable from benign traffic. No single-flow analysis method, LLM or otherwise, can reliably detect attacks that

have been designed to resemble benign traffic at the individual flow level. The v3 temporal clustering experiment (Section 5.7.1) partially addresses this limitation, recovering 58% recall on Infiltration by presenting agents with multi-flow clusters rather than isolated records, though at the cost of an elevated 19.2% false positive rate that underscores the fundamental difficulty of this attack category.

7.3 Future Work

The AMATAS v3 temporal clustering experiment (Section 5.7.1) demonstrated that grouping flows by source IP recovers substantial detection capability on Infiltration, raising recall from 0% to 58% under the best condition. However, the elevated false positive rate of 19.2% (nearly four times the worst-case FPR observed in the remaining thirteen Stage 1 experiments) indicates that the clustering mechanism requires further refinement. Promising directions include adaptive clustering thresholds that vary the DNS-query trigger based on historical IP behaviour, post-clustering confidence calibration to suppress false positives without sacrificing recall, and a dedicated Infiltration-specific agent trained on multi-stage intrusion patterns. Additionally, extending the clustering evaluation to DDoS-HOIC and other attack types where temporal context may improve detection would establish the generalisability of the v3 architecture beyond the single category tested here.

The ablation study identifies adaptive agent weighting as an immediate priority. The Devil's Advocate's fixed 30 per cent contribution suppresses recall on DDoS-HOIC by 40 percentage points relative to the no-DA configuration, whilst leaving brute force recall unchanged. A weighting scheme that scales agent contributions according to inter-agent confidence variance (reducing the DA's weight when specialist agents are in strong agreement and increasing it when they diverge) would address this limitation without sacrificing the false positive control that motivates the adversarial mechanism. This represents the most immediately actionable architectural improvement identified by the evaluation. Extending the ablation to the remaining eleven categories, particularly Bot (F1=85%) and the DoS variants, would establish whether the DA's counterproductive effect on HOIC generalises to all moderate-confidence attack types, and the four-agent configuration's 90% recall on HOIC at 49% cost motivates investigation of attack-type-adaptive agent selection, where a meta-classifier selects the optimal agent subset based on the Tier 1 RF's confidence profile.

Evaluation across alternative frontier LLMs is a priority for external validity. Claude Sonnet 4, Llama-3.3-70B, and Gemini 1.5 Pro are candidates for comparative evaluation; each would require re-calibration of the Devil's Advocate weight and Orchestrator consensus thresholds against the model's specific reasoning and confidence

patterns. Ferrag et al. [12] document substantial performance variation across deep learning architectures on the same dataset, and equivalent or greater sensitivity to model choice should be expected for LLM-based systems where reasoning patterns differ more fundamentally between model families.

A production deployment integrating MCP-based tool access against live traffic with real IP addresses represents a natural extension of the comparison experiments. The null result observed on anonymised data does not preclude substantial tool utility on production traffic, where AbuseIPDB reputation scores, geolocation data, and MITRE ATT&CK technique lookups could provide contextual enrichment that the agents' parametric knowledge cannot. Such a deployment would require addressing the security considerations discussed in Section 6.5, particularly tool response verification and permission boundaries, however, the MCP server infrastructure developed for the comparison experiments can be adapted to support this extension.

The held-out test partition (8.05 million flows excluded from all development and validation decisions) serves as the primary evaluation of AMATAS generalisability and should be treated as a one-shot evaluation with no post-hoc parameter adjustment. Longer-term directions include LoRA-adapted specialist models replacing each agent with a fine-tuned variant trained on agent-specific analytical labels, a streaming deployment architecture for real-time flow processing with asynchronous LLM queuing, and a formal user study providing empirical grounding for the explainability claims that this thesis argues for on structural grounds.

References

[1] IBM Security. *Cost of a Data Breach Report 2024*. Tech. rep. Armonk, NY: IBM Corporation, 2024.

[2] J. Hesford, D. Cheng, A. Wan, L. Huynh, S. Kim, H. Kim and J. B. Hong. 'Expectations Versus Reality: Evaluating Intrusion Detection Systems in Practice'. In: *Proceedings of the 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks — Supplemental Volume (DSN-S 2025)*. 2025. arXiv: 2403.17458 [cs.CR].

[3] B. A. Alahmadi, L. Axon and I. Martinovic. '99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms'. In: *Proceedings of the 31st USENIX Security Symposium (USENIX Security '22)*. 2022, pp. 2783–2800.

[4] Anthropic. *Model Context Protocol Specification*. 2024. URL: https://modelcontextprotocol.io (visited on 01/03/2026).

[5] P. R. B. Houssel, P. Singh, S. Layeghy and M. Portmann. 'Towards Explainable Network Intrusion Detection Using Large Language Models'. In: *Proceedings of the 2024 IEEE/ACM International Conference on Big Data Computing, Applications and Technologies (BDCAT 2024)*. 2024, pp. 67–72. DOI: 10.1109/BDCAT63179.2024.00021.

[6] M. Minsky. *The Society of Mind*. New York: Simon and Schuster, 1986.

[7] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum and I. Mordatch. 'Improving Factuality and Reasoning in Language Models through Multiagent Debate'. In: *arXiv preprint arXiv:2305.14325* (2023). arXiv: 2305.14325 [cs.CL].

[8] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu and Z. Liu. 'ChatEval: Towards Better LLM-Based Evaluators through Multi-Agent Debate'. In: *arXiv preprint arXiv:2308.07201* (2023). arXiv: 2308.07201 [cs.CL].

[9] C.-W. Chiang, Z. Lu, Z. Li and M. Yin. 'Enhancing AI-Assisted Group Decision Making through LLM-Powered Devil's Advocate'. In: *Proceedings of the 29th*

International Conference on Intelligent User Interfaces (IUI '24). 2024, pp. 103–119. DOI: 10.1145/3640543.3645199.

[10] M. Roesch. 'Snort — Lightweight Intrusion Detection for Networks'. In: *Proceedings of the 13th USENIX Conference on System Administration*. USENIX Association, 1999, pp. 229–238.

[11] Open Information Security Foundation. *Suricata — Open Source IDS/IPS/NSM Engine*. 2010. URL: https://suricata.io (visited on 01/03/2026).

[12] M. A. Ferrag, L. Maglaras, S. Moschoyiannis and H. Janicke. 'Deep Learning for Cyber Security Intrusion Detection: Approaches, Datasets, and Comparative Study'. In: *Journal of Information Security and Applications* 50 (2020), p. 102419. DOI: 10.1016/j.jisa.2019.102419.

[13] J. L. Leevy and T. M. Khoshgoftaar. 'A Survey and Analysis of Intrusion Detection Models Based on CSE-CIC-IDS2018 Big Data'. In: *Journal of Big Data* 7.1 (2020), p. 104. DOI: 10.1186/s40537-020-00382-x.

[14] G. Haixiang, L. Yijing, J. Shang, G. Mingyun, H. Yuanyue and G. Bing. 'Learning from Class-Imbalanced Data: Review of Methods and Applications'. In: *Expert Systems with Applications* 73 (2017), pp. 220–239. DOI: 10.1016/j.eswa.2016.12.035.

[15] S. M. Lundberg and S.-I. Lee. 'A Unified Approach to Interpreting Model Predictions'. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 4765–4774.

[16] M. T. Ribeiro, S. Singh and C. Guestrin. '"Why Should I Trust You?": Explaining the Predictions of Any Classifier'. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 1135–1144. DOI: 10.1145/2939672.2939778.

[17] M. Gupta, C. Akiri, K. Aryal, E. Parker and L. Praharaj. 'From ChatGPT to ThreatGPT: Impact of Generative AI in Cybersecurity and Privacy'. In: *IEEE Access* 11 (2023), pp. 80218–80245. DOI: 10.1109/ACCESS.2023.3300381.

[18] Y. Feng and K. Sakurai. 'Network Intrusion Detection: Evolution from Conventional Approaches to LLM Collaboration and Emerging Risks'. In: *arXiv preprint arXiv:2510.23313* (2025). arXiv: 2510.23313 [cs.CR].

[19] L. Gutiérrez-Galeano, J. Domínguez-Jiménez, J. Schäfer and I. Medina-Bulo. 'LLM-Based Cyberattack Detection Using Network Flow Statistics'. In: *Applied Sciences* 15.12 (2025), p. 6529. DOI: 10.3390/app15126529.

[20] Y. Li, Z. Xiang, N. D. Bastian, D. Song and B. Li. 'IDS-Agent: An LLM Agent for Explainable Intrusion Detection in IoT Networks'. In: *NeurIPS 2024 Workshop on Open-World Agents*. 2024.

[21] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang and C. Wang. 'AutoGen: Enabling Next-Generation LLM Applications via Multi-Agent Conversation'. In: *arXiv preprint arXiv:2308.08155* (2023). arXiv: 2308.08155 [cs.AI].

[22] Y. Yang, Y. Ma, H. Feng, Y. Cheng and Z. Han. 'Minimizing Hallucinations and Communication Costs: Adversarial Debate and Voting Mechanisms in LLM-Based Multi-Agents'. In: *Applied Sciences* 15.7 (2025), p. 3676. DOI: 10.3390/app15073676.

[23] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le and D. Zhou. 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models'. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 24824–24837.

[24] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan and Y. Cao. 'ReAct: Synergizing Reasoning and Acting in Language Models'. In: *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*. 2023.

[25] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda and T. Scialom. 'Toolformer: Language Models Can Teach Themselves to Use Tools'. In: 36 (2023), pp. 68539–68551.

[26] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel and D. Kiela. 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks'. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.

[27] Y. Qin, S. Hu, Y. Lin, W. Chen, N. Ding, G. Cui, Z. Zeng, Y. Huang, C. Xiao, C. Han, Y. R. Fung, Y. Su, H. Wang, C. Qian, R. Tian, K. Zhu, S. Liang, X. Shen, B. Xu, Z. Zhang, Y. Cong, Y. Zhang, L. Hou, J. Su, M. Sun and B. Zhou. 'Tool Learning with Foundation Models'. In: *ACM Computing Surveys* 56.7 (2024), pp. 1–40. DOI: 10.1145/3656580.

[28] D. L. Marino, C. S. Wickramasinghe and M. Manic. 'An Adversarial Approach for Explainable AI in Intrusion Detection Systems'. In: *Proceedings of the 44th Annual Conference of the IEEE Industrial Electronics Society (IECON 2018)*. 2018, pp. 3225–3231. DOI: 10.1109/IECON.2018.8591457.

[29] I. Lage, E. Chen, J. He, M. Narayanan, B. Kim, S. J. Gershman and F. Doshi-Velez. 'Human Evaluation of Models Built for Interpretability'. In: *Proceedings of the AAAI Workshop on Network Interpretability for Deep Learning*. 2019, pp. 1–5.

[30] M. Chromik and A. Butz. 'Human-XAI Interaction: A Review and Design Principles for Explanation User Interfaces'. In: *Human-Computer Interaction – INTERACT 2021 (IFIP TC 13)*. Vol. 12936. Lecture Notes in Computer Science. Springer, 2021, pp. 619–640. DOI: 10.1007/978-3-030-85616-8_36.

[31] S. Kalafatidis, N. Papageorgopoulos, A. Kartakoullis and G. Ledakis. 'LLM-Enhanced Intrusion Detection for Containerized Applications: A Two-Tier Strategy for SDN and Kubernetes Environments'. In: *Lecture Notes in Computer Science (ARES 2025 EU Projects Symposium Workshops)*. Vol. 15998. Springer, 2025, pp. 55–73. DOI: 10.1007/978-3-032-00642-4_4.

[32] M. Sarhan, S. Layeghy, N. Moustafa and M. Portmann. 'NetFlow Datasets for Machine Learning-Based Network Intrusion Detection Systems'. In: (2021), pp. 117–135. DOI: 10.1007/978-3-030-86534-4_9.

[33] C. R. Sunstein. 'The Law of Group Polarization'. In: *Journal of Political Philosophy* 10.2 (2002), pp. 175–195. DOI: 10.1111/1467-9760.00148.

[34] L. Breiman. 'Random Forests'. In: *Machine Learning* 45.1 (2001), pp. 5–32. DOI: 10.1023/A:1010933404324.

[35] MITRE Corporation. *MITRE ATT&CK: Adversary Tactics, Techniques, and Common Knowledge*. 2023. URL: https://attack.mitre.org (visited on 01/03/2026).

[36] I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani. 'Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization'. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018)*. Funchal, Madeira, Portugal: SciTePress, 2018, pp. 108–116. DOI: 10.5220/0006639801080116.

[37] A. Kenyon, L. Deka and D. Bhowmik. 'Are Public Intrusion Datasets Fit for Purpose: Characterising the State of the Art in Intrusion Event Datasets'. In: *Computers & Security* 99 (2020), p. 102022. DOI: 10.1016/j.cose.2020.102022.

[38] M. Ring, S. Wunderlich, D. Scheuring, D. Landes and A. Hotho. 'A Survey of Network-Based Intrusion Detection Data Sets'. In: *Computers & Security* 86 (2019), pp. 147–167. DOI: 10.1016/j.cose.2019.06.005.

[39] G. C. Cawley and N. L. C. Talbot. 'On Over-Fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation'. In: *Journal of Machine Learning Research* 11 (2010), pp. 2079–2107.

[40] R. Sommer and V. Paxson. 'Outside the Closed World: On Using Machine Learning for Network Intrusion Detection'. In: *Proceedings of the 2010 IEEE Symposium on Security and Privacy*. Oakland, CA: IEEE, 2010, pp. 305–316. DOI: 10.1109/SP.2010.25.

[41] M. F. A. Razak, N. B. Anuar, R. Salleh and A. Firdaus. 'The Rise of "Malware": Bibliometric Analysis of Malware Study'. In: *Journal of Network and Computer Applications* 75 (2017), pp. 58–76. DOI: 10.1016/j.jnca.2016.08.022.

[42] S. Mukherjee, S. Saha and C. Bhatt. 'Threat Intelligence Integration in Network Intrusion Detection: A Review of Tools and Methodologies'. In: *International Journal of Cybersecurity Intelligence and Cybercrime* 6.2 (2023), pp. 14–31.

[43] A. Maloyan and D. Namiot. 'Breaking the Protocol: Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents'. In: *arXiv preprint arXiv:2601.17549* (2025). arXiv: 2601.17549 [cs.CR].